WNDRVR
SUPPORT NETWORK

VXWORKS 7 SECURITY PROGRAMMER'S GUIDE, SR0650 EDITION 1

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Security Programmer's Guide, SR0650 Edition 1

14 August 2020

1. VXWORKS SECURITY OVERVIEW

The VxWorks® 7 Security Programmer's Guide describes VxWorks software-based security features that protect connected devices at every stage of their operation, from boot-up to power-down.

Secure Boot

Secure boot for various platforms to verify binaries at every stage of the boot-up process.

Secure ELF Loader

Secure ELF loaders with a public certificate load code modules that have been signed with the corresponding private key.

Advanced User Management

Protects devices from unauthorized access and enables the definition and enforcement of user-based policies and permissions.

Encrypted Containers

Supports TrueCrypt-compatible AES-encrypted file containers using the loopFs file system and the VxWorks version of tcplay.

Full Disk Encryption

Provides "data at rest" protection for sector-addressable devices (like hard disks), using the XEX-AES algorithm.

Trusted Computing

Provides hardware-based control over what software runs on a system.

Security Events Facility

Generates security event-based warning messages based on conditions that indicate a potential security risk.

Trusted Execution Environment with OP-TEE

Provides a secure parallel execution environment that isolated from VxWorks by the ARM TrustZone security hardware.

VxWorks Configuration for Achilles Level II Certification

Achilles certification demonstrates the robustness of a system from a network point of view.

GDOI

Supports both a client and a statically-configured server, using the Group Domain of Interpretation (GDOI) cryptographic protocol for group key management.

SCEP

Manages digital certificates using the Simple Certificate Enrollment Protocol (SCEP). For VxWorks, only the SCEP client is implemented.

Network Security

Incorporates the latest Wind River security protocols, such as Wind River SSL, SSH, IPsec, and IKE to effectively secure network communications.

For information about networks security features, see the following guides:

- VxWorks 7 IKE Programmer's Guide
- VxWorks 7 IPsec Programmer's Guide
- VxWorks 7 EAP Programmer's Guide
- VxWorks 7 SSH Programmer's Guide
- VxWorks 7 SSL Programmer's Guide

WNDRVR

- VxWorks 7 Radius and Diameter Programmer's Guide
- VxWorks 7 Firewall Programmer's Guide
- VxWorks 7 NAT Programmer's Guide
- VxWorks 7 Cryptography Libraries Programmer's Guide

2. SECURE BOOT WITH UEFI

1. Secure Boot with UEFI

A secure boot process can be implemented for VxWorks® with the proprietary Wind River VxWorks UEFI boot loader, on Intel boards, by using a VxWorks image and a UEFI boot loader image that have been digitally signed. Optionally, a system can also be configured to use an encrypted VxWorks image and application images (that are also signed).

The UEFI firmware is configured to validate the UEFI boot loader, the UEFI boot loader is signed for validation by the firmware and configured to validate the VxWorks image, and the VxWorks image is signed so that it can be validated and booted by the UEFI boot loader.

The VxWorks UEFI boot loader ships in source code form. It is a standalone application, and does not include a VxWorks image.

OpenSSL is used to create and encode (Base64) the signature.

Note: Secure boot with UEFI is supported for 64-bit (and not for 32-bit).

2. Creating a UEFI Boot Loader and VxWorks Image for Secure Boot

A high level overview of the process is as follows:

1. Configure your VSB with support for the UEFI boot loader.
2. Optionally add your own key and certificate files (instead of those produced by the build).
3. Optionally configure the project with support for loading a VxWorks image and application objects that are both encrypted and signed.
4. Build the VSB.
5. Configure your VIP with support for the UEFI boot loader.
6. Optionally configure the project with support for loading a VxWorks image that is encrypted as well as signed.
7. Build the VIP.
8. Copy the key and database files, the signed UEFI boot loader, and signed VxWorks image to a USB drive.
9. Configure the UEFI firmware.
10. Boot the system with the signed UEFI boot loader and the signed VxWorks image.
11. To boot an encrypted VxWorks image, copy the KEP to the VxWorks secrets repository, copy the encrypted VxWorks image to the boot media, and reboot.

    VSB Configuration and Build on page 3 Configure your VSB with support for the UEFI boot loader. Optionally add your own key and certificate files (instead of those produced by the build). Optionally configure the project with support for loading a VxWorks image and and application objects that are both encrypted and signed.
    VIP Configuration and Build on page 5 Configure your VIP with support for the UEFI boot loader. Optionally configure the project with support for encrypting a VxWorks image (as well as signing it) and loading it.

2.1. VSB Configuration and Build

Configure your VSB with support for the UEFI boot loader. Optionally add your own key and certificate files (instead of those produced by the build). Optionally configure the project with support for loading a VxWorks image and and application objects that are both encrypted and signed.

About This Task

Procedure

1. Create a VSB project based on the itl_generic BSP.
2. Add the **SECURE_LOADER** layer.
3. Optionally, use your own key and certificate files instead of those produced by the build.

   As a convenience for development, the build automatically produces signed images. If you want to use the your own key and certificate files (and not the defaults produced by the build), change the configuration options to point to your files, or copy your files to the default location and file names. The build does not overwrite existing files. If the files specified by the configuration parameters do not exist, new ones are generated. The default output location is *vsbDir* **/secureLoader/**.

   SECURE_LOADER_PRIVATE_PK_FILE

   Defines a file that contains the private PK in Privacy Enhanced Mail (PEM) format. The private PK is the private key used for signing the Key Exchange Key (KEK) on the platform in secure mode.

   SECURE_LOADER_PUBLIC_PK_CERT_FILE

   Defines a file that contains the public PK certificate in Privacy Enhanced Mail (PEM) format. The public PK certificate is used to generate the PK EFI signature list file for validating the KEK on the platform in secure mode.

   SECURE_LOADER_PRIVATE_KEK_FILE

   Defines a file that contains the private KEK in Privacy Enhanced Mail (PEM) format. The private KEK is used for signing the signature database on the platform in secure mode.

   SECURE_LOADER_PUBLIC_KEK_CERT_FILE

   Defines a file that contains the public KEK certificate in Privacy Enhanced Mail (PEM) format. The public KEK certificate is used to generate the KEK EFI signature list for validating the signature database on the platform in secure mode.

   SECURE_LOADER_PRIVATE_SIGNING_KEY_FILE

   Defines a file that contains the private signing key in Privacy Enhanced Mail (PEM) format. The private signing key is used for signing and validating signed images on the platform in secure mode.

   SECURE_LOADER_PUBLIC_SIGNING_CERT_FILE

   Defines a file that contains one or more public signing certificates in Privacy Enhanced Mail (PEM) format. The public signing certificates are used to generate the DB EFI signature list for validating signed images on the platform in secure mode.

4. Optionally, to use encryption for VxWorks, kernel applications (DKMs), RTP applications, and shared libraries, select the secure loader encryption option and set the parameters for the KEP ID that will be used with the secrets repository and KEP itself.
   1. Select SECURE_LOADER_ENCRYPTION.

   This option provides for encrypting an image before it is signed, whether VxWorks itself, kernel applications (DKMs), RTP applications, or share libraries.

   2. Set SECURE_LOADER_KEP_ID to the ID string that will be used to retrieve the key encrypting password (KEP) from the VxWorks secrets repository (the **vault** directory). A maximum of 64 characters can be used for the string.
   3. Set SECURE_LOADER_KEP_SECRET to the KEP that is used to generate the key encrypting key (KEK) that used for encryption and decryption. A maximum of 256 characters can be used for the string.

   With regard to the length of the KEP, it should contain at least as many bits of the entropy as the key length of encryption algorithm that is used (in this case, typically 256 bits). Otherwise it is easier for an attacker to determine the KEP by brute-force than the encryption key. If only lower case characters are used, each character has approximately 4.5 bits of entropy: 256/4.5 = 57

(round up to 60 characters). If upper and lower, numbers and special characters are used, then each character has approx 6 bits of entropy: 256/6 = 43 characters.

5. Add any other layers or options required for your system.
6. Build the VSB.

   Included in the VSB build output are the required key and database files in *vsbDir* **/secureLoader**:
   - The PK EFI signature list file **PK.sig**.
   - The KEK EFI signature list file **KEK.sig**
   - The DB EFI signature list file **db.sig**.

Next Steps

VIP Configuration and Build on page 5.

Parent topic: Creating a UEFI Boot Loader and VxWorks Image for Secure Boot on page 3

2.2. VIP Configuration and Build

Configure your VIP with support for the UEFI boot loader. Optionally configure the project with support for encrypting a VxWorks image (as well as signing it) and loading it.

Before You Begin

VSB Configuration and Build on page 3

Procedure

1. Create a VIP based on your VSB.
2. Add the **INCLUDE_BOOT_LOADER** component.
3. Optionally, add support for using an encrypted VxWorks image and application binaries.
   1. Add **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**. This triggers a configuration error, which you resolve with the next step.
   2. Set the **SEC_VAULT_KEY_ENCRYPTING_PW** parameter to the password used to encrypt keys in the secrets repository and key store. The parameter value should be set to a random string of at least 40 characters containing both lower and upper case letters, numbers, and special characters. No default is provided by intention, to ensure that unique values are used. You must define a password, otherwise your VIP will not build.
   3. Set **SEC_VAULT_FILE_ROOT** to the directory that will hold the UEFI boot loader file (**BOOTX64.EFI**) and the VxWorks image (which is renamed to **bootapp.sys**). This directory must be named **/EFI/BOOT** on the boot drive. For example, "**/bd0:1/EFI/BOOT**". **SEC_VAULT_FILE_ROOT**also defines the top-level directory where the key store and vault directory structure (the sub-directories **keys** and **vault**) will be created when keys or passwords are imported.
   4. Add **INCLUDE_SEC_SECRET_CMD** for the secrets repository management command. You will use the command, after booting the system, from the kernel shell to add the key encrypting password to the secrets vault. Adding this component automatically adds the shell itself.
4. Build the VIP.

   In addition to unsigned images, the build output includes signed images, and optionally images that are both encrypted and signed:

   - The signed UEFI boot loader image is *vipDir* **/loader/obj/uefi_x86_64/BOOTX64.EFI.signed.**
   - Signed and signed-encrypted VxWorks images are built in *vipDir* **/default**. The files are **vxWorks.signed** and **vxWorks.encrypted**.

WNDRVR

- If you have configured VxWorks without **INCLUDE_STANDALONE_SYM_TBL** and without encryption, the build also produces **vxWorks.sym.signed** in *vipDir* **/default**.
  If your configuration has either **INCLUDE_STANDALONE_SYM_TBL** or encryption (which automatically adds **INCLUDE_STANDALONE_SYM_TBL**), a separate symbol table file is not produced. The symbol table is included in the VxWorks image.

The signed versions of the images are created as a convenience for development. The images are signed using the private signing key from the file identified in the VSB configuration with **SECURE_LOADER_PRIVATE_SIGNING_KEY_FILE**, and the first public signing certificate from the file identified in the VSB configuration with SECURE_LOADER_PUBLIC_SIGNING_CERT_FILE.

Next Steps

Copying Key and Database Files, UEFI Boot Loader, and VxWorks Image to FAT32 USB Flash Drive on page 6.

CAUTION: **For production systems, you can use the images that were automatically signed (and optionally encrypted) as part of the build, or sign the unsigned images using signtool (see** signtool Reference on page 60**). It is, however, critical that the key used for signing, and the password used to generate the encryption and decryption keys, are stored and managed in a secure way. The production images must be signed (and optionally encrypted) on a dedicated host with appropriate security procedures.**

Parent topic: Creating a UEFI Boot Loader and VxWorks Image for Secure Boot on page 3

3. Copying Key and Database Files, UEFI Boot Loader, and VxWorks Image to FAT32 USB Flash Drive

Copy the key and database files, the boot loader, and VxWorks to a USB drive. You will use the drive with these files to re-configure your target firmware and boot the system.

About This Task

Note: You must use this procedure first even if you are going to use encrypted-signed images with your system. You must first boot the system with signed binaries so that you can then add the KEP ID and KEP secret to the system. Then you can boot the encrypted VxWorks image.

Before You Begin

- VSB Configuration and Build on page 3
- VIP Configuration and Build on page 5

Procedure

1. Copy the **PK.sig**, **KEK.sig**, and **db.sig** files to the root directory of the USB drive.
2. Copy **BOOTX64.EFI.signed** to **/EFI/BOOT/BOOTX64.EFI** on the drive.

WNDRVR

The UEFI firmware requires that **BOOTX64.EFI** be in the **EFI/BOOT** directory of the boot drive, which is **/bd0:1** in the case of a USB drive. The **BOOTX64.EFI** loader will look for both the secrets repository and the VxWorks image (renamed **bootapp.sys**, below) in the same directory.

3. Copy **vxWorks.signed** to **/EFI/BOOT/bootapp.sys** on the drive.
4. If your VIP build also produces a separate symbol table file (**vxWorks.sym.signed**) copy it to **/EFI/BOOT/bootapp.sys** as well.

   Your build will not include a separate symbol table if your configuration has either **INCLUDE_STANDALONE_SYM_TBL** or encryption support (which automatically adds **INCLUDE_STANDALONE_SYM_TBL**). The symbol table is included in the VxWorks image.

Next Steps

- Configuring Firmware and Booting VxWorks on page 7.
- And if you are going to use encrypted images, Booting with an Encrypted VxWorks Image on page 11.

4. Configuring Firmware and Booting VxWorks

The VxWorks secure boot feature is supported for all Intel targets with UEFI secure boot. The following examples illustrate firmware configuration and booting VxWorks for several boards.

After the UEFI firmware is configured to validate the UEFI boot loader, the system can be booted with the signed UEFI loader and signed VxWorks image.

> Note: You must first boot your system with a signed VxWorks image even if you are going to use an encrypted image with your system. This is because you must first boot the system with signed binaries so that you can then add the KEP ID and KEP secret to the system. Then you can boot the encrypted VxWorks image (as described in Booting with an Encrypted VxWorks Image on page 11).

Configure the SkyLake NUC Firmware and Boot VxWorks with the UEFI Boot Loader on page 7 Configure the firmware with the required PK, KEK and DB EFI signature lists, and boot the system from a USB flash drive.
Configure the SkyLake GIGABYTE GA-H170N-WIFI Firmware and Boot VxWorks with the UEFI Boot Loader on page 8 Configure the firmware with the required PK, KEK and db EFI signature lists, and boot the system from a USB flash drive.
Configure the CamelBack Mountain Firmware and Boot VxWorks with the UEFI Boot Loader on page 9 Configure the firmware with the required PK, KEK and db EFI signature lists, and boot the system from a USB flash drive.

4.1. Configure the SkyLake NUC Firmware and Boot VxWorks with the UEFI Boot Loader

Configure the firmware with the required PK, KEK and DB EFI signature lists, and boot the system from a USB flash drive.

Before You Begin

Copying Key and Database Files, UEFI Boot Loader, and VxWorks Image to FAT32 USB Flash Drive on page 6.

Procedure

1. Insert the USB drive into the target board and power it on.

WNDRVR

2. Press **F2** to go to the BIOS interface.
3. Go to the Secure Boot dialog, and then:
    1. Select **Secure Boot**.
    2. Select **Clear Secure Boot Data**.
    3. Press F10 to save and exit.

   This clears the PK, KEK, and DB variables from the UEFI firmware so you can reset them with your own in a later step.

4. After the target has rebooted, press **F2** to go to the BIOS interface.
5. Go to the Secure Boot dialog, disable **Secure Boot**, and save the changes.

   You must disable secure boot in order to boot the VxWorks UEFI loader, because the PK, KEK, and DB variables are not currently set in the UEFI firmware.

6. Reset the board, and once the UEFI loader has started, press any key to stop the process.
7. Press **M** to enter the boot manager.
8. From the UEFI boot manager options list:
    1. Press **7** to load your DB EFI signature list, then press any key to return to the option list.
    2. Press **6** to load your KEK EFI signature list file, then press any key to return to the option list.
    3. Press **5** to load your PK EFI signature list file, then press any key to return to the option list.
    4. Press **4** to delete the boot path.
    5. Press any key to return to the option list.
9. Press **2** to reset the configuration.
10. Press **F2** go to the BIOS interface.
11. Go to the Secure Boot Config dialog, and then select **Secure Boot** and save the change.

    You have reconfigured the firmware, selected the UEFI secure boot option, and are ready to perform a secure boot with your PK, KEK, and DB variables.

    CAUTION: **If you want to use PXE for booting your board, go to** Using UEFI PXE Boot on Intel SkyLake NUC on page 11**. Do not reset the board at this point.**

12. Reset the board.

    The UEFI firmware loads and starts the signed UEFI boot loader. Then the UEFI boot loader loads and starts the signed VxWorks image.

Parent topic: Configuring Firmware and Booting VxWorks on page 7

4.2. Configure the SkyLake GIGABYTE GA-H170N-WIFI Firmware and Boot VxWorks with the UEFI Boot Loader

Configure the firmware with the required PK, KEK and db EFI signature lists, and boot the system from a USB flash drive.

Before You Begin

Copying Key and Database Files, UEFI Boot Loader, and VxWorks Image to FAT32 USB Flash Drive on page 6.

Procedure

1. Insert the USB drive into the target board and power it on.

2. Press **DEL** to go to the BIOS interface.
3. Select **BIOS Features**, and then:
    1. Set **Secure Boot** to Enabled.
    2. Change **Secure Boot Mode** to Custom.
    3. Select **Key Management**.
4. Select **Delete All Secure Boot Variables**.

   This clears the PK, KEK, and DB variables from the UEFI firmware so you can reset them with your own in a later step.

5. Select **BIOS Features**.
6. Set **Secure Boot** to Disabled.

   You must disable secure boot in order to boot the VxWorks UEFI loader, because the PK, KEK, and DB variables are not currently set in the UEFI firmware.

7. Save the changes and exit (**F10**).
8. Reset the board, and press any key to stop the VxWorks UEFI loader boot process.
9. Press **M** to enter the boot manager.
10. From the UEFI boot manager options list:
    1. Press **7** to load your DB EFI signature list, then press any key to return to the option list.
    2. Press **6** to load your KEK EFI signature list file, then press any key to return to the option list.
    3. Press **5** to load your PK EFI signature list file.
    4. Press any key to return to the option list.
    5. Press **4** to delete the boot path.
11. Press any key to return to the UEFI boot loader.
12. Press **2** to reset the configuration.
13. Press **DEL** go to the BIOS interface.
14. Select **BIOS Features**.
    1. Set **Secure Boot** to Enabled.
    2. Save the changes and exit (**F10**).

    You have reconfigured the firmware, selected the UEFI secure boot option, and are ready to perform a secure boot with your PK, KEK, and DB variables.

15. Reset the board.

    The UEFI firmware loads and starts the signed UEFI boot loader. Then the UEFI boot loader loads and starts the signed VxWorks image.

Parent topic: Configuring Firmware and Booting VxWorks on page 7

4.3. Configure the CamelBack Mountain Firmware and Boot VxWorks with the UEFI Boot Loader

Configure the firmware with the required PK, KEK and db EFI signature lists, and boot the system from a USB flash drive.

Before You Begin

Copying Key and Database Files, UEFI Boot Loader, and VxWorks Image to FAT32 USB Flash Drive on page 6.

WNDRVR

Procedure

1. Insert the USB drive into the target board and power it on.
2. Press **F2** to go to the BIOS interface.
3. Select **EDKII Menu**.
4. From the EDIII menu, select **Secure Boot Configuration**.
5. From the Secure Boot Configuration menu, clear **Attempt Secure Boot**.

   You must disable secure boot in order to boot the VxWorks UEFI loader, because you will clear the PK, KEK, and DB variables in the UEFI firmware.

6. Set **Secure Boot Mode** to Custom Mode.
7. Select **Custom Secure Boot Options**.
8. From the Custom Secure Boot Options menu, select **PK Options**.
9. From the PK Options menu, select **Delete PK**.
10. From the Custom Secure Boot Options menu, select **KEK Options**.
11. From the KEK Options menu, select **Delete KEK**.
12. Select **DB Options**.
13. From the DB Options menu, select **Delete Signature**.
14. Save (**F10**).

    This clears the PK, KEK, and DB variables from the UEFI firmware so you can reset them with your own in a later step.

15. Reset the board.
16. Press any key to pause the boot process.
17. Press **M** to enter the boot manager.
18. From the UEFI boot manager options list:
    1. Press **7** to load your DB EFI signature list, then press any key to return to the option list.
    2. Press **6** to load your KEK EFI signature list file, then press any key to return to the option list.
    3. Press **5** to load your PK EFI signature list file, then press any key to return to the option list.
    4. Press **4** to delete the boot path, then press any key to return to the option list.
    5. Press any key to return to UEFI boot manager.
19. Press **2** to reset the configuration.
20. Press **F2** to go to the BIOS interface.
21. Select **EDKII Menu**.
22. From the EDKII menu, select **Secure Boot Configuration**.
23. From the Secure Boot Configuration menu, select **Attempt Secure Boot**, and save the change (**F10**).
24. Select **Boot Maintenance Manager**.
25. From the Boot Maintenance Manager menu, select **Boot Options**.
26. From the Boot Options menu, select **Change Boot Order**.
27. From the Change Boot Order menu, move UEFI USB to the top of the list.
28.

    You have reconfigured the firmware, selected the UEFI secure boot option, and are ready to perform a secure boot with your PK, KEK, and DB variables.

29. Reset the board.

    The UEFI firmware loads and starts the signed UEFI boot loader. Then the UEFI boot loader loads and starts the signed VxWorks image.

Parent topic: Configuring Firmware and Booting VxWorks on page 7

5. Using UEFI PXE Boot on Intel SkyLake NUC

Copy the signed UEFI loader and the signed VxWorks image to the TFTP directory of the PXE server. Configure the BIOS for secure boot with PXE.

About This Task

Note: PXE boot should not be used in production systems. It is described here as a convenience as a convenience for the development process.

Before You Begin

- Perform the steps described in Copying Key and Database Files, UEFI Boot Loader, and VxWorks Image to FAT32 USB Flash Drive on page 6, except for copying the UEFI boot loader and VxWorks to the USB drive. They are not needed on the drive.
- Perform the steps described in Configure the SkyLake NUC Firmware and Boot VxWorks with the UEFI Boot Loader on page 7, but do not reset the board. Further configuration is required.

Procedure

1. From the Skylake NUC BIOS interface, Boot Order dialog, select the UEFI tab and enable UEFI boot.
2. In the Boot Order dialog, select the Legacy tab, and disable Legacy boot.
3. From the Boot Configuration dialog, change Network Boot to UEFI PXE.
4. Save the changes (**F10**).
5. Copy the signed UEFI loader to the TFTP directory of the PXE server.
6. Copy the signed VxWorks image to the TFTP directory of the PXE server.
7. Reset the board.

   The UEFI firmware secure boot facility downloads the UEFI loader from the PXE server, validates it, and then loads and starts it. Then, the UEFI loader downloads the VxWorks image from the PXE server, validates it, and then loads and starts it.

6. Booting with an Encrypted VxWorks Image

To boot an encrypted VxWorks image, you must first boot the system with a signed (un-encrypted) VxWorks image, create the secrets repository with the KEP in it, and copy the encrypted VxWorks image to the boot media. Then reboot the system with an encrypted image.

About This Task

The reason that you must first boot with an un-encrypted image is that you need to create the secrets repository with the KEP in it before you can boot an encrypted image.

WNDRVR

Before You Begin

1. VSB Configuration and Build on page 3
2. VIP Configuration and Build on page 5
3. Copying Key and Database Files, UEFI Boot Loader, and VxWorks Image to FAT32 USB Flash Drive on page 6
4. Configuring Firmware and Booting VxWorks on page 7

About This Task

Note: The same UEFI boot loader can be used for both encrypted and unencrypted VxWorks images.

Procedure

1. Boot VxWorks using the signed bootloader and signed VxWorks image.
2. From the kernel shell, add the KEP to the secrets repository.

   The ID and the secret string parameters for the secret import command must match the ones defined with the VSB variables **SECURE_LOADER_KEP_ID** and **SECURE_LOADER_KEP_SECRET**. For example, using "kepID" and "kepSecret" (respectively):

   ```
   -> cmd secret import "kepId" "kepSecret"
   ```

   The KEP is stored in the **/vault** subdirectory of the directory defined by the VIP **SEC_VAULT_FILE_ROOT** parameter. So, for example, if it is set to "/bd0:1/EFI/BOOT", you can verify that the KEP has been added by displaying the ID as follows:

   ```
   -> cd "/bd0:1/EFI/BOOT/vault"
   value = 0 = 0x0
   -> ll
   drwxrwxrwx 1 0 0 4096 Jan 1 1980 ./
   drwxrwxrwx 1 0 0 4096 Jan 1 1980 ../
   -rwxrwxrwx 1 0 0 16 Jan 1 1980 kepId
   value = 0 = 0x0
   ```

   For information about the secrets repository, see Key Store and Secrets Repository on page 48.

   Optionally, you can remove the shell from your VSB configuration after you have added the KEP to the secrets repository.

3. Copy the encrypted VxWorks image to the USB drive.

   Copy the image to the **/EFI/BOOT** directory on the boot drive, and rename it from **vxWorks.encrypted** to **bootapp.sys**.

   For example:

   ```
   cp /home/xeno/workspace/vip_SKYLAKE_64_smp/default/vxWorks.encrypted /bd0:1/EFI/BOOT/bootapp.sy
   s
   ```

4. Reboot the system.

Next Steps

- For production systems, after performing this procedure on one target device, you can just copy the UEFI vault files to other devices.

3. SECURE BOOT FOR NXP T2080 AND P4080

1. Secure Boot

VxWorks supports secure boot for the NXP T2080 and P4080.

To implement secure boot for VxWorks with the NXP T2080 or P4080 you will use the NXP Linux SDK for QorIQ Processors, VxWorks 7, and additional software provided by Wind River. This document illustrates implementation of secure boot for the T2080QDS and P4080DS.

In addition to this document, please consult the VxWorks 7 documentation set for VxWorks development and Workbench on-chip debugging. Also see the NXP documentation for the SDK and for the T2080 or P4080, including the processor reference manual and security reference manual (which are available from from http://www.nxp.com).

2. Required Third-Party Software

Implementation of secure boot requires the use of a Linux host, and the NXP SDK for QorIQ processors.

- Linux host system.
- The NXP Linux SDK for QorIQ Processors, which is available from http://www.nxp.com.

3. Install and Build NXP SDK

Implementing secure boot for the T2080 or P4080 first requires using the NXP Linux SDK for QorIQ Processors to produce the U-Boot image that is used with your secure boot system.

The following list provides an overview of the steps that you must perform. After you have downloaded and the SDK and mounted the ISO file, see the NXP SDK documentation for detailed instructions.

Note: Ensure you choose a recent u-boot version that does not contain any known vulnerabilities.

1. Download the SDK.

   The NXP Linux SDK for QorIQ Processors is available from http://www.nxp.com Make sure to:
   - Select the version of the SDK that is appropriate for your hardware. For example, if your device version is 1.0, use the SDK version 1.8.
   - Select the source version of the ISO file. For example, **QoriQ Linux SDK v2.0 SOURCE.iso** or **QorIQ-SDK-v1.8-SOURCE-20150619yocto.iso**.

2. Install the SDK.
3. Prepare the host environment.
   This involves installing the Linux packages that are required for Yocto.
4. Set up Poky.
   This involves configuring the Poky build system for your hardware platform.

5. Build the SDK.
   The output of the build goes to *sdkInstallDir* **/build**_*boardName* **/tmp/deploy/images/***boardName* . For example, in **/home/xeno/QorIQ-SDK-V2.0-20160527-yocto/build_t2080qds/tmp/deploy/images/t2080qds/**.
   The output includes the U-Boot binary that will be used to create your secure boot system.

4. Generate Keys and Sign Images

Use NXP SDK tools and a Wind River script to generate the required keys and binaries, and to sign the images.

Before You Begin

Install and Build NXP SDK on page 14.

Create a **uVxWorks** image and DTB. For information in this regard, see the VxWorks 7 documentation set.

About This Task

Note: The examples used in these instructions are for a T2080QDS platform. The process is similar for the P4080DS (pathnames and addresses differ).

Procedure

1. Copy (and rename) the U-Boot image to the NXP code signing tool (**cst/**) directory.

   The U-boot image is produced by the SDK build in the SDK **images** directory with an extended, descriptive file name that needs to be changed to **u-boot.bin** when it is copied.

   For example, from:

   /home/xeno/QorIQ-SDK-V2.0-20160527-yocto/build_t2080qds/tmp/deploy/images/t2080qds/u-boot-secure-boot-2016.01+fslgit-r0.bin

   to:

   /home/xeno/QorIQ-SDK-V2.0-20160527- yocto/build_t2080qds/tmp/sysroots/x86_64-linux/usr/bin/cst/u-boot.bin.

2. Copy the **uVxWorks** image and DTB from your VxWorks 7 workspace to the code signing tool directory.

   For example, from:

   /home/xeno/workspace/t2080-VIP/default/

   to:

   /home/xeno/QorIQ-SDK-V2.0-20160527- yocto/build_t2080qds/tmp/sysroots/x86_64-linux/usr/bin/cst/

3. Optionally, create a boot script.
   1. Create the boot script text file.

      In the code signing directory (**cst/**), to which you have copied your files, create **bootscript.txt** with this contents for the T2080QDS:

```
esbc_validate 0xE9000000
esbc_validate 0xE9100000
bootm 0xE8020000 - 0xE8800000
```

The addresses are for the flash boot addresses or CSF header. The addresses for each board type are provided in the NXP documentation.

2. Generate the boot script binary.

Use the mkimage command, as follows:

```
% ../mkimage -T script -C none -d bootscript.txt bootscript
```

4. Generate keys and sign the images.

You can use the **secureboot.sh** script (code provided by Wind River), or follow the manual steps described in the NXP SDK documentation. If you follow the manual steps, you should consult the **secureboot.sh** code to see what image file name changes are required for VxWorks (see Code for secureboot.sh on page 24).

To use the script, create a **secureboot.sh** file in the code signing directory (**cst/**) with the code from Code for secureboot.sh on page 24. Then execute the script and select the key length. For example:

```
% sudo ./secureboot.sh

Please enter key length(1024/2048/4096):2048

generate 2048 key pair...
generate One time programmable master key...
t2080qds sign successfully.
```

Any of the three options for key length can be used.

The script signs the images and produces the key pair:

- **bootscript** – boot script binary file
- **gen_otpmk.out** – one time programmable master key (OTPMK) binary file
- **hdr_bs.out** – boot script header binary file
- **hdr_dtb.out** – DTB header binary file
- **hdr_uboot.out** – the CSF (Command Sequence File) header file binary.
- **hdr_vxworks.out** – VxWorks header binary file
- **SRK_output.out** – record for public key hash
- **srk.pri** – RSA 2048-bit private key used for signing procedure
- **srk.pub** – RSA 2048-bit public key used for signing procedure

The public hash key is stored in **SRK_output.out** and the OTPMK is stored in **gen_otpmk.out**.

Note: If you want to regenerate the output you must first remove **srk.pub**, **srk.pri**, **SRK_output.out** and **gen_otpmk.out**.

5. Program Fuses for the T2080

Program the super root key hash (SRKH) and one time programmable master key (OTPMK) fuses.

Before You Begin

Generate Keys and Sign Images on page 15

Procedure

1. Set up the T2080QDS board for fuse NOR flash programming.

   Turn off the power and set the DIP switches:

Switch	1	2	3	4	5	6	7	8
SW1	OFF	OFF	OFF	ON	OFF	OFF	ON	ON
SW2	ON	ON	ON	ON	ON	ON	ON	OFF
SW3	OFF	OFF	ON	OFF	ON	ON	ON	OFF
SW4	ON	OFF	ON	ON	ON	ON	ON	ON
SW5	ON	OFF	OFF	OFF	ON	ON	ON	ON
SW6	OFF	ON	OFF	OFF	ON	ON	ON	ON
SW7	OFF	ON	ON	OFF	OFF	ON	ON	OFF
SW8	OFF	OFF	OFF	ON	OFF	ON	ON	OFF
SW9	ON	ON	ON	OFF	OFF	OFF	ON	ON
SW10	ON	ON	ON	OFF	ON	OFF	OFF	OFF
SW11	ON	OFF	ON	ON	ON	ON	OFF	OFF
SW12	ON	ON	ON	ON	ON	ON	ON	ON

2. Configure the board to boot U-Boot from vbank 4.

   Set SW6[1:4] to 0b0100.

3. Enable POVDD at 1.8V.

   Set SW10[8] to 0b1.

4. Enable PROG SFP.

Enable J27.

5. Power on the board and press the ENTER key to stop at the U-Boot countdown prompt.
6. Program the SRKH fuses.

   Write the RSA public key hash to the CCSR SRKHR[0:7] registers. The CCSR SRKHR[0:7] registers are located at CCSR offset E_8000h base + 23Ch (The CCSR base address is 0xFE00_0000). The following example uses a key hash from **SRK_output.out**: 034d229fc108433e5e2da56d1842a39eb42bf90add643f7a7219e546698c67a7 (which is generated as described in Generate Keys and Sign Images on page 15; and will be different with each generation).

```
=> mm 0xfe0e823c

fe0e823c: 00000000 ? e814394d
fe0e8240: 00000000 ? eb4b3c5e
fe0e8244: 00000000 ? a74d8688
fe0e8248: 00000000 ? 0c92fa19
fe0e824c: 00000000 ? 58173dfa
fe0e8250: 00000000 ? 67a8f87b
fe0e8254: 00000000 ? 89750515
fe0e8258: 00000000 ? 34487261
```

7. Program the OTPMK fuses.

   Write the OTPMK values to the CCSR OTPMKR[0:7] registers. The CCSR OTPMKR[0:7] registers are located at CCSR offset E_8000h base + 21Ch.

```
=> mm 0xfe0e821c
fe0e821c: 00000000 ? 11111111
fe0e8220: 00000000 ? 22222222
fe0e8224: 00000000 ? 33333333
fe0e8228: 00000000 ? 44444444
fe0e822c: 00000000 ? 55555555
fe0e8230: 00000000 ? 66666666
fe0e8234: 00000000 ? 77777777
fe0e8238: 00000000 ? 88888888
```

8. Review the register programming, and correct it if necessary.

   Use the md command to verify that you have set the correct values for SRKHR[0:7] and OTPMKR[0:7].

   If they are incorrect, reset the board and reprogram the fuses.

9. Commit the register values to the fuse array (burn the fuses).

   CAUTION: **Once the SRKHR and OTPM fuses are burned, they are longer be programmable.**

   Use the mm command with the CCSR INGR register address (CCSR offset 0x000E_8020).

```
=> mm fe0e8020

fe0e8020: 00000000 ? 00000002
fe0e8024: 00000000 ? .
```

   Note: After writing to the fuse, be sure to wait at least 20 milliseconds for the fuse to program before cycling power.

10. Validate fuse programming.

    Disable POVDD with SW10[8] = 0b0.

    Use the md command to review the CCSR SRKHR[0:7] registers and ensure that the SRKH was properly read from the fuse array.

    Also review the CCSR HPSR register to ensure that the OTPMK is populated with a nonzero value with no parity errors.

    Power up the T2080QDS and hit the ENTER key to stop at the U-Boot prompt.

    Check if the SRKH was burned correctly. For example:

```
=> md fe0e821c
fe0e821c: ffffffff ffffffff ffffffff ffffffff
fe0e822c: ffffffff ffffffff ffffffff ffffffff
```

    Check the SM_HP status register. The value 09 means that all the keys were burned correctly. It is in the check (secure boot) state.

```
=> md fe314014

fe314014: 80000900 00000000 00000000 00000000
```

6. Program Fuses for the P4080

Program the super root key hash (SRKH) and one time programmable master key (OTPMK) fuses.

Before You Begin

Generate Keys and Sign Images on page 15

Procedure

1. Set up the P4080DS board for fuse NOR flash programming.

   Turn off the power and set the DIP switches:

Switch	1	2	3	4	5	6	7	8
SW1	OFF	ON	ON	OFF	ON	OFF	ON	ON
SW2	ON	ON	ON	ON	ON	ON	ON	ON
SW3	OFF	OFF	OFF	OFF	ON	ON	OFF	OFF
SW4	ON	ON	ON	ON	ON	ON	ON	OFF
SW5	OFF	OFF	OFF	OFF	OFF	OFF	OFF	OFF
SW6	OFF	OFF	OFF	OFF	OFF	OFF	ON	ON

Switch	1	2	3	4	5	6	7	8
SW7	OFF	ON	OFF	OFF	ON	ON	ON	ON
SW8	ON	OFF	ON	ON	OFF	OFF	ON	OFF
SW9	ON	ON	ON	OFF	OFF	ON	OFF	OFF
SW10	OFF	OFF	OFF	OFF	OFF	OFF	ON	ON

2. Configure the board to boot U-Boot from vbank 0.

   Set SW6[1:4] to 0b0000.

3. Set POVDD to 1.5V.

   Set SW8[6] to 0b0.

4. Enable POVDD.

   Set SW6[8] to 0b1.

5. Power on the board and press the ENTER key to stop at the U-Boot countdown prompt.
6. Program the SRKH fuses.

   Write the RSA public key hash to the CCSR SRKHR[0:7] registers. The CCSR SRKHR[0:7] registers are located at CCSR offset 0x000E_807C (The CCSR base address is 0xFE00_0000). The following example uses a key hash from **SRK_output.out**: 034d229fc108433e5e2da56d1842a39eb42bf90add643f7a7219e546698c67a7 (which is generated as described in Generate Keys and Sign Images on page 15; and will be different with each generation).

```
=> mm fe0e807c
fe0e807c: 00000000 ? 034d229f
fe0e8080: 00000000 ? c108433e
fe0e8084: 00000000 ? 5e2da56d
fe0e8088: 00000000 ? 1842a39e
fe0e808c: 00000000 ? b42bf90a
fe0e8090: 00000000 ? dd643f7a
fe0e8094: 00000000 ? 7219e546
fe0e8098: 00000000 ? 698c67a7
fe0e809c: 00000000 ? .
```

7. Program the OTPMK fuses.

   Write the OTPMK values to the CCSR OTPMKR[0:7] registers. The CCSR OTPMKR[0:7] registers are located at CCSR offset 0x000E_805C. For example, given the following OTPMK values from **gen_otpmk.out**:

   88888888777777776666666655555555444444443333333322222222211111111.

```
=> mm fe0e805c
fe0e805c: 00000000 ? 88888888
fe0e8060: 00000000 ? 77777777
fe0e8064: 00000000 ? 66666666
```


WNDRVR

```
fe0e8068: 00000000 ? 55555555
fe0e806c: 00000000 ? 44444444
fe0e8070: 00000000 ? 33333333
fe0e8074: 00000000 ? 22222222
fe0e8078: 00000000 ? 11111111
fe0e807c: c308f87f ? .
```

8. Review the register programming, and correct it if necessary.

   Use the md command to verify that you have set the correct values for SRKHR[0:7] and OTPMKR[0:7].

   If they are incorrect, reset the board and reprogram the fuses.

9. Commit the register values to the fuse array (burn the fuses).

   CAUTION: **Once the SRKHR and OTPM fuses are burned, they are longer be programmable.**

   Use the mm command with the CCSR INGR register address (CCSR offset 0x000E_8020).

```
=> mm fe0e8020

fe0e8020: 00000000 ? 00000002
fe0e8024: 00000000 ? .
```

   Note: After writing to the fuse, be sure to wait at least 20 milliseconds for the fuse to program before cycling power.

10. Validate the fuse programming.

    Disable POVDD with SW6[8] = 0b0.

    Use the md command to review the CCSR SRKHR[0:7] registers and ensure that the SRKH was properly read from the fuse array.

    Also review the CCSR HPSR register to ensure the OTPMK is populated with a nonzero value with no parity errors.

    Power up the P4080DS and hit ENTER key to stop at U-Boot prompt.

    Check if the SRKH was burned correctly. For example:

```
=> md fe0e807c
fe0e821c: c308f87f ec640c6c 1a41a72b f5d7b563
fe0e822c: 8cfd81a1 4965b287 901287b0 5dedea1f
```

    Check the SM_HP status register. The value 09 means that all the keys were burned correctly. It is in check (secure boot) state.

```
=> md fe314014

fe314014: 80000900 00000000 00000000 00000000
```


WNDRVR

7. Program Binaries into Flash

Program **u-boot.bin**, **hdr_uboot.out**, **rcw_66_15_1800MHz_sb.bin** for the (T2080QDS) or the **rcw_13g_sben_1500mhz_rev2.bin** (for the P4080DS), **bootscript** (optional), **hdr_bs.out**, **t2080qds.dtb**, **hdr_dtb.out**, **hdr_vxworks.out**, **uVxWorks** into flash.

About This Task

The **t2080qds.dtb** and **p4080ds.dtb** files are VxWorks DTBs.

The **hdr_vxworks.out** file is the same as **hdr_linux.out** file, which is renamed with the **secureboot.sh** script.

Before You Begin

Program Fuses for the T2080 on page 17 or Program Fuses for the P4080 on page 19

Procedure

1. Copy the files to the appropriate TFTP boot directory on the host.

   The files listed above are in the SDK code signing tool (**cst/**) directory. For example, for the T2080QDS: **/home/xeno/QorIQ-SDK-V2.0-20160527- yocto/build_t2080qds/tmp/sysroots/x86_64-linux/usr/bin/cst/**

2. Program **u-boot.bin** on vbank 4.

```
=>tftp 1000000 u-boot.bin
=>protect off EBF40000 +$filesize
=>erase EBF40000 +$filesize
=>cp.b 1000000 EBF40000 $filesize
=>protect on EBF40000 +$filesize
```

3. Program **hdr_uboot.out** on vbank 4.

```
=>tftp 1000000 hdr_uboot.out
=>protect off ECB00000 +$filesize
=>erase ECB00000 +$filesize
=>cp.b 1000000 ECB00000 $filesize
=>protect on ECB00000 +$filesize
```

4. Program the RCW file on vbank 4.

   For the P4080DS, the RCW file is **rcw_13g_sben_1500mhz_rev2.bin**.

   For the T2080QDS, the RCW file is **rcw_66_15_1800MHz_sb.bin**. For example:

```
=>tftp 1000000 rcw_66_15_1800MHz_sb.bin
=>protect off EC000000 +$filesize
=>erase EC000000 +$filesize
=>cp.b 1000000 EC000000 $filesize
=>protect on EC000000 +$filesize
```

5. Program **bootscript** on vbank 4.

```
=>tftp 1000000 bootscript
=>protect off ECA00000 +$filesize
=>erase ECA00000 +$filesize
=>cp.b 1000000 ECA00000 $filesize
=>protect on ECA00000 +$filesize
```

6. Program **hdr_bs.out** on vbank 4.

```
=>tftp 1000000 hdr_bs.out
=>protect off ECE00000 +$filesize
=>erase ECE00000 +$filesize
=>cp.b 1000000 ECE00000 $filesize
=>protect on ECE00000 +$filesize
```

7. Program the DTB file on vbank 4.

   For the P4080DS, the DTB file is **p4080ds.dtb**.

   For the T2080QDS, the DTB file is **t2080qds.dtb**. For example:

```
=>tftp 1000000 t2080qds.dtb
=>protect off EC800000 +$filesize
=>erase EC800000 +$filesize
=>cp.b 1000000 EC800000 $filesize
=>protect on EC800000 +$filesize
```

8. Program **hdr_dtb.out** on vbank 4.

```
=>tftp 1000000 hdr_dtb.out
=>protect off ED100000 +$filesize
=>erase ED100000 +$filesize
=>cp.b 1000000 ED100000 $filesize
=>protect on ED100000 +$filesize
```

9. Program **hdr_vxworks.out** on vbank 4.

```
=>tftp 1000000 hdr_vxworks.out

=>protect off ED000000 +$filesize
=>erase ED000000 +$filesize
=>cp.b 1000000 ED000000 $filesize
=>protect on ED000000 +$filesize
```

10. Program **uVxWorks** on vbank 4

```
=>tftp 1000000 uVxWorks
=>protect off EC020000 +$filesize
=>erase EC020000 +$filesize
=>cp.b 1000000 EC020000 $filesize
=>protect on EC020000 +$filesize
```

11. Power off the board.

WNDRVR

8. Boot System and Verify Security State

Boot the system to verify the security state with the ISBC and ESBC.

About This Task

The internal secure boot code (ISBC) is contained in internal ROM to ensure that the code cannot be modified by an attacker. The external secure boot code (ESBC) is the signed **u-boot.bin** image.

Note: If the ITS fuse is not set, U-Boot will still boot the VxWorks image, even if it is not validated. It will boot in the “Non-Secure” security monitor state, and an error code will be written to the CCSR SCRATCHRW2 register. A list of error codes can be found in the NXP SDK documentation.

Procedure

1. Boot the board from vbank 4.

   For the P4080QDS vbank 4 is SW7[1:4] = 0b0100.

   For the T2080QDS vbank 4 is SW6[1:4] = 0b0100.

   ESBC validation requires approximately 2-3 seconds. Boot up messages appear after a successful ESBC validation.

2. Press the ENTER key to stop at the U-Boot console.
3. Check the security state.

   The CCSR HPSR register reports the security state, and is located at 0xFE31_4014. In this example, the output indicates the "trusted" security monitor state.

   ```
   [VxWorks Boot]: d 0xfe314014

   0xfe314010: 8000 0d00 0000 0000 0000 0000
   ```

4. Check for ISBC violations.

   The CCSR SCRATCHRW2 register reports ISBC violations, and is located at 0xFE0E_0204. In this example, there are no violations.

   ```
   [VxWorks Boot]: d 0xfe0e0204

   0xfe0e0200: 0000 0000 0000 0000 0000 0000
   ```

9. Code for secureboot.sh

The **secureboot.sh** code is used to generate the public and private keys, and to sign the images.

Shell Script Code

```
#!/bin/bash
# Copyright (c) 2016 Wind River Systems, Inc.
```

```
#
# The right to copy, distribute, modify or otherwise make use
# of this software may be licensed only pursuant to the terms
# of an applicable Wind River license agreement.


PROG_PATH="$(pwd)"
FILE_PATH="usr/bin/cst"

UBOOT_NAME="$PROG_PATH/u-boot.bin"
IMAGE_NAME="$PROG_PATH/uVxWorks"
SCRIPT_FILE="$PROG_PATH/bootscript.txt"
PRIV_KEY="$PROG_PATH/srk.pri"
PUB_KEY="$PROG_PATH/srk.pub"
P4080DS="p4080ds"
T2080QDS="t2080qds"

if [[ ! "$PROG_PATH" =~ "$FILE_PATH" ]];then
        echo "script path error"
        exit 1
fi


if [[ "$PROG_PATH" =~ "$P4080DS" ]];then
        INPUT_PATH="input_files/uni_sign/p3_p4_p5"
        BOARD_TYPE="p4080ds"
        DTB_FILE="p4080ds.dtb"
elif [[ "$PROG_PATH" =~ "$T2080QDS" ]];then
        INPUT_PATH="input_files/uni_sign/t1_t2_t4"
        BOARD_TYPE="t2080qds"
        DTB_FILE="t2080qds.dtb"
else
        echo "error path."
        exit 1
fi


#judge dtb file
if [[ ! -e "$PROG_PATH/$DTB_FILE" ]];then
        echo "$DTB_FILE is missing."
        exit 1
fi

#judge u-boot.bin file
if [[ ! -e "$UBOOT_NAME" ]];then
        echo "u-boot.bin is missing."
        exit 1
fi

#judge uVxworks file
if [[ ! -e "$IMAGE_NAME" ]];then
        echo "uVxWorks is missing."
        exit 1
fi

#add script file
touch "$SCRIPT_FILE"
```

```
echo "esbc_validate 0xE9000000">"$SCRIPT_FILE"
echo "esbc_validate 0xE9100000">>"$SCRIPT_FILE"
echo "bootm 0xE8020000 - 0xE8800000">>"$SCRIPT_FILE"

#make script bin
../mkimage -T script -C none -d bootscript.txt bootscript > /dev/null 2>&1


#gen key

if [ ! -e "$PRIV_KEY" ]||[ ! -e "$PUB_KEY" ];then
        while :
        do
                read -p "Please enter key length(1024/2048/4096):" length
                case $length in
                        1024|2048|4096)
                                echo "generate $length key pair..."
                                ./gen_keys $length > /dev/null 2>&1
                        break
                ;;
                        *)
                                echo "key pair length is wrong."
                ;;
                esac
        done
else
        echo "key pair exist."
fi

#sign u-boot

if [ "$BOARD_TYPE" == "p4080ds" ];then
        ./uni_sign $INPUT_PATH/input_uboot_secure > SRK_output.out
else
        ./uni_sign $INPUT_PATH/input_uboot_nor_secure > SRK_output.out
fi

#sign bootscript
./uni_sign $INPUT_PATH/input_bootscript_secure > /dev/null 2>&1

#sign vxworks
sed -i "s/uImage.bin/uVxWorks/g" $INPUT_PATH/input_uimage_secure
./uni_sign $INPUT_PATH/input_uimage_secure > /dev/null 2>&1
mv -f hdr_linux.out hdr_vxworks.out

#sign dtb
sed -i "s/uImage.dtb/$DTB_FILE/g" $INPUT_PATH/input_dtb_secure
./uni_sign $INPUT_PATH/input_dtb_secure > /dev/null 2>&1

#gen random
if [ ! -e "$PROG_PATH/gen_otpmk.out" ];then
        echo "generate One time programmable master key..."
else
        echo "$BOARD_TYPE sign successfully"
        exit 1
fi
```

```
if [ "$BOARD_TYPE" == "p4080ds" ];then
        ./gen_otpmk_drbg 1 > gen_otpmk.out
else
        ./gen_otpmk_drbg 2 > gen_otpmk.out
fi

echo "$BOARD_TYPE sign successfully."
```

10. Common Boot Issues

Common issues with booting and their resolution.

Once the platform has been setup for secure boot, there are several checks that can be made if the platform fails to boot properly, or not at all.

Board Does Not Boot

Check that the proper PBL/RCW, U-Boot and CSF header images are programmed into flash. If any of them are missing, the board may not boot at all. Boot from alternate flash bank to verify that the images are correct.

If the ITS fuse is set, the signature contained in the CSF header may be incorrect, in which case the board will fail to boot. Boot from alternate flash bank to correct the signature.

The dip switch settings for SW7 may be set to boot from an incorrect flash bank. Check the SW7 dip switch settings.

Board Boots in Non-Secure Mode

Check the value in the DCFG_SCRATCHRW2 register for the secure boot error code. The most likely problem is an incorrect signature in the CSF Header. Verify that properly signed images are programmed into flash (including PBL/RCW, CSF header, U-Boot and boot script).

The SRKH values programmed into fuses may differ from the public key used to sign the images. Verify that the proper keys were used.

No Output on UART Console

Check the status register of the security monitor block. See the NXP SDK reference manual for details about the register. The OTPMK_ZERO, OTMPK_SYNDROME and PE bits should be 0. If not, there is an error in the blown OTPMK fuse.

If the OTMPK fuse is correct, check the DCFG SCRATCHRW3 register for an error code. For a list of error codes, see the ISBC error codes in the reference manual. If the error code is 0, then check the security monitor state in the HPSR register (0x9 is the "check state").

If the ITS fuse is set to 1, then internal secure boot code (ISBC) has reset the board. This may be due to one of the following reasons:

- The hash of the public key used to sign the ESBC u-boot does not match the value in SRK hash fuse.
- Signature verification of the image failed.

Check the security monitory to see if it is in the trusted state (0xd) or non-secure state (0xb). If it is in the non-secure state, check, the entry point field in the CSF header. If entry point is correct, ensure that the U-Boot image has been signed with the correct input file

Get U-Boot Prompt Instead of Linux Prompt

You have not booted in secure boot mode. You will not get a U-Boot prompt with a secure boot. You would reach this stage if ITS is set to 0, and you are running a standard U-Boot image.

U-Boot Hangs or Board Resets

A validation failure has occurred with U-Boot. An error code and description will be printed on the U-Boot console.

4. SECURE BOOT FOR NXP SABRESD I.MX6

1. Secure Boot

VxWorks supports secure boot for NXP SabreSD i.MX6q and i.MX6SX.

High Assurance Boot

Using a trusted bootloader and operating system is the first step towards a secure execution environment. For the i.MX6 family of processors, you can create a secure bootloader by using the High Assurance Boot (HAB) component of the on-chip ROM. The HAB employs asymmetric encryption for authentication. It can authenticate a U-Boot image by using digital signatures, and then U-Boot can authenticate a VxWorks image that has been signed in the same manner.

Note: Ensure you choose a recent u-boot version that does not contain any known vulnerabilities.

When you configure a system with HAB, you must select the configuration that is appropriate for the HAB version of your hardware:

- For i.MX6 hardware that supports HAB 4.1.2 and later, users should set up Secure Boot with Fast Authentication, then set up Secure Loader with i.MX6 trust store with one of the SRKs. In that case, the RTPs are signed using one of the SRKs.
- For i.MX6 hardware that supports HAB versions earlier than 4.1.2, users should set up Secure Boot with normal HAB authentication and use a static trusted key store provider. Typically, users would set up the static trusted key store to contain the same IMG key that was used to sign the VxWorks image. Then when signing RTPs and DKMs and shared libraries, users would use the IMG private key instead of the SRK.

To see which version of HAB chip is used in your hardware, see the NXP application note AN4581, Secure Boot on i.MX 50, i.MX 53, i.MX 6 and I.MX 7 Series using HABv4. Appendix D shows the HAB Version/Chip matrix.

Boot-time Authentication

At boot time, before loading the U-boot image, the HAB performs the authentication process. It decrypts the signature using the private key. It then calculates the digest for the image and compares it with the decrypted one. If they match, the U-Boot image qualifies as "trusted", and is loaded and run. The U-Boot image then authenticates the VxWorks image, and loads and runs it.

The board can only load a signed U-Boot image, and U-Boot and only load and run a signed VxWorks image, which makes the system secure. An attacker can only see the public keys, and the private keys are protected by the organization that has created them, so attackers have no way to make a valid certificate for their own image.

Additional documentation that you may find useful includes: *Secure Boot on i.MX50, i.MX53, and i.MX 6 Series using HABv4* (available from http://www.nxp.com), and *Encrypted boot loader on SabreSD i.MX6q board* (available from http://community.nxp.com).

2. Required Third Party Software

Implementation of secure boot requires the use of Linux (natively or with VirtualBox on Windows), as well several third-party applications.

- Ubuntu, version later than 14.04.
- NXP Code Signing Tool (CST), which is available from http://www.nxp.com. The version of the tool used for this release of VxWorks is 2.3.2. (NXP CST is only available for Linux.)

- OpenSSL 1.0.0d or later, which is available from http://www.openssl.org. You can check if it is already installed by running the command openssl version.
- GNU objcopy 2.15 or later, which is available from www.fsf.org. You can check if it is already installed by running the command objcopy -V.
- Perl 5, which is available from http://www.perl.org. You can check if it is already installed by running the command perl –v.
- Linaro GCC compiler, which is available from http://www.linaro.org.

3. Command Sequence File

The command sequence file (CSF) is a key component of a HAB.

The CSF includes commands, the super root key (SRK) table, signatures, and certificates. It is used both by the host tools that create the signature and the HAB firmware that validates the signature. The CSF file is included in the signature data generated by the code signing tools (CST), and it is appended to the image file.

The data parts that will be covered by the signature are specified in the CSF:

```
[Authentication Data]
   Blocks = addr offset length file
   Blocks = addr offset length file
```

offset and *file* are used in the signature creation. The entire file is included in the signature calculation, except if *offset* greater than zero, then *offset* bytes are skipped in the beginning of the file.

addr and *length* are used in the verification of the signature. They identify the memory sections that will be included when the HAB firmware verifies the signature.

Device Tree Blob

The signature must identify the device tree blob (DTB).

If your system uses **vxworks.bin**, you can meet this requirement by adding **INCLUDE_STANDALONE_DTB** to the VxWorks image project (VIP).

If your system uses uVxWorks, use the *file* parameter in the second **Blocks** entry in the CSF to identify the DTB.

Determining the addr Field in the CSF

The method you use to determine the *addr* field in the CSF depends on whether you are using **vxWorks.bin** or **uVxWorks**.

For **vxWorks.bin**, examine the Load Address in the build output. vxWorks.bin is booted by the **go** u-boot command. The **go** command jumps to the start address given as the argument.

For **uVxWorks**, uVxWorks is booted by the **bootm** u-boot command. The **bootm** command takes the address where the image with a u-boot header resides in memory and the address to the DTB blob as argument. The **bootm** command then moves the image to its final destination, based on the info in the u-boot header, and thereafter jumps to the start address. The precise start address depends on whether the **bootm** command verifies the signature before or after the image is moved. The timing of verification will affect the address that needs to be entered into the CSF file.

Code Signing Tool on page 31 The NXP code signing tool (CST) is used to create pairs of public and private keys.

WNDRVR

3.1. Code Signing Tool

The NXP code signing tool (CST) is used to create pairs of public and private keys.

These key sets are used to sign U-Boot and VxWorks and to program the eFUSE.

The CST can create two types of key hierarchies—fast authentication or normal authentication.

When fast authentication is used, the CST generates only super root keys (SRK). The secure ELF loader uses the i.MX6 trusted key store provider. The SRK key is used to sign the U-boot, the VxWorks kernel image, and VxWorks ELF objects (RTPs, DKMs, and shared libraries).

When normal authentication is used, the CST generates IMG and credential store framework (CSF) keys, in addition to the SRK keys. The secure ELF loader uses the static trusted key store. The IMG key is used to sign all images.

The CST creates a hash of the public key digest, which is burned into the i.MX6 board's eFUSE. At boot time, the HAB uses the hash to verify that the public key in the U-Boot image is valid. Once the digest is burned into the eFUSE, it cannot be changed.

For both the U-Boot and VxWorks images, the private keys are used to encrypt the hash of the image, the result of which is called the signature. The signatures and the public keys are then added to the images.

The CST can generate 1-4 SRKs. These keys can be revoked by setting a corrresponding revocation bit in the eFUSEs.

Parent topic: Command Sequence File on page 30

4. Generating Keys and Programming the eFUSEs

Use NXP CST to generate the public keys, generate the fuse commands, and execute the fuse commands with U-Boot.

About This Task

Procedure

1. Change directory to the **keys/** directory in NXP CST tool installation.

   The **keys/** directory contains the private key files used for high assurance boot (HAB) code signing. This directory also contains the scripts to generate the public key infrastructure (PKI) tree.

2. In the **keys/** directory, create or update the file **serial** with an eight-digit number for its content.

   For example, edit the file adding **12345678**. This is used as a serial number by OpenSSL.

3. In the **keys/** directory, create or update the file **key_pass.txt** with two identical passphrases for its content.

   Using passphrases prevents unauthorized use of a private key, even by someone with root privileges who can access the **ssh** directory (which holds the private keys). The passphrase must be at least 5 characters long. A good passphrase should be at least 10 characters long, and include random upper and lower case letters, numbers, and symbols.

   The passphrase must appear twice, once each on separate lines in the file.

   For example, the contents of **key_pass.txt** could look something like this:

WNDRVR

```
mX!_e@-P7$
mX!_e@-P7$
```

4. In the **keys/** directory, execute the **hab4_pki_tree.sh** script.

   Respond to the prompts as follows:

```
Do you want to use an existing CA key (y/n)?: n
Do you want to use Elliptic Curve Cryptography (y/n)?: n
Enter key length in bits for PKI tree: 2048
Enter PKI tree duration (years): 10
How many Super Root Keys should be generated?: 4
Do you want the SRK certificates to have the CA flag set? (y/n)?: y
```

   The script indicates whether or not key and certificate generation is successful or not.

5. In the **crts/** directory, generate the super root key (SRK) table and eFUSE hash.

   For example:

```
$ cd ../crts
$ ../linux64/srktool -h 4 -t SRK_1_2_3_4_table.bin -e SRK_1_2_3_4_fuse.bin -d sha256 -c ./SRK1_
sha256_2048_65537_v3_ca_crt.pem,./SRK2_sha256_2048_65537_v3_ca_crt.pem,./SRK3_sha256_2048_65537
_v3_ca_crt.pem,./SRK4_sha256_2048_65537_v3_ca_crt.pem -f 1
```

   This command generates the **SRK_1_2_3_4_fuse.bin** file.

6. From the VxWorks 7 **_bootloader** sub-directory under your BSP directory, burn the U-Boot image to your SD card.

   For example:

```
$ cd /home/xeno/WindRiver/vxworks-7/pkgs_v2/os/board/freescale/fsl_imx6/_bootloaders
$ sudo dd if=u-boot-imx6q-sabresd.imx of=/dev/sdc bs=512 seek=2 && sync && sync
```

7. In the **crts/** directory, generate the fuse commands that you will use with U-Boot.

```
$ hexdump -e '/4 "0x"' -e '/4 "%X""\n"' SRK_1_2_3_4_fuse.bin | awk '{print "fuse prog 3 " NR-1
, $0}'
```

   The output displays a set of eight commands, similar to this one:

```
fuse prog 3 0 0xAAA9A0CB
```

8. Boot the target board.
9. At the U-Boot command line, run each of the eight fuse commands.

   CAUTION: **Once this is done, the fuses are no longer programmable.**

5. Creating a Secure and Signed U-Boot Image, and Closing the Board

To sign U-Boot, first update the U-Boot timer file and the board's configuration and header files, then add an authentication call to **bootm.c**, and finally sign the image using **ubootSign.pl** (or your own CSF script). Close the board with the U-Boot fuse command.

About This Task

The NXP i.MX 6 series chips use a special u-boot file format consisting of an image vector table (IVT), device configuration data (DCD), the **u-boot.bin** image, and command sequence file (CSF). The CSF includes commands, the super root key (SRK) table, signatures, and certificates. The mkimage command (which is invoked by make) adds the IVT and DCD to generate **u-boot.imx**, before **u-boot.bin** is created. The CSF binary is generated by the code signing tool (CST) and attached to **u-boot.bin**.

Procedure

1. Clone the specific U-boot version you want to build from its git repository.

```
$ git clone git://git.denx.de/u-boot.git
```

2. Run the following command to switch to a new branch that contains the secure features.

   The subsequent steps are very specific to commit 6b3943f1b04be60f147ee540fbd72c4c7ea89f80 or branch v2016.03.

```
$ git checkout v2016.03
```

3. Update the U-boot **timer.c** file to enable secure features in U-Boot.

   Add the following code to the end of **arch/arm/imx-common/timer.c**:

```
unsigned long usec2ticks (unsigned long usec)
        {
        ulong ticks;
        if (usec < 1000)
            ticks = ((usec * (get_tbclk ()/1000)) + 500) / 1000;
        else
            ticks = ((usec / 10) * (get_tbclk () / 100000));
        return ticks;
        }
```

4. Update the board's configuration file.

   - For i.MX6q, **board/freescale/mx6sabresd/mx6q_4x_mt41j128.cfg**.
   - For i.MX6SX, **board/freescale/mx6sxsabresd/imximage.cfg**.
   - For i.MX6q sabrelite, **board/boundary/nitrogen6x/nitrogen6q.cfg**.

   In the configuration file for your board, after the line:

```
BOOT_FROM sd
```

   Add the line:

```
CSF 0x2000
```


WNDRVR

For example:

```
/*
* Boot Device : one of
* spi, sd (the board has no nand neither onenand)
*/
BOOT_FROM        sd
CSF 0x2000
```

This information is used by mkimage (which is called by make) when it builds the IVT header for the CSF region.

5. Update the board's header file.

   Add the following definitions:

```
#define CONFIG_SECURE_BOOT
#define CONFIG_SYS_FSL_SEC_COMPAT    4 /* HAB version */
#define CONFIG_FSL_CAAM
#define CONFIG_CMD_DEKBLOB
#define CONFIG_SYS_FSL_SEC_LE
#define CONFIG_FAT_WRITE
```

   to the header file, in the location specified below:

   - For i.MX6q, in **include/configs/mx6sabresd.h**, add the definitions before the line:

```
#define CONFIG_MACH_TYPE 3980
```

   - For i.MX6SX, in **include/configs/mx6sxsabresd.h**, add the definitions before the line:

```
#define CONFIG_SYS_MALLOC_LEN (3 * SZ_1M)
```

   - For i.MX6q sabrelite, in **include/configs/nitrogen6x.h** add the definitions before the line:

```
#define CONFIG_MACH_TYPE 3769
```

   This information is used to invoke he HAB functions that U-Boot uses to verify the VxWorks signature.

6. Optionally, remove all other boot commands except bootm.

   The commands are in several files under **u-boot/cmd/**.

7. Add the authenticate_image( ) function to **bootm.c**

   In **cmd/bootm.c**, in the do_bootm( ) function, before this line:

```
return do_bootm_subcommand(cmdtp, flag, argc, argv);
```

   Make the conditionalized call to authenticate_image( ), as follows:

```
#ifdef CONFIG_SECURE_BOOT
    {
```


WNDRVR

```
        unsigned long addr = simple_strtoul (argv[0], NULL, 16);
        extern uint32_t authenticate_image(
                        uint32_t ddr_start, uint32_t image_size);
        if (authenticate_image((uint32_t) addr,
                        image_get_image_size((image_header_t *) addr)) == 0) {
                printf("Authenticate UImage Fail, Please check\n");
                return 1;
        }
    }
#endif
```

This ensures that the bootm command will only boot a **uVxWorks** image that is authenticated (a signed **uVxWorks** image).

8. Build the U-Boot image.

   If not done already, download the Linaro GCC compiler and define the **CC** environment variable, then build the U-boot image using make as shown below.

```
$ export CC=$OPTEE_HOME/toolchains/gcc-linaro-arm-linux-gnueabihf-4.9-2014.08_linux/bin/arm-lin
ux-gnueabihf-
```

   For example, for i.MX6q:

```
$ make ARCH=arm CROSS_COMPILE=${CC} distclean
$ make ARCH=arm CROSS_COMPILE=${CC} mx6qsabresd_defconfig
$ make ARCH=arm CROSS_COMPILE=${CC}
```

   For example, for i.MX6SX:

```
$ make ARCH=arm CROSS_COMPILE=${CC} distclean
$ make ARCH=arm CROSS_COMPILE=${CC} mx6sxsabresd_config
$ make ARCH=arm CROSS_COMPILE=${CC}
```

   This produces a **u-boot.imx** file, with padding.

   In the example above, the environment variable **${CC}** translates to $OPTEE_HOME/toolchains/gcc-linaro-arm-linux-gnueabihf-4.9-2014.08_linux/bin/arm-linux-gnueabihf-.

9. Sign the image using **ubootSign.pl** (or your own CSF script).

   The **ubootSign.pl** script is provided in *installDir* **/vxworks-platform/guests/vxworks-7/pkgs_v2/security/secure_loader/imx6_sign/**.

   For usage help, use the -h option.

   The first parameter to **ubootSign.pl** is the target board identifier, and the second is the relative pathname for the U-Boot image (**uboot.imx**) .

   - For i.MX6q, the target board identifier is imx6q-sabresd.
   - For i.MX6SX, the target board identifier is imx6sx-sabresd.
   - For Sabrelite, the target board identifier is imx6q-sabrelite.

   For example:

WNDRVR

```
./ubootSign.pl imx6x-sarbresd ../../uboot.s/uboot.imx
```

The output is the signed image **uboot-signed.imx**.

10. Write the signed **uboot-signed.imx** image to an SD card.

    For example:

```
$ sudo dd if=u-boot-signed.imx of=/dev/sdc bs=512 seek=2 ; sync
```

11. Turn on the board and use hab_status to check for any errors.

    The hab_status command will determine if HAB has verified the **u-boot.imx** image. If authentication is successful, the output is as follows:

```
=> hab_status

secure boot enabled

HAB Configuration: 0xf0, HAB State: 0x66
No HAB events Found!
```

12. Close the board.

    Once you have verified that U-Boot can be loaded, close the board with the fuse command. For example:

```
=> fuse prog 0 6 0x2
```

CAUTION: **Once you have executed this command, the HAB will only authenticate a U-Boot image with the correct signature.**

6. Signing and Booting VxWorks

Sign the VxWorks image using **uVxWorksSign.pl** (or your own CSF script). Boot it with the download address used when the image was signed.

About This Task

The uImage format is supported by U-Boot with the secure boot feature. The signed **uVxWorks** image consist of the original **uVxWorks** image, an image vector table (IVT), and command sequence file (CSF).

Procedure

1. Create a **uVxWorks** image, configured as required for your system.
2. Sign the image using **uVxWorksSign.pl** (or your own CSF script).

   The **uVxWorksSign.pl** script is provided in *installDir* **/vxworks-7/pkgs_v2/security/secure_loader/imx6_sign/**.

   For usage help, use the -h option.

WNDRVR

The parameters for **uVxWorksSign.pl** are the relative pathname for the VxWorks image, and the download address—which is board specific:

- for i.MX6q use 0x10800000
- for i.MX6SX use 0x88800000
- for Sabrelite use 0x10800000

For example:

```
./uVxWorksSign ../../../../vxworks/vxworks-7/workspace/vipimx6/uVxWorks 0x88800000
```

The output is the signed image **uVxWorks-signed.bin**.

3. Using the signed U-Boot image, boot VxWorks with the bootm command, using the board specific address where the image was loaded in the example script.

```
=> bootm 0x88800000
```

The board can only load a signed U-Boot image, and U-Boot and only load and run a signed **uVxWorks** image. The board is secured.

7. Signing and Loading RTPs and DKMs

For the i.MX6 trusted key store, RTPs and DKMs must be signed using any one of the four SRKs burned into the SRK eFuses, as long as the SRK has not been revoked.

The SRK public/private key pair is generated by NXP CST as described in Secure Boot on page 29, but the private key **SRK*_sha256_2048_65537_v3_ca_key.pem** is encrypted with the passkey used during the steps in Secure Boot on page 29. Run the following command first to get the plain text private key for use in signing the RTP:

```
openssl rsa -in SRK4_sha256_2048_65537_v3_ca_key.pem -out
   SRK4_sha256_2048_65537_v3_ca_key_plain.pem
```

This command prompts you for the passkey. Eenter the passkey used in the Secure Boot on page 29 steps.

Then sign the RTP, using the following command:

```
signtool --key=keys/SRK4_sha256_2048_65537_v3_ca_key_plain.pem --
cert=crts/SRK4_sha256_2048_65537_v3_ca_crt.pem --type=ELF rtp.vxe.signed rtp.vxe
```


WNDRVR

5. SECURE BOOT FOR XILINX ZYNQ-7000

1. Secure Boot

Secure boot can be provided for VxWorks on Zynq-7000 device, using quad serial peripheral interface (QSPI) and secure digital (SD) modes.

The principle objectives of a secure boot are to keep proprietary partitions confidential, and to prevent an attack from loading a modified partition. Creating secure a boot involves generating and programming cryptographic keys, and creating a secure boot image that includes VxWorks.

> Note: This document does not describe how to build the first-stage boot loader (FSBL) and generate the programmable logic (PL) bit stream for Zynq-7000.

Secure Boot Image and Authentication

A secure boot image includes the FSBL, U-Boot, the FPGA bitstream, the OS, and other application partitions. It is booted by Zynq-700 boot-ROM code. The boot-ROM code is masked programmed ROM code which runs at power-up. The boot-ROM code determines the boot mode, initializes the memory used in booting, and if in a boot mode other than JTAG, loads the FSBL into the on-chip memory controller (OCM).

> Note: Ensure you choose a recent u-boot version that does not contain any known vulnerabilities.

Authentication is used to load partitions in a chain of trust. The boot-ROM code loads the FSBL. The FSBL and U-Boot boot loaders then load the hardware and software partitions.

bootgen

The bootgen program generates a single image image file in binary or MCS format. An input BIF file lists the partitions, specifies load addresses for each partition, and specifies whether they should be encrypted or authenticated (or both).

bootgen can be used in debug mode or release mode.

Debug mode is easier to use, and meets many users' security requirements. For debug mode, RSA signatures for the partitions do not need to be provided by the user. The user provides private RSA keys in the BIF, and bootgen generates the hashes and the signatures.

Release mode provides improved security for RSA keys. It uses public keys and signatures from the BIF.

2. Hardware

The instructions provided in this document are based on the ZC702 Evaluation Board Setup ZC702 Evaluation Board.

The hardware configuration is as follows:

- The power cable is connected to the 12V J60 Connector.
- The USB cable is connected from the PC to the USB UART J17 Connector.
- The SD card is inserted into J64 SDIO slot.
- The boot mode (SW16) is set to SD card.

WNDRVR

The PL bit stream file should include security IP. The file is provided in the Xilinx reference design package. For information, see *Secure Boot of Zynq-7000 All Programmable SoC* (Application Note: Zynq-7000 AP SoC), which is available from http://www.xilinx.com.

3. Required Third-Party Software

Implementation of secure boot requires the use of several third-party applications.

- Linux or Windows host system.
- Perl 5.2.4. For Windows it is available from http://www.activestate.com.
- OpenSSL. For Windows, use the version available from http://www.cygwin.com, or the version available from http://www.slproweb.com (version v1.0.2h). Be sure to update the PATH environment variable for the OpenSSL executable.
- Xilinx SDK. Use SDK 2016.1 Webinstall (Windows 64), which is available from http://www.xilinx.com. The SDK contains the bootgen tool, program tools and the development environment used to build the secure key driver.
- Xilinx secure boot reference design, which is available from secure.xilinx.com.

4. Generate Keys

Generate AES and RSA keys.

About This Task

Five cryptographic keys are used:

- AES 256 bit key
- RSA 2048 bit Primary Secret Key
- RSA 2048 bit Primary Public Key
- RSA 2048 bit Secondary Secret Key
- RSA 2048 bit Secondary Public Key

The RSA authentication in Zynq uses primary and secondary keys. The primary keys authenticate the secondary keys. The secondary keys authenticate partitions. In bootgen debug mode, the user only provides private keys in the BIF file.

Procedure

1. Generate the AES Key.

   The AES key can be stored in a **.nky** file, and referenced in the BIF file using the **[aeskeyfile]** attribute.

   If do not use an **.nky** file, set the **[aeskeyfile]** attribute in the BIF and use bootgen to generate the key in file specified with the **[aeskeyfile]** attribute when creating a secure boot image.

   For example, the contents of the BIF file can look like this:

   ```
   {
   ...
   [aeskeyfile] efuse.nky
   ```

```
...
}
```

2. Generate RSA keys.

   Generate the primary and secondary RSA keys with the following OpenSSL commands:

```
openssl genrsa -out psk.pem 2048
```

```
openssl genrsa -out ssk.pem 2048
```

   The key files are in PEM format.

3. Extract the public keys from the private key files.

   With RSA, the public key is contained in the private key file. Using the following commands to extract the public keys from the private key files:

```
openssl rsa -pubout -in psk.pem -out ppk.pem
```

```
openssl rsa -pubout -in ssk.pem -out spk.pem
```

4. Convert the RSA private key files from PEM format to text format.

   Use the following Perl script commands:

```
perl convert_key.pl psk.pem psk.pem
```

```
perl convert_key.pl ssk.pem ssk.pem
```

   The Perl script **convert_key.pl** is provided in the Xilinx reference design package, **zc702_secure_key** directory.

5. Generate Hash of PPK

Use bootgen to generate the hash of the PPK.

Procedure

1. Open the Xilinx Software Command Line Tool.
2. Create a BIF file for the hash.

   Create **gen_hash_ppk.bif** with this content:

```
gen_hash_ppk:
{
[pskfile] psk.pem
[sskfile] ssk.pem
```

```
[bootloader, authentication=rsa] fsbl.elf
}
```

3. Use bootgen to generate the hash of the PPK.

```
bootgen -image gen_hash_ppk.bif -efuseppkbits hash_ppk.txt
```

This produces the **hash_ppk.txt** file, which contains the hash of the PPK. Using eFUSEs for the hash of the PPK is an efficient use of the hardware.

6. Create Secure Key Driver

Use the Xilinx SDK to create the secure key driver.

About This Task

Two example source files are provided: **xilskey_efuse_example.c** and **xilskey_input**. The functionality of the secure key driver is controlled by editing the these files. The **xilskey_input.h** file can be edited to program PS eFUSEs, PL eFUSEs, or both.

Procedure

1. Start the Xilinx SDK GUI.

   Accept the default workspace, or select another one.

2. Create your application project.
   1. Click **File > New > Application Project**.
   2. Name your project **secure_key_driver**.
   3. Select your hardware platform.
   4. For **Board Support Package**, select **Create New**, and enter **secure_key_driver_bsp**, and click **Finish**.
3. In the **Templates** dialog, select **Empty Application**, and click **Finish**.
4. Set the BSP settings.
   1. Right-click on the **secure_key_driver_bsp** project name, and select **Board Support Package Settings**.
   2. In the left panel, select **Overview > drivers > ps7_cortexa9_0**.
   3. Change the archiver and compiler **Value** from arm-none-eabi- to arm-xilinx-eabi-.
   4. Remove the -mfloat-abi=hard **Value** from **extra_compiler_flags**.
   5. In the left panel, select **Overview**.
   6. In the **Supported Libraries** section, select the **xilskey** library (version 5.0).
5. Remove the contents of *installDir* **\SDK\2016.1\data\embeddedsw\lib\sw_services\xilskey_v5_0\src.**
6. Copy the contents of **xilskey_v1_01_a\src** to the **xilskey_v5_0/src** directory.
7. Copy **xilskey_v1_02_a\src\include\xilskey_bbram.h** to the **xilskey_v5_0/src/include** directory.
8. In the Xilinx SDK GUI, rebuild the BSP project.

   Right-click on the BSP project name and select **Re-generate BSP sources**.

9. Copy **xilskey_efuse_example.c** and **xilskey_input.h** into the **src** directory of **secure_key_driver** project.

   The two files are in the reference design package directory **xapp1175\zc702_secure_key_driver\src**.

WNDRVR

10. Program the PL eFUSE.

    For RSA authentication, the hash of the PPK is stored in the PL eFUSE array. For AES decryption, the key is stored in either the PS eFUSE or BBRAM. Program the PL eFUSE with the secure key driver or Xilinx iMPACT. To program the PL eFUSE, four MIO outputs must be routed out of the MIO outputs into the JTAG pins. These are external connections which must be made on the printed circuit board.

11. Edit **xilskey_input.h**.
    1. If used, define **XSK_EFUSEPS_DRIVER** and **XSK_EFUSEPL_DRIVER**.
    2. Set **XSK_EFUSEPS_RSA_KEY_HASH_VALUE** to the value of the PPK hash that is in **hash_ppk.txt**.
    3. Set **XSK_EFUSEPS_ENABLE_RSA_KEY_HASH** to **TRUE** to program the RSA PPK hash.
    4. Set **XSK_EFUSEPS_ENABLE_RSA_AUTH** to **TRUE** to enable RSA authentication.
    5. If programming the PL, set **XSK_EFUSEPL_AES_KEY** to the value of the AES key, and set **XSK_EFUSEPL_PROGRAM_AES_AND_USER_LOW_KEY** to **TRUE** to program the AES key.

    During development, it is usually simpler to use iMPACT to program the AES key. For information, see the *Zynq-7000 All Programmable SoC Secure Boot Getting Started* (UG1025).

12. In the Xilinx SDK GUI, change the project's build settings.
    1. Right-click on the project name **secure_key_driver**, and select **C/C++ Build Settings**.
    2. Change the command **ARM v7 gcc assembler** to arm-xilinx-eabi-.
    3. Change the command **ARM v7 gcc compiler** to arm-xilinx-eabi-gcc
    4. Change the command **ARM v7 gcc linker** to arm-xilinx-eabi-gcc
    5. Change the command **ARM v7 print size** to arm-xilinx-eabi-size
    6. Remove the flag -mfloat-abi=hard from **ARM v7 gcc compiler > Miscellaneous**, and from **ARM v7 gcc linker > Miscellaneous**.
13. Build the application project for the secure key driver.

    If a build error occurs, remove the **obj** and **-p** directories under **libsrc\xilskey_v5_0** in the BSP project, and perform these steps again.

7. Run Secure Key Driver

Generate a new **BOOT.BIN** file with the key and key driver. Then run **BOOT.BIN** to program the key into board's eFUSE.

About This Task

Procedure

1. Create a new secure driver BIF file.

   Create the file with this content:

```
secure_key_driver_image:
    {
    [bootloader] fsbl.elf
    system.bit
    secure_key_driver.elf
}
```

2. Generate a new **BOOT.BIN** file.

```
bootgen -image secure_key_driver.bif -o i BOOT.BIN -w on
```

3. Copy **BOOT.BIN** to an SD card and install it on the target board.
4. Power up the board to program the eFUSE.

Note: For some zc702 boards the OTP default setting is write-protected. In this case, programming the eFUSE will fail. In the event of failure, you must change the secure key driver BSP source. Remove the following lines from **libsrc/xilskey_v5_0/src/xilskey_eps.c**:

```
/**
 * If eFUSE Array is write protected
 * no more writes are possible
 */
if (XilSKey_EfusePs_IsEfuseWriteProtected() == TRUE {
    ReValue = XSK_EFUSEPS_ERROR_EFUSE_WRITE_PROTECTED;
    goto ExitCtrlLock;
}
```

8. Create Debug-Mode Secure Boot Image

Debug mode is easier to use, and meets many users' security requirements.

About This Task

For debug mode, RSA signatures for the partitions do not need to be provided by the user. The user provides private RSA keys in the BIF, and bootgen generates the hashes and the signatures.

Procedure

1. Copy the FSBL and FPGA bit stream files that are provided in Xilinx reference package to your workspace.

   Copy these files to the workspace directory that holds the **.bif** and **.pem** files:

   - **zc702_secure_key/system.bit**
   - **zc702_linux_trd/fsbl.elf**
2. Start the Xilinx Software Command Line Tool.
3. Create a **VxWorks_secure.bif** file for VxWorks.
4. Add the content as appropriate for VxWorks without encryption, or VxWorks with encryption.

Option	Description

Without encryption.	`The_VxWorks_image:`
	`{`
	`  [pskfile] psk.pem`
	`  [sskfile] ssk.pem`
	`  [bootloader,authentication=rsa] fsbl.el`

WNDRVR

Option	Description

	`f`
	` [authentication=rsa] system.bit`
	` [authentication=rsa] u-boot.elf`
	` [authentication=rsa, load=0x200000] VxWorks.bin`
	`}`

With encryption.	`The_VxWorks_image:`
	`{`
	` [aeskeyfile] efuse.nky`
	` [pskfile] psk.pem`
	` [sskfile] ssk.pem`
	` [bootloader,authentication=rsa] fsbl.elf`
	` [authentication=rsa] system.bit`
	` [authentication=rsa] u-boot.elf`
	` [encryption=aes, authentication=rsa, load=0x200000] VxWorks.bin`
	`}`

5. Run bootgen command to create the secure boot image, as appropriate for VxWorks without or with encryption.

Option	Description
Without encryption.	**`bootgen -image VxWorks_secure.bif -o i BOOT.BIN -w on`**
With encryption.	Assuming that the AES key is stored in PL eFUSE: **`bootgen -p zc702 -image VxWorks_secure.bif -o i BOOT.BIN -encrypt efuse -w on`**

9. Create Release-Mode Secure Boot Image

Release mode provides improved security for RSA keys. It uses public keys and signatures from the BIF file.

About This Task

The **xil_rsa_sign** application is designed to emulate a hardware security module (HSM). It is provided in the **xc702_secure_key directory** (as and executable for Windows, and as source code for Linux). Creating a secure boot image in debug mode does not use an HSM, and therefore this tool is not used for debug images.

WNDRVR

Procedure

1. Copy the FSBL and FPGA bit stream files that are provided in Xilinx reference package to your workspace.

   Copy the following files to the workspace directory that holds your **.bif** and **.pem** files:

   - **zc702_secure_key/system.bit**
   - **zc702_linux_trd/fsbl.elf**
2. Create the SPK BIF file.

   Create the file **spk.bif** with this content:

```
spk:
{
[spkfile] spk.pem
}
```

3. Generate the SPK hash file.

```
bootgen -image spk.bif -generate_hashes
```

   The **spk.pem.sha256** file is generated.

4. Generate the SPK signature.

```
xil_rsa_sign -gensig -sk psk.pem -data spk.pem.sha256 -out spk.pem.sha256.sig
```

5. Create a BIF file for the partition hashes.

   Create **bootimage_partitions.bif** with this content:

```
generate_partition_hash:
{
[ppkfile] ppk.pem
[spkfile] spk.pem
[spksignature] spk.pem.sha256.sig
[bootloader, authentication=rsa] fsbl.elf
[authentication=rsa] system.bit
[authentication=rsa] u-boot.elf
[authentication=rsa, load=0x200000] VxWorks.bin
}
```

6. Generate the partition hashes.

```
bootgen -image bootimage_partitions.bif -generate_hashes
```

   The partition hash files are created: **fsbl.elf.0.sha256**, **ImageHeaderTable.sha256**, **VxWorks.bin.0.sha256**, **u-boot.elf.0.sha256**, **u-boot.elf.1.sha256**, and **system.bit.0.sha256**.

7. Generate signatures for the partitions.

```
xil_rsa_sign -gensig -sk ssk.pem -data ImageHeaderTable.sha256 -out ImageHeaderTable.0.sha256.s
ig
```

```
xil_rsa_sign -gensig -sk ssk.pem -data fsbl.elf.0.sha256 -out fsbl.elf.0.sha256.sig
```

```
xil_rsa_sign -gensig -sk ssk.pem -data VxWorks.bin.0.sha256 -out VxWorks.bin.0.sha256.sig
```

```
xil_rsa_sign -gensig -sk ssk.pem -data system.bit.0.sha256 -out system.bit.0.sha256.sig
```

```
xil_rsa_sign -gensig -sk ssk.pem -data u-boot.elf.0.sha256 -out u-boot.elf.0.sha256.sig
```

These operations generate signatures for the SSK and the partitions.

8. Create a BIF file for the pre-signed image.

   Create **bootimage_presign.bif** with this content:

```
bootimage_presign:
  {
  [ppkfile] ppk.pem
  [spkfile] spk.pem
  [spksignature] spk.pem.sha256.sig
  [headersignature] ImageHeaderTable.0.sha256.sig
  [bootloader, authentication=rsa,  presign=fsbl.elf.0.sha256.sig] fsbl.elf
  [authentication=rsa, presign=system.bit.0.sha256.sig] system.bit
  [authentication=rsa, presign=u-boot.elf.0.sha256.sig] u-boot.elf
  [authentication=rsa,  presign=VxWorks.bin.0.sha256.sig, load=0x200000] VxWorks.bin
 }
```

9. Create the **BOOT.BIN** image.

```
bootgen -image bootimage_presign.bif -o i BOOT.BIN -w on
```

10. Verify **BOOT.BIN**.

```
xil_rsa_sign -verysign -file BOOT.BIN
```

If verification fails, re-generate the RSA keys (see Generate Keys on page 39), and then re-do this procedure (Run Secure Key Driver on page 42). This is a known Xilinx issue.

10. Run Secure Boot with VxWorks Image

Run secure boot with the VxWorks image.

Procedure

1. Add **BOOT.BIN** to the target.

Copy **BOOT.BIN** to a bootable SD card and insert it into the zc702 board.

2. Power on the board.

   The signed partitions are verified and loaded.

   The output indicates that VxWorks has been loaded into memory at 0x200000, which is the entry-point for VxWorks. This address is specified in the Zynq-7000 BSP reference.

3. Start VxWorks.

   At the U-Boot command prompt, enter:

```
go 0x200000
```

6. KEY STORE AND SECRETS REPOSITORY

1. Key Store and Secrets Repository

The key store stores public key pairs. The key pairs that can be used include Rivest Shamir Adleman (RSA), digital signature algorithm (DSA), and elliptic curve cryptography (ECC) keys. The secrets repository stores passwords and symmetric encryption keys.

The key store is used for user passwords (in clear text) that are used by applications to log users onto other systems.

The public key pairs, and the passwords and symmetric encryption keys, are stored encrypted in files. A key-encrypting password (KEP) is used to generate the encryption key. The same key-encrypting password that is used for the secrets repository is also used for the key store. The key-encrypting is password produced by a KEP provider (for which there are several options).

The user defines the base directory for the key pairs and passwords when the system is configured. The sub-directories **keys** and **vault** are automatically created below it when keys or passwords are added. For example:

```
/sd0:1/keys/mySslKey
/sd0:1/vault/myPw
```

The key store and secrets repository provide APIs and shell command for working with those facilities, such as for creating, importing, and deleting keys.

Note: The key store and secrets repository replaced the legacy secure key database. The legacy secure key database is supported for backwards compatibility reasons, but is deprecated. The secure key database APIs are now mapped to the new key store and secrets repository APIs. This means that a particular key ID can be accessed via both the key store and secrets repository, and secure key database APIs.

2. Key-Encrypting Password Providers

There are several options for key-encrypting password providers.

The key-encrypting password (KEP) used to encrypt the keys in the key store, and the secrets in the secrets repository, is derived from a text password. This password string is used as input to the PKCS#5 for the generation of an encryption key and initialization vector. The KEP is returned by the provider for both get and import operations. It is needed to derive the key for encryption (import) and decryption (get).

The following KEP providers may be used (one per system):

- password obfuscator KEP provider
- TPM 2.0 KEP provider
- a custom KEP provider

Password Obfuscator KEP Provider

A password obfuscator transforms a password string into a new string used as input to the PKCS#5 for the generation of an encryption key and initialization vector. The main protection that the password obfuscator provides is that an attacker cannot simply run the strings command on the image to get a list of passwords decrypt the files in the key store and secrets repository.

Using this KEP provider requires:

WNDRVR

- VSB configuration with the **SECURITY_MISC** layer.
- VIP configuration with the **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR** component.
- Setting the **SEC_VAULT_KEY_ENCRYPTING_PW** parameter to a random-string password.

  The random string should consist of at least 40 characters containing both lower and upper case letters, numbers, and special characters. No default is provided by intention, to ensure that unique values are used. You must define a password, otherwise your VIP will not build.

TPM 2.0 KEP Provider

The TPM 2.0 KEP provider seals a password string with TPM 2.0 hardware. If a key-encrypting password is not already stored in the TPM device, the KEP provider automatically generates a random string for the password and stores it.

Using this KEP provider requires:

- VSB configuration with the **SECURITY_MISC** and **TPM2_TSS** layers.
- VIP configuration with the **INCLUDE_SEC_VAULT_KEP_TPM** component.
- Setting the **DRV_TPM_INSTANCE_DEV_NAME** parameter to the I/O system name for the TPM device.

If your boot loader (such as UEFI) does not support TPM, you will need to use the password obfuscator password provider for the boot loader itself.

BLOB KEP Provider for NXP 5.x Security Engine

NXP 5*x* security engines protect data in a cryptographic data structure called a binary large object (BLOB), which provides both confidentiality and integrity protection. NXP 5*x* security engines are included, for example, with the T1040 and T2080 processors.

The security engine provides a method for protecting user-defined data across system power cycles. The data is encrypted so that it can be safely stored non-volatile storage before the chip is powered down. Each time the BLOB protocol is used to protect data, a different randomly-generated key is used to encrypt the data. This key is itself encrypted using a key encryption key, and the resulting encrypted key is then stored along with the encrypted data. The key-encryption key is derived from the chip's master secret key, so it can be recreated when the chip is powered up again.

Using this KEP provider requires:

- VSB configuration with the **CRYPTOMISC** layer.
- VIP configuration with **INCLUDE_IAF** and **INCLUDE_SEC_VAULT_KEP_SEC**.
- Setting the **SEC_KEP_BLOB_FILENAME** parameter to the KEP BLOB file name. It is stored at the same level as the **keys** and **vault** sub-directories.
- Setting the **SEC_VAULT_FILE_ROOT** parameter to the name of the directory holding the **keys** and **vault** sub-directories.

Then when the target is booted, the secKepSet( ) function must be called to set the key encrypting password. It uses the filename set with the **SEC_KEP_BLOB_FILENAME** configuration parameter, and the directory name set with the **SEC_VAULT_FILE_ROOT** parameter. If the call is successful, the KEP data is encrypted into an encrypted BLOB by the SEC 5.x engine using its key, and stored in a file called **kep.blob**, in the vault directory defined by **SEC_VAULT_FILE_ROOT**. If the KEP blob file already exists, the secKepSet( ) call fails.

CAUTION: **Do not hard-code the key-encryption password in the call to secKepSet( ). This would be unsafe as the key-encryption password would be included in the image in clear text.**

Custom KEP Provider

Users can install a custom KEP provider using the secVaultSetKepCb( ) function.

WNDRVR

3. Key Store APIs and Shell Command

The key store provides APIs and a shell command for key management.

Key Store APIs

secKeyStoreGet( )

```
EVP_PKEY* secKeyStoreGet(const char* keyId);
```

Returns the key pair with the specified **keyId**. The returned **EVP_PKEY** must be freed by the caller by calling EVP_PKEY_free( ).

secKeyStoreX509Get( )

```
EVP_PKEY* secKeyStoreX509Get(X509* x509, char* keyId, int keyIdLen);
```

Returns the key pair matching the public key in the X509 certificate given as input argument. **keyId** and **keyIdLen** are output parameters. **keyId** is a pointer to a buffer where the **keyId** of the key pair will be stored. **keyIdLen** holds the length of the **keyId** buffer. The returned **EVP_PKEY** must be freed by the caller by calling EVP_PKEY_free( ).

secKeyStoreImport( )

```
STATUS secKeyStoreImport(const char* keyId, const EVP_PKEY* key);
```

Imports **key** into the key store and gives it key ID **keyId**.

secKeyStoreDelete( )

```
STATUS secKeyStoreDel(const char* keyId);
```

Deletes a key from the key store.

Key Store Shell Command

```
[vxWorks *]# keyStore
keystore [options]
   list
   import <id> <file> <key-type> - imports the <key-type> specific key in <file> and gives it id <
id>
                                   <key-type> could be 'tpm2' or 'pkey'. If it is not specified, i
t uses
                                   'pkey' by default
   delete <id> - deletes <id> from the key store
   print <id> - prints the public key of <id>
   find <cert file> - prints the id of the key matching the cert in <cert file>
   help|-h

[vxWorks *]#
```

4. Secrets Repository APIs and Shell Commands

The secrets repository provides APIs and a shell command for data management.

WNDRVR

Secrets Repository APIs

secSecretImport( )

```
STATUS secSecretImport(const char* id, const unsigned char* secret, int len);
```

Imports data into the secrets repository. Data can be any sensitive data. Typical examples would be passwords that must be available in clear text, encryption keys, IKE pre-shared keys, WiFi passwords, etc.

The **id** argument holds the identity, or name, of the data to import. The **secret** argument points to the data that is to be imported. The **len** argument holds the number of bytes to import.

secSecretGet( )

```
int secSecretGet(const char* id, unsigned char* secret, int* secretLen);
```

Get data from the secrets repository. The data stored as **id** is written to the data buffer. The **secret** argument points to the data. The **secretLen** argument holds the length of the data buffer.

secSecretDel( )

```
STATUS secSecretDel(const char* id);
```

Delete data from the secrets repository. Data that is deleted from the secrets repository cannot be restored.

Secrets Repository Shell Command

```
[vxWorks *]# secret
secret [options]:
        list - list all secret's id
        import <id> [secret] - import <secret> into vault and give it id <id>
        delete <id> - delete <id> from the vault
        verify <id> [secret] - verify <id> with <secret>
        help|-h

[vxWorks *]#
```

5. Key Store Usage

Usually it is a system administrator who generates a key pair on the target or the host system, and then adds them to the VxWorks key store with an import function or shell command. Application code makes use of the keys by way of their key store ID.

VxWorks Configuration

The VSB must include the **OPENSSL** and **SECURITY_MISC** layers. If you want to use key store to manage the key pairs generated by the TPM2 OpenSSL engine, you must add the **TPM2_TSS** layer to your VSB.

The VIP must include:

- **INCLUDE_IPCRYPTO**
- **INCLUDE_SEC_KEY_STORE**

WNDRVR

> **Note:** When you add this component, you will get a configuration error, because it automatically adds **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**, and its parameter **SEC_VAULT_KEY_ENCRYPTING_PW** must be set. For security reasons, the parameter is not set to a default value.

- **INCLUDE_SEC_KEY_STORE_CMD** (for key store management commands)
- **INCLUDE_GENRSA_CMD** (for the genrsa key generation command)
- **INCLUDE_DSA_CMD** (for the dsa key generation command)
- **INCLUDE_SEC_SECRET**
- **SEC_VAULT_FILE_ROOT** and set it to the base directory for the key store and secrets repository
- **INCLUDE_TPM2_OPENSSL_ENGINE** is required if you want the key store to manage the key pairs generated by the TPM2 OpenSSL engine.
- A KEP provider (see Key-Encrypting Password Providers on page 48)
- A storage system.

Creating Keys

RSA keys and DSA keys can be generated from the VxWorks kernel shell (with the command interpreter) using the genrsa and dsa commands. For example:

```
[vxWorks *]# genrsa 2048 -out my-rsa-key-pair.pem
```

RSA keys can also be generated from the TPM2 OpenSSL engine if the layer **TPM2_TSS** is included in the VSB. For example:

```
[vxWorks *]# tpm2tss-genkey.vxe -a rsa -s 2048 my-tpm2-rsa-key-pair.pem --tcti=libtpm20tss.so.1:/tpm0
```

The syntax for these commands can be displayed using the --help option.

Importing Keys

Key pairs can be imported using the secKeyStoreImport( ) function or the keystore command. In either case, a *keyID* is assigned to the key pair when it is imported. The *keyID* is used by any application that needs to access the key pair in the store. VxWorks applications such as SSH must be configured to use the same *keyID* as is used when the key pair is imported.

As an example, assume that an application listens to incoming SSL connections. This application needs to have an X509 certificate and a matching private/public key pair. The X509 certification does not need to be protected against unauthorized reading. It can simply be stored in the file system. The key pair, however, is sensitive and should be stored in the key store.

Using the keystore shell command to import the key, for example:

```
[vxWorks *]# keyStore import mySslKey mySslKey.pem
```

This command imports the key pair and assigns them the ID **mySslKey**. The ID then identifies the key pair in the key store. For example:

```
[vxWorks *]# keyStore list
mySslKey
[vxWorks *]# ls "/ram0/keys"
/ram0/keys/.
/ram0/keys/..
/ram0/keys/mySslKey
```


WNDRVR

CAUTION: **For security reasons, the system administrator should now delete mySslKey.pem from the file system.**

Note: In this example, the system has been configured with a temporary key store in RAM, which can be useful for the development and test process. The system was configured with **INCLUDE_RAM_DISK** and **INCLUDE_RAM_DISK_FORMAT_ANY**. The value of **SEC_VAULT_FILE_ROOT** has been set to **"/ram0"**. For a production system, you would use a persistent file system.

Runtime Use

At runtime, the application can then fetch the SSL key from the key store using the ID, with code like this:

```
EVP_PKEY* sslKey;

sslKey = keyStoreGet("mySslKey");
if (!sslKey)
   /*  error handling */

/* use the sslKey as appropriate for your application */

/* Once done with the key, free it */
EVP_PKEY_free(sslKey);
```

6. Secrets Repository Usage

Arbitrary secrets such as passwords, encryption keys, and shared secrets can be added to the VxWorks secrets repository with an import function or shell command. Application code makes use of the secrets by way of their secrets ID.

VxWorks Configuration

The VSB must include the **OPENSSL** layer.

The VIP must include:

- **INCLUDE_IPCRYPTO**
- **INCLUDE_SEC_SECRET**
- **SEC_VAULT_FILE_ROOT** set to the base directory for the key store and secrets repository
- **INCLUDE_SEC_SECRET_CMD** (for secrets repository management commands)
- A KEP provider (see Key-Encrypting Password Providers on page 48)
- A storage system.

Importing Secrets

At runtime, use the secSecretImport( ) function, or the secret import shell command (using the command interpreter), to store secrets in the secrets repository. For example:

```
[vxWorks *]# secret import myPw myPassWordFile
```

This command imports the password and assigns it the ID **myPw**. The ID then identifies the password in the secrets repository. For example:

WNDRVR

```
[vxWorks *]# secret list
myPw
[vxWorks *]# ls "/ram0/vault"
/ram0/vault/.
/ram0/vault/..
/ram0/vault/myPw
```

The secret ID can be used by any application that needs to access the password.

Note: In this example, the system has been configured with a temporary key store in RAM, which can be useful for the development and test process. The system was configured with **INCLUDE_RAM_DISK** and **INCLUDE_RAM_DISK_FORMAT_ANY**. The the value of **SEC_VAULT_FILE_ROOT** has been set to **"/ram0"**. For a production system, you would use a persistent file system.

WNDRVR

7. TRUSTED KEY STORE

1. Trusted Key Store

VxWorks can be configured with a trusted key store for public key certificates. The trusted key store is then used to determine whether or not the owner of the public key is trusted. Signature verification is done by the entity calling the trusted key store.

The trusted key store is used, for example, with the secure ELF loader facility to validate the private signatures of kernel applications, RTP applications, shared libraries, plug-ins, and the system symbol table (when separate from VxWorks) before they are loaded.

2. Static Trusted Key Store

With the static trusted key store feature, the public key certificate is embedded in the VxWorks image.

The VSB project is configured with support for the static key store, and the public key. During the build process the trusted public key is added to the user configlette, **staticTrustStoreCfg.c**, which is copied to the VIP project. The public key is configured by adding a PEM-encoded X509 certificate in the configlette. The X509 certificate is only used as a container for the public key. No certificate validation is done by the static trusted key store itself (no checking of subject name, validity times, signature, and so forth).

To use the static trusted key store:

- The VxWorks VSB project requires the **OPENSSL** and **SECURITY_MISC** layers.
- The public key must be specified with the VSB option SECURE_LOADER_PUBLIC_SIGNING_CERT_FILE.
- The VIP project requires the **INCLUDE_STATIC_TRUST_STORE** component.

3. UEFI Key Database Trusted Key Store

The UEFI key database can be used for a trusted key store. This feature can only be used with the secure UEFI boot loader, and not with a standard UEFI boot loader.

VxWorks can be configured with support for using the UEFI database as a trusted key store. The PK, KEK and db EFI signature lists that are produced by the VSB build must be configured into the UEFI firmware. They are required for the UEFI firmware to authenticate the UEFI loader. The UEFI key database trusted key store enables the secure ELF loader to use the UEFI signature list (the white list) to determine whether a key used to sign an ELF object is trusted or not. From a system administrator's point of view, this provides a single point of configuration. The key (or keys) that are used to sign all software, including the UEFI loader, VxWorks kernel image, ELF objects (DKMs, RTPs and shared libraries), is configured in the UEFI key db.

To use the UEFI database for a trusted key store:

- The VxWorks VSB project requires the **SECURE_LOADER** layer.
- The VIP project requires the **INCLUDE_UEFI_DB_TRUST_STORE** and **INCLUDE_BOOT_LOADER** components.
- The **UEFI_DB_SIZE** parameter must be set to the required size of the UEFI database section. The default value of ((4 * 1000) + 24) allows storage of 4 average-sized certificates. Note that the 24 bytes are reserved for system use.
- As part of your UEFI firmware configuration, the **PK.sig**, **KEK.sig**, and **db.sig** must be added. The files are are created in the VSB output directory *vsbDir* **/secureLoader**.

WNDRVR

4. i.MX6 Trusted Key Store

The trusted public key for the i.MX6 is stored in a super root key (SRK). The SRK is an RSA key pair (private and public). It is used to sign the image. The SRK hash is stored in an i.MX6 eFUSE (one-time programmable non-volatile memory).

> Note: The i.MX6 trusted key store can only be used when the i.MX6 High Assurance Boot (HAB) is configured for fast authentication. If the HAB is configured for normal authentication, the static trusted key store provider must be used, in which case the SRK key is used to sign the RSA key that is used to sign the image. The public IMG key needs to be put into the static trusted key provider in this configuration.

To use an I.MX6 trusted key store:

- The VxWorks VSB project requires the OPENSSL and SECURITY_MISC layers.
- The VIP project requires the **INCLUDE_IMX_TRUST_STORE** component.
- You must copy the **SRK_1_2_3_4_table.bin** file (which was generated with your secure boot support VIP project) to the trusted key store VIP directory, and change the file name to **SRK_table.bin**.

5. Custom Trusted Key Store Provider

You can implement a custom trusted key store provider for the certificate store.

To do so, you must do the following:

- Implement this API:

```
STATUS secTrustStorePubKeyValidate(const char* trustStoreId, const X509* cert, const unsigned
int keyUsage);
```

- Create a CDF component and add it to **SELECT_TRUST_STORE_PROVIDER**. The **INIT_RTN** of the new component must call secTrustStorePubKeyValidateProviderSet( ) to register the new provider.

The static trusted key store and the trusted key store for i.MX6 are examples of trusted key stores. They are implemented using the same API as you would use with a custom trusted key store provider.

WNDRVR

8. SECURE ELF LOADER

1. Secure Loading for ELF Object Files

VxWorks can be configured with secure ELF loaders and a public certificate for loading code modules that have been signed with the corresponding private key.

The secure loaders validate and load signed kernel applications, RTP applications, shared libraries, plug-ins, and the symbol table (when separate from the VxWorks image). Any objects that cannot be validated are not loaded, and an error message is generated. If VxWorks is configured for security events, an event is also generated if the load operation fails.

The public certificate is stored in the trusted key store. For information about the trusted key store, see Trusted Key Store on page 55.

The signtool facility is used with the private key to sign ELF object files. The signed version of the file contains additional ELF sections with the signature value and the X509 certificate of the signing key.

As a convenience during development, you can alternatively use the objects that are automatically signed as part of the build process, using the private key that is defined with VSB project configuration.

Note: To use the secure ELF loader with UEFI secure boot (and optionally encrypted images), you must base your application code on a VSB that is configured and built as described in VSB Configuration and Build on page 3.

2. Implementing Secure ELF Loading in VxWorks

To implement secure ELF loading in VxWorks, you must configure your VSB and VIP projects with secure loader support, select and configure a trusted key store, and define the public and private keys.

About This Task

The VSB configuration process allows you to specify a private key file, or to use a default, for signing applications, shared libraries, plug-ins, and the symbol table (when separate from the VxWorks image) automatically when they are built. The automatic signing feature that is part of the VSB build also produces signed versions of the VxWorks shared libraries. Alternatively, you can sign the standard objects separately with the signtool facility (see signtool Reference on page 60).

Before You Begin

Determine which trusted key store you are going to use for your public key file. See:

- Trusted Key Store on page 55
- Static Trusted Key Store on page 55
- UEFI Key Database Trusted Key Store on page 55
- i.MX6 Trusted Key Store on page 56

Procedure

1. Create a VSB project, and add the **SECURE_LOADER** layer along with any other layers required for your system.

2. Optionally, specify your own private key file that will be used for automatically signing applications, shared libraries, plug-ins, and the symbol table (when separate from the VxWorks image) when they are built based on this VSB.

   Set **SECURE_LOADER_PRIVATE_SIGNING_KEY_FILE** to the path to your private key file. A default is otherwise generated by the VSB build. The build does not overwrite an existing file.

3. If you are using the static trusted key store, identify the public certificate file.

   Optionally set **SECURE_LOADER_PUBLIC_SIGNING_CERT_FILE** to the file path of your public certificate file. A default is otherwise generated by the VSB build. The build does not overwrite an existing file.

4. Build the VSB.

   If you have not specified your own private key and public certificate files with your VSB configuration, the files produced by the build are generated in *vsbDir* **/secureLoader**.

   The automatic signing feature also produces versions of the VxWorks shared libraries that are signed with the private key. They are built under *vsbDir* **/user/lib/common/**. For example, **libc.so.signed**. The **.signed** extension must be removed from shared libraries and plug-ins for use at runtime.

5. Create a VIP, and add **INCLUDE_SECURE_LOADER**.

   If you do not add this component, signature checking will not be performed.

6. If you are using the UEFI key database trusted key store, add **INCLUDE_BOOT_LOADER**.
7. Add and configure a trusted key store provider.
   - Static Trusted Key Store on page 55
   - UEFI Key Database Trusted Key Store on page 55
   - i.MX6 Trusted Key Store on page 56
8. Add any other components required for your system.
9. Build the project.

   The VxWorks image includes the public key that is used to validate signed objects when they are loaded.

Next Steps

Automatically Signing and Encrypting Application Objects on page 58

3. Automatically Signing and Encrypting Application Objects

As a convenience for development, the build system automatically creates signed versions of applications, shared libraries, plug-ins, and the symbol table (when separate from the VxWorks image). If you have configured your VSB for encrypted images, objects that are both encrypted and signed are also created.

About This Task

The objects are signed using the private key identified in the VSB configuration with **SECURE_LOADER_PRIVATE_SIGNING_KEY_FILE**.

For production systems, you would not normally use this method, but sign objects separately with the signtool facility. See signtool Reference on page 60.

Procedure

1. Create your application, shared library, or plug-in project, based on it on the VSB that you used for the VxWorks image that includes the secure loaders.
2. Code and build your project.

   An unsigned and a signed version of the object file are created in your project directory. If you have based your code on a VSB that supports object encryption, an object that is both encrypted and signed is also built. For example:

   - **foo-DKM.out**, which is unsigned
   - **foo-DKM.out.signed, which is signed**
   - **foo-DKM.out.encrypted, which is both signed and encrypted**

   RTPs and DKMs are signed using a different method. For more information on this procedure, see Signing and Loading RTPs and DKMs on page 37.

4. Using Signed Objects

Store application, shared library, and plug-in objects in trusted storage. Remove the **.signed** file name extension from shared libraries and plug-ins in runtime storage.

About This Task

Procedure

- Use trusted storage.

  CAUTION: **Kernel applications, RTP applications, shared libraries, and plug-ins should only be loaded directly from trusted storage, meaning storage that does not allow an attacker to leverage the fact that the loader accesses an ELF binary file more than once during a load operation. An NFS mounted disk on a remote host, for example would make this kind of attack easier. If the ELF binaries cannot be kept in trusted storage, then at runtime they should first be copied to a RAM drive, and then loaded from the RAM drive. For information about implementing a RAM drive, see the** *VxWorks 7 Debugging Guide:* Error Detection and Reporting Configuration Reference**.**

- Remove the **.signed** extension when you copy shared library and plug-in files from their project build directories to their runtime storage location.

  By default, signing an object creates a file name with the **.signed** extension (for example, **myFooLib.so.signed**). If the **.signed** extension is not removed from the shared library or plugin files used at runtime, the libraries will not be loaded. All other objects can be used with the extension or without it.
- Do not remove the **.signed** file name extension from a symbol table file. When VxWorks is configured for secure loading (with **INCLUDE_SECURE_LOADER**), only a symbol table with the **.signed** extension will be loaded. A symbol table without the extension will be ignored.
- If you use Workbench GUI facilities to start kernel or RTP applications (using **Run/Debug Kernel Task** or **Run/Debug Real-Time Process**), you must change the filepath to that of the signed object, as by default the un-signed file is specified.

9. SIGNTOOL OBJECT SIGNING FACILITY

1. signtool Reference

The signtool facility is used to sign kernel applications, RTP applications, shared libraries, and plug-ins with a private key.

It should also be used for secure loading of the downloaded symbol table or the passthrough file system symbol table (which are implemented with **INCLUDE_NET_SYM_TBL** and **INCLUDE_PASSFS_SYM_TBL**).

The signed version of the file includes an ELF section containing the (private-key) encrypted hash value of the file, which the loader uses to verify the object before loading.

signtool Syntax

```
signtool --key=keyFile --cert=certFile --type=[ELF|UEFI] [--output=outputFile] inputFile
```

The --cert option identifies the public signing certificate specified with the VSB option SECURE_LOADER_PUBLIC_SIGNING_CERT_FILE.

The type **UEFI** is used for the **UEFI** boot loader. The type ELF is used for the VxWorks image, symbol table, applications, plug-ins, and shared libraries.

signtool Usage

Run signtool from the VxWorks 7 development shell within Workbench (**Project > Open Development Shell > VxWorks 7**).

Note: Symbol table files must have the **.signed** file name extension. When VxWorks is configured for secure loading (with **INCLUDE_SECURE_LOADER**), only a symbol table with the **.signed** extension will be loaded. A symbol table without the extension will be ignored.

signtool Example

```
C:\tmp> signtool --key=privatekey.pem --cert=publiccert.pem --type=ELF --output=fooLib.so.signed f
ooLib.so
```

```
C:\tmp> signtool --key=privatekey.pem --cert=publiccert.pem --type=UEFI --output=BOOTX64.EFI.signe
d BOOTX64.EFI
```

```
C:\tmp> signtool --key=privatekey.pem --cert=publiccert.pem --type=ELF --output=vxWorks.signed vxW
orks
```

10. USER AUTHENTICATION AND MANAGEMENT

1. User Authentication and Management

The VxWorks user authentication and management facility provides for securing access to a VxWorks system.

The user management facility can be used for all services in VxWorks for which you need to authenticate users. It is particularly intended for services offering login capabilities such as the shell (console, telnet, rlogin, and SSH login), FTP, and SFTP. Password and login policies can also be implemented (for example for password complexity and login timeouts).

Once a user is logged in, the service that user is interacting with (for example, the shell or the FTP server), is automatically owned by that user. That is, the service task's user ID and group ID are set to those of the user (and cannot be changed unless the user is root). With the user privileges feature, users can also be restricted to specific operations of these services.

2. Basic VxWorks Configuration

Configuration for user authentication and management involves selection of both VSB and VIP features.

VSB Configuration

Select **USER_MANAGEMENT**.

For additional configuration options, see the topics for the specific features of user authentication and management.

VIP Configuration

To select an authentication back end, use **SELECT_USER_AUTHENTICATION_BACKEND**. By default, **INCLUDE_LOCAL_AUTH** is selected. The **INCLUDE_LOCAL_AUTH** component requires the user database facility (**INCLUDE_USER_DATABASE**).

For the user database facility, add the **INCLUDE_USER_DATABASE** component to a VIP project based on your VSB project. For information about setting the associated parameters, see User Database on page 61. In addition, add file system support for the user database file.

Alternatively, you can select **INCLUDE_AD_LDAP_AUTH** to include support for Active Directory (AD)/Lightweight Directory Access Protocol (LDAP) authentication. This component requires an AD/LDAP authentication server; a local user database is not required. For more information, see AD/LDAP User Authentication on page 69.

A third option is to add your own customized authentication method to the back end. For more information, see Adding a Custom User Database on page 71.

For the shell and login facility, add the **INCLUDE_SHELL** and the **INCLUDE_SHELL_SECURITY** components.

For additional configuration options, see the topics for the specific features of user authentication and management.

3. User Database

The user database is a text file that stores user account records.

WNDRVR

If it does not already exist, database is created automatically when the first user logs on and is prompted to create their password. Once the database is created, it is maintained across reboot and on/off cycles. A maximum of 65,534 users are permitted. The user database facility is provided with the **INCLUDE_USER_DATABASE** component.

The user database manager uses a hashed message authentication code (HMAC) to detect tampering with the user database file. The HMAC is calculated over the entire user database file. If the HMAC does not validate, a configurable callback is called. A custom callback can be installed that takes the appropriate action. The default callback disables logins.

Location Configuration

The database should be stored on a non-volatile storage device. This storage device should have a configurable file system, making it possible for the manager of the user database to access it through regular file I/O operations.

The location at which the user database is created (automatically) is determined by the **UDB_STORAGE_PATH** configuration parameter, which is associated with the **INCLUDE_USER_DATABASE** component. This parameter's default value is NULL, so it must be set to a valid path, and the file name.

For example, assuming use of the HRFS file system on a USB flash drive that is automatically mounted (**/bd0**), and that the user database file should be named **vxUserDB.txt**, then the following would be assigned to **UDB_STORAGE_PATH**:

"/bd0/vxUserDB.txt"

CAUTION: **If you use a mass storage device to store the user database text file, take into consideration that the storage device may not be ready when the user database manager is initialized. If the mass storage device is not up by the time the user database manager is initialized, the system displays an error message such as Cannot find user database! and prevents you from adding other users or changing passwords. Once the system has fully booted and the storage device is ready, you can use the database.**

Hash Key Configuration

You must specify the secret hash key for computing the HMAC with the **UDB_HASH_KEY** and **UDB_HASH_KEY_LEN** configuration parameters. The default value for the parameter **UDB_HASH_KEY_LEN** is 256. The hash key should have random bytes encoded in *\xnn* format, where *nn* can vary from 00 to ff. The longer the hash key, the higher the quality of cryptographic hashing.

For example, the following could be assigned to **UDB_HASH_KEY** (without line breaks):

```
"\x48\x61\x6d\x6c\x65\x74\x2e\x20\x54\x6f\x20\x62\x65\x2c\x20\x6f\x72\x20\x6e\x6f\x74\x20\x74\x6f\
x20\x62\x65\x2d\x20\x74\x68\x61\x74\x20\x69\x73\x20\x74\x68\x65\x20\x71\x75\x65\x73\x74\x69\x6f\x6
e\x3a\x0a\x57\x68\x65\x74\x68\x65\x72\x20\x27\x74\x69\x73\x20\x6e\x6f\x62\x6c\x65\x72\x20\x69\x6e\
x20\x74\x68\x65\x20\x6d\x69\x6e\x64\x20\x74\x6f\x20\x73\x75\x66\x66\x65\x72\x0a\x54\x68\x65\x20\x7
3\x6c\x69\x6e\x67\x73\x20\x61\x6e\x64\x20\x61\x72\x72\x6f\x77\x73\x20\x6f\x66\x20\x6f\x75\x74\x72\
x61\x67\x65\x6f\x75\x73\x20\x66\x6f\x72\x74\x75\x6e\x65\x0a\x4f\x72\x20\x74\x6f\x20\x74\x61\x6b\x6
5\x20\x61\x72\x6d\x73\x20\x61\x67\x61\x69\x6e\x73\x74\x20\x61\x20\x73\x65\x61\x20\x6f\x66\x20\x74\
x72\x6f\x75\x62\x6c\x65\x73\x2c\x0a\x41\x6e\x64\x20\x62\x79\x20\x6f\x70\x70\x6f\x73\x69\x6e\x67\x2
0\x65\x6e\x64\x20\x74\x68\x65\x6d\x2e\x20\x54\x6f\x20\x64\x69\x65\x2d\x20\x74\x6f\x20\x73\x6c\x65\
x65\x70\x2d\x0a\x4e\x6f\x20\x6d\x6f\x72\x65\x3b\x20\x61\x6e\x64\x20\x62\x79\x20\x61\x20\x73\x6c\x6
5\x65\x70\x20\x74\x6f\x20\x73\x61\x79\x20\x77"
```

CAUTION: **For security reasons, do not use the hash key provided in this example. It is for illustrative purposes only. Using this hash key may compromise the security of your system.**

CAUTION: **If the INCLUDE_SECURITY component is included in a VIP, but the UDB_STORAGE_PATH or UDB_HASH_KEY parameters are left set to null, the user database manager will not be initialized properly and the user database will not be created.**

User Database Creation

The user database does not initially exist until the first user (considered the initial user) logs in, at which point it is created automatically by the user management facility at the location specified by the **UDB_STORAGE_PATH** configuration parameter.

The configuration parameter **UDB_PROMPT_INITIAL_USER** controls whether or not the console prompts for the initial user's userid and password. By default this parameter is set to FALSE and should be enabled if you want the initial user to be prompted to provide a password. The system does not challenge the initial user. Any users who subsequently attempt to log on are challenged, because it is assumed that the initial user is the "administrator," whose task is to set up the other users' accounts, including their initial passwords.

Note: If you enable login by including the components **INCLUDE_USER_IDENTIFICATION** and **INCLUDE_USER_DATABASE** without enabling the parameter **UDB_PROMPT_INITIAL_USER**, you must have installed a user database file as part of the production process. Otherwise you will not be able to login to your system.

Steps that can be followed to create a user database file on a deployed device:

1. Create and build a VSB with the **USER_MANAGEMENT** layer enabled.
2. Create a VIP based on your VSB with the components **INCLUDE_USER_IDENTIFICATION** and **INCLUDE_USER_DATABASE** included. Enable the parameter **UDB_PROMPT_INITIAL_USER** and build the image.
3. Boot the board/device with the VIP image.
4. Login though the shell and you will be prompted for a password as the initial user. Then the initial user database file is created automatically by the user management facility at the location specified by the **UDB_STORAGE_PATH** configuration parameter. Save the user database file to your host machine (for example you can transfer it using FTP or SFTP).
5. In the production process, along with the software and configuration data you must also install the saved user database file from the host machine to the device that will be deployed.

User Database Management

The following functions can be used for database management.

Function	Description
udbInfoGet( )	Get information about the user database.
udbReset( )	Reset the user database.
udbInfoShow( )	Show information about the user database

For more formation about these and additional functions, see the **udbMgr** library reference.

4. User Account Management

The user account management facility provides for managing user and group accounts, for blocking (and unblocking) user access, and for monitoring the source of logins (console, SSH, and so on). It also provides a framework for implementing custom login and logout policy enforcers.

WNDRVR

When a user account is created, a unique user ID and group ID are associated with it, either automatically or explicitly. If they are not specified by the account creator, the user management facility automatically assigns a unique user ID (starting with 2) and a default group ID (100). The default group ID is set with the **DEFAULT_PRIMARY_GROUP_ID** configuration parameter.

Initial User

If the shell login facility is included in the system (with **INCLUDE_SHELL_SECURITY**), the first user to log on (or the first to log on when the user database is empty) is considered the *initial user*. The parameter UDB_PROMPT_INITIAL_USER controls whether or not the console prompts for the initial user's userid and password, if the user database file does not exist. By default this parameter is set to FALSE. Enabling prompting simplifies startup, but it can also provide an attack surface for an attacker. If the attacker is able to remove the user database file from the physical media, the console will prompt for the initial user at the next reboot. The initial user is prompted for their password as part of logging on, and assigned the user ID of 2 (zero is not valid and 1 is reserved for the root user). The initial account is essentially an "administrator" account, as it is used to create all other accounts.

If the shell login facility is not included, the initial user account is not automatically created, and all users must be registered from the shell with the useradd command (with the command interpreter), or programmatically with the userAdd( ) function.

CAUTION: **For security reasons, do not deploy a system without having created an initial user. Otherwise, whoever logs on first will get full access to the system.**

Additional Users

All additional user accounts should be created by the initial user. By default, the system automatically sets the user ID associated with each newly registered user. The value of the user ID is an increment of 1 from the previously assigned user ID. The user ID cannot be changed. Use the useradd command or the userAdd( ) function to create a new user account.

When you create any user account, you must provide a password to associate with that account. The user can update this password later. For example:

```
-> userAdd "moimoi","foo_passwd"
```

```
-> userPasswordUpdate "moimoi", "foo_passwd", "foo_new_passwd"
```

Root User

A root user (super user) can be created, using the user ID of 1 and the group ID of 1. For example:

```
-> userAdd "root","root_passwd", 1, 1
```

The name **root** is merely a useful convention that has no functional significance.

Creation of a root user is only significant when restrictions are placed on user privileges (see Defining User Privileges on page 68), in which case the root user has permissions for all commands.

User Account Management Facilities

The following functions and commands can be used to manage user accounts from kernel and RTP applications, or from the kernel shell. Use the commands from the shell's command interpreter, and the functions programmatically.

Function	Command	Description
userAdd( )	useradd	Create a new user account.
userDelete( )	userdel	Delete a user account.
userAuthenticate( )	-	Authenticate a user name and password.
userLogout( )	-	Report that a user has logged out.
userPasswordUpdate( )	passwd	Change a password.
userGroupUpdate( )	user mod -g	Update the group ID for an existing user.
userLastLoginTimeGet( )	-	Get time of last login for a user.
userLastLoginTimeSet( )	-	Set last login time of a user.
userLastLoginUpdate( )	-	Set last login time for the given login name to the current time.
userLastPwChangeTimeSet( )	-	Get the last-pasword-change time for the given login name.
userNameUpdate( )	-	Update the login name for an existing user.
userEnableSet( )	user mod -d user mod -e	Set the enable flag for a user in the user database.
userEnableGet( )	-	Get the enable flag for a user from the user database.
userAuthBlock( )	-	Block user authentication.
userAuthUnblock( )	-	Unblock user authentication.
getpwuid( )	-	Search the user database for a user ID.
getpwuid_r( )	-	Search the user database for a user ID (re-entrant version).
getpwnam( )	-	Search the user database for a name.
getpwnam_r( )	-	Search the user database for a name (reentrant version).
pwValidGet( )	-	Get password valid status.
userDefinedGet( )	-	Get user defined status.

WNDRVR

For more information see the **userIdentLib** and **userMgtShellCmd** API references.

User Login Context Monitoring

In addition to the functions (and shell commands) that are available in both kernel and user-mode variants for general user account management (see above), additional functions can be used in kernel-mode applications for monitoring the source from which login attempts are made (for example, a serial port, an SSH session, and so on).

Function	Description
loginCtxInit( )	Initialize login context.
loginCtxInterfaceSet( )	Set the login interface ID.
loginCtxInterfaceGet( )	Get the login interface ID.
loginCtxFdSet( )	Set the login file descriptor.
loginCtxFdGet( )	Get the login file descriptor.
userCtxAuthenticate( )	Authenticate a user by name and password.

For more information see the **userIdentLib** API reference.

For examples of implementation, see **maxFailedLoginPolicy.c** and **unusedAccountPolicy.c** in *installDir* **/vxworks-7/pkgs_v2/security/user_management/user_management_policy/src**.

Custom Login and Logout Policy Enforcers

The user account management facility provides a framework that can be used to implement custom login and logout policy enforcers. Policy enforcers are implemented as callbacks that are invoked every time a login or logout request is received. They can be added with loginPolicyEnforcerAdd( ) and removed with loginPolicyEnforcerDelete( ). For detailed information, see the **loginPolicyLib** API reference.

5. Password and Login Policies

Additional user management components can be used to define password and login policies, such as password complexity and login timeouts.

VSB Configuration

For password and login policy support, add the **USER_MANAGEMENT** layer to your VSB project.

Password Policy VIP Configuration

INCLUDE_PASSWORD_POLICY

Provides support for password policy enforcement.

INCLUDE_PW_LENGTH_POLICY

Provides for a minimum and maximum password-length policy. The configuration parameters are:

- **MIN_PW_LEN** controls the minimum password length that will be accepted by the user management module. Setting this parameter to 0 disables the minimum length check.
- **MAX_PW_LEN** controls the maximum password length that will be accepted by the user management module. Setting this parameter to 0 disables the maximum length check.

INCLUDE_PW_COMPLEXITY_POLICY

Provides for configuring a minimum number of lower case, upper case and special characters that must be used in a password. The configuration parameters are:

- **MIN_LOWER_CASE** controls minimum number of lower case characters in a password that will be accepted by the user management module. Setting this parameter to 0 disables the requirement for lower-case characters in the password.
- **MIN_UPPER_CASE** controls minimum number of upper-case characters in a password that will be accepted by the user management module. Setting this parameter to 0 disables the requirement for upper case characters in the password.
- **MIN_NUMBER** controls minimum number of numbers in a password that will be accepted by the user management module. Setting this parameter to 0 disables the requirement for a number in the password.
- **MIN_SPECIAL** controls minimum number of special characters in a password that will be accepted by the user management module. Setting this parameter to 0 disables the requirement for special characters in the password.

Login Policy VIP Configuration

INCLUDE_LOGIN_POLICY

Provides support for login policy enforcement.

INCLUDE_UNUSED_ACCOUNT_POLICY

Provides for automatically disabling unused accounts after a (configurable) timeout. An account is considered unused when no successful login has been performed for the account. The configuration parameter is:

- **UNUSED_ACCOUNT_TIMEOUT** controls the number of seconds that an account can be unused before it is disabled. Setting this parameter to 0 will disable automatic disabling of unused accounts.

INCLUDE_MAX_FAILED_LOGIN_ATTEMPTS_POLICY

Provides for configuring a maximum allowable number of failed login attempts during a specified time interval. If the maximum number of failed login attempts is exceeded during the specified time interval, the account will be temporarily disabled. The account will be enabled again when no login attempts have been made during the specified time interval. The configuration parameters are:

- **MAX_FAILED_LOGIN_ATTEMPTS** defines the number of unsuccessful login attempts that can be made before the account is temporarily disabled. Setting this parameter to 0 disables this policy.
- **MAX_FAILED_LOGIN_ATTEMPTS_TIME_INTERVAL** defines the time interval during which the number of unsuccessful login attempts is counted. The time interval is measured in minutes.

Custom Login and Password Policies

You can develop custom login and password policies for your own use, in addition to the policies provided by Wind River.

Custom policy enforcers must use the LOGIN_POLICY_ENFORCER_CB( ) and PW_CHANGE_POLICY_ENFORCER_CB( ) functions as signatures. These functions are declared in *installDir* **/vxworks-7/pkgs_v2/security/user_management/user_management_policy/h/userMgntPolicy.h**.

The pwValidGet( ) and userDefinedGet( ) functions can be used in a login policy hook to get information about login and password change requests (they cannot be used in a password policy hook).

Use the loginPolicyEnforcerAdd( ) and pwPolicyEnforcerAdd( ) functions to register your policies.

For more information about the functions that you can use in your implementation, see the **loginPolicyLib** and **pwPolicyLib** API reference entries.

6. Defining User Privileges

By default, user privileges for the kernel shell (as well as FTP and SFTP) are not limited. They can, however, be limited by defining execution rights for elements of these services on a user or group basis with the user privileges facility.

About This Task

A privileges manifest file defines user privileges. Among other default restrictions defined in the manifest, the kernel shell's C interpreter is disabled and the command interpreter must be used until the manifest is edited. This provides a basic level of initial security by preventing unlimited access to all kernel functions.

Note that the root user is not subject to any restrictions, regardless of the privileges manifest.

For the security of a deployed system, the privileges manifest file can be stored in ROMFS (a read-only file system; see the *VxWorks 7 File Systems and I/O Programmer's Guide:Overview of ROMFS*.) During development it can be stored in the host's file system.

To define user privileges, perform the following operations:

Procedure

1. Create a VSB and select the **USER_MANAGEMENT** layer.
2. Build the VxWorks source code.
3. Create a VIP based on your VSB.
4. Add the following components:
   - **INCLUDE_USER_IDENTIFICATION**
   - **INCLUDE_USER_DATABASE**
   - **INCLUDE_SHELL**
   - **INCLUDE_SHELL_SECURITY**
5. For a deployed system also add **INCLUDE_STANDALONE_SYM_TBL**.

   This will link the symbol table with the kernel.

6. Set the **UDB_STORAGE_PATH** parameter to the location of the user database.

   See User Database on page 61.

7. Set the **UDB_HASH_KEY** and **UDB_HASH_KEY_LEN** parameters to the hash key and length of the hash key.

   See *VxWorks 7 Security Guide: User Database on page 61*.

8. Build VxWorks.
9. Boot VxWorks and register all the users who are expected to log on to the system.

See *VxWorks 7 Security Programmer's Guide: User Account Management on page 63*.

10. Add **INCLUDE_USER_PRIVILEGES** to your VIP.
11. Rebuild VxWorks.

    This creates a **privilege_manifest** directory in the VIP, which contains the default privilege manifest file, **prvlgManifest.txt**.

12. Edit the **prvlgManifest.txt** file so that it defines the appropriate privileges for the users. For instructions in this regard, see the comments at the top of the file.

    If you wish to retain a copy of the **prvlgManifest.txt** file after deleting the VIP project, make a copy of it outside of the project.

13. If you are going to store **prvlgManifest.txt** in the ROMFS file system (which is recommended for a deployed system):
    - Add **INCLUDE_ROMFS** to the VIP.
    - Add **prvlgManifest.txt** to **/romfs**.

    By default the **PRIVILEGE_MANIFEST_PATH** parameter is set to **/romfs/prvlgManifest.txt**, so it does not have to be changed.

14. If you are not going store **prvlgManifest.txt** in ROMFS:
    - Copy it to the appropriate location on the target or host system.
    - Set the **PRIVILEGE_MANIFEST_PATH** parameter to the location of the file. (By default it is set to **/romfs/prvlgManifest.txt**.)
15. Rebuild VxWorks.

    For additional information, see the **privilegesLib** API reference entry, which also describes the privilegesCheck( ) function.

7. Custom Tampering Policy

You can develop a custom tampering policy handler for use with the user database.

The information in the user database is protected through an HMAC for each record in the user database (see User Database on page 61). Because the HMAC is computed over the entire user database file, any tampering can be detected. If there is a discrepancy, the tampering policy handler is executed. The default handler blocks user authentication and logs tampering detection events to the error detection and reporting log, if this feature is included in the system.

If you want to implement a custom tampering policy handler, use the **INCLUDE_UDB_CUSTOM_TAMPERING_POLICY** component. For detailed information, see the **udbMgr** API reference.

8. AD/LDAP User Authentication

VxWorks can be configured with support for Active Directory (AD) and Lightweight Directory Access Protocol (LDAP) authentication.

AD/LDAP authentication uses a central authentication server that stores and manages user login data. VxWorks authenticates a user by communicating with the authentication server. The AD/LDAP server connection parameters can be configured both statically and at runtime.

This LDAP authentication support is designed for use with both OpenLDAP and Windows AD servers. StartTLS must be enabled in the LDAP/AD server.

WNDRVR

VSB Configuration

To enable AD/LDAP authentication in your VxWorks system, you must enable the **USER_MANAGEMENT** layer in your VxWorks source build.

VIP Configuration

Use the **SELECT_USER_AUTHENTICATION_BACKEND** component to select an authentication back end. By default, **INCLUDE_LOCAL_AUTH** component is selected.

Use the following parameters to customize this support for your system:

Parameter	Description
AD_LDAP_HOST	Defines the AD/LDAP server address which can be an IPv4/IPv6 address or a DNS name. The default value is **127.0.0.1**. You must customize this value for your LDAP server.
AD_LDAP_PORT	Defines the listening port of the AD/LDAP server. The default value is 389 for LDAP.
AD_LDAP_BASEDN	Defines the base distinguished name (DN) used when searching for users. For example, **dc=my-domain,dc=com** and **dc=my-ad,dc=com**. You must change this value to the base DN for your LDAP server.
AD_LDAP_MANAGER_DN	Defines the manager distinguished name (DN) which is used to bind to the LDAP server that is used to search for the user's DN. The manager DN must have permission to search for the user in the base DN. If this parameter is left null, the system attempts an anonymous bind. If your LDAP server disables anonymous bind and you do not customize this value, the anonymous bind fails.
AD_LDAP_MANAGER_DN_PASSWD_ID	The AD/LDAP manager DN password should be stored in a security vault. This parameter defines the vault ID. If it is left null, the system attempts to use an anonymous bind.

The **INCLUDE_AUTH_CACHE_USER** component allows you to cache a remotely authenticated user into a local user database. This support allows a remotely authenticated user to log in when disconnected from the authentication server. This support requires the user database facility (see User Database on page 61).

Use the following parameters to customize this support for your system:

Parameter	Description
CACHE_USER_LIFETIME	Specifies the number of days that user information can be cached locally. A value of 0 indicates that the user information can be cached indefinitely. A cached user cannot be logged in locally once the specified cache lifetime is expired. The cache lifetime is reset each time a

Parameter	Description
	successful login is performed against the remote LDAP server.
MAX_CACHED_USERS	Specifies the maximum number of cached users allowed locally. A value of 0 indicates that there is no limit to the number of locally cached users. When the maximum number of cached users is reached, the oldest cached user is replaced.

Use the **INCLUDE_AD_LDAP_AUTH_CMD** component to include support for the auth ldap command. This command is used to configure LDAP configuration parameters at run time. To display help information for this command, use help auth ldap.

Runtime Configuration with userAuthLdap APIs

The **userAuthLdap** APIs can be used to at runtime to re-configure the server connection parameters that are defined statically with the **INCLUDE_AD_LDAP_AUTH** component parameters.

Function	Desription
userAuthLDAPGetHost( )	Get the LDAP server address.
userAuthLDAPGetPort( )	Get the LDAP listening port number.
userAuthLDAPGetBaseDN( )	Get the LDAP base distinguished name (DN).
userAuthLDAPGetManagerDN( )	Get the LDAP manager distinguished name (DN).
userAuthLDAPGetManagerDNPassword( )	Get the LDAP manager distinguished name (DN) password.
userAuthLDAPSetHost( )	Set the LDAP host server address.
userAuthLDAPSetPort( )	Set the LDAP listening port parameter.
userAuthLDAPSetBaseDN( )	Set the LDAP base distinguished name (DN).
userAuthLDAPSetManagerDN( )	Set the LDAP manager distinguished name (DN).
userAuthLDAPSetManagerDNPassword( )	Set the LDAP manager distinguished name (DN) password.

9. Adding a Custom User Database

Customized authentication method involves creation of a custom user database with custom login capabilities and a corresponding CDF component for the user database.

Procedure

1. Implement your custom user database, using the following interfaces:

```
typedef STATUS (* USER_CTX_AUTH_HOOK) (const char * loginName, const char * clearPassword, uid_
t * pUserId, gid_t * pGroupId, LOGIN_CTX* ctx);
```

```
typedef STATUS (* USER_LOGOUT_HOOK) (const char * loginName, LOGIN_CTX* ctx);
```

   For example :

```
STATUS myUserDbAuth (const char * loginName, const char * clearPassword, uid_t * pUserId, gid_
t * pGroupId, LOGIN_CTX* ctx);
```

```
STATUS myUserDbLogout (const char * loginName, LOGIN_CTX* ctx);
```

2. Create a CDF component for the custom user database.

   You can create a CDF component like **INCLUDE_MY_USER_DB**.

3. Add the CDF component **INCLUDE_MY_USER_DB** to authentication back end **SELECT_USER_AUTHENTICATION_BACKEND**.

   For example :

```
Component INCLUDE_MY_USER_DB {
        …
        _CHILDREN SELECT_USER_AUTHENTICATION_BACKEND
        INIT_RTN myUserDbInit();
        …
}
```

4. Register myUserDbAuth( ) and myUserDbLogout( ) as the authentication and logout hooks respectively in the **INIT_RTN**.

   For example :

```
myUserDbInit(void)
{
     userCtxAuthHookSet(myUserDbAuth);
     userLogoutHookSet(myUserDbLogout);

     /*user authentication could return uid and gid or not*/
     returnUidGid = TRUE;
}
```

Note: The flag **returnUidGid** indicates whether the authentication function e.g. myUserDbAuth( ) will set the user ID (uid) and group ID (gid) or not. By default uid and gid will be set . But if myUserDbAuth( ) does not set uid and gid, then set the flag **returnUidGid** to FALSE in myUserDbInit( ) routine.

WNDRVR

10. Login Banner

An additional user management component can be used to create user-defined login banners. Login banners can be used to meet cyber security requirements as defined by some regulatory and standards organizations.

Login Banner VSB Configuration

For login banner support, add the USER_MANAGEMENT layer to your VSB project.

Login Banner VIP Configuration

INCLUDE_LOGIN_BANNER

This component allows a textual banner to be output wherever a login prompt is presented to the user.

LOGIN_BANNER_TEXT

Provides the user-defined banner text string. To create a new line in the output insert **\r\n** in the text string.

WNDRVR

11. LOOPFS, IMAGE FILES, AND ENCRYPTED CONTAINERS

1. Loop File System and Tcplay

The loop file system (LoopFs) is a facility that mounts an image file as a virtual block device.

With LoopFs, the contents of the file are treated as the virtual block device's sector data. Data is mapped one-to-one. If the image file contains a valid layout of file system format, such as DOSFS, HRFS or CDROMFS (ISO image), the new virtual block device is mounted by the appropriate file system driver.

The tcplay tool creates encrypted containers.

The LoopFs file system can be configured with support for mounting image files that have been encrypted with the VxWorks version of tcplay, which is partially compatible with the FreeBSD version of tcplay, and with the GPL TrueCrypt facility. VxWorks tcplay uses the Advanced Encryption Standard (AES) specification for encryption based on OpenSSL. It currently supports AES-128-XTS and AES-256-XTS for encryption and or decryption, and supports multi-key files and hidden volume, like tcplay for other hosts. The VxWorks API provides functions that can be used programmatically or from the shell (the shell command tcplay is not supported).

The LoopFs file system also provides an extension for mounting an image file with a non-flat layout (that is, a mapping that is not one-to-one). Making use of this extension requires that you write XBD access functions, custom XBD service task functions and virtual block device parameters. The loopFsMountFile( ) function takes additional parameters for virtual block devices to support this functionality (see the API reference for more information).

Mount ISO image with LoopFs

LoopFs can be used for mounting an ISO image (ISO 9660 image). To enable ISO mounting, you must enable **FS_CDROMFS** layer in VSB, and enable **INCLUDE_CDROMFS** and **INCLUDE_LOOPFS** in VIP.

After system start up, use loopFsMountImgFile( ) to mount ISO image.

```
-> loopFsMountImgFile "/bd16a/sysroot.iso", "/cdrom1"
value = -140737482209840 = 0xffff8000005dc5d0 = _sysTableEnd + 0xffff8000001ce5d0
-> ll "/cdrom1"

Listing Directory /cdrom1:
drwxr-xr-x  0 0       0                2048 Jan  2 2020 ./
drwxr-xr-x  0 0       0                2048 Jan  2 2020 ../
drwxr-xr-x  0 0       0                2048 Jan  2 2020 lib/
drwxr-xr-x  0 0       0                2048 Jan  2 2020 usr/
value = 0 = 0x0
```

Configuring VxWorks With loopFs and Encrypted Image Support on page 75 To configure VxWorks with support for loopFs, and for mounting a tcplay-encrypted image file, you must select the required options and components for your VSB and VIP projects.
Mounting and Unmounting an Image File on page 75 To mount and unmount an image file, use APIs from the the **loopFsLib** library.
Working With an Encrypted Image File on page 76 Common operations used in working with an encrypted image file involve mounting the image, displaying information about mounted tcplay device, and un-mounting the tcplay volume.
Adding Support for Auto-Mounting Encrypted Images on page 77 If you have configured VxWorks with the loopfs file system and tcplay, you can provide additional support for automatically mounting encrypted images.

WNDRVR

1.1. Configuring VxWorks With loopFs and Encrypted Image Support

To configure VxWorks with support for loopFs, and for mounting a tcplay-encrypted image file, you must select the required options and components for your VSB and VIP projects.

About This Task

Procedure

1. Create a VSB project.
2. For the loopFs file system, select the **BDM** layer.
3. For mounting a tcplay-encrypted image file, also select **OPENSSL** and **FSAPI_TCPLAY**.
4. Build the source code.
5. Create a VIP project based on your VSB project.
6. For loopFs file system support, add **INCLUDE_LOOPFS**.
7. For support for mounting a tcplay-encrypted file system, add **INCLUDE_TC_DRIVER**.
8. To automatically unmount inaccessible images, set the **TC_AUTO_UNMOUNT** parameter to **TRUE**. (If an image is located on a removable device, the image is inaccessible when the device is unplugged.)

Parent topic: Loop File System and Tcplay on page 74

1.2. Mounting and Unmounting an Image File

To mount and unmount an image file, use APIs from the the **loopFsLib** library.

About This Task

Procedure

1. To mount an image file as a virtual block device, use the loopFsMountImgFile( ) function.

   For example:

```
-> ls "/bd0"
/bd0/VXBOOT.IMG
/bd0/FOO.TC
value = 0 = 0x0
-> loopFsMountImgFile "/bd0/VXBOOT.IMG", "/loop0"
value = 31252640 = 0x1dce0a0
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 host:
  3 /bd0
  3 /loop0
value = 0 = 0x0
-> ls "/loop0"
/loop0/README.TXT
/loop0/BOOTROM.SYS
```


WNDRVR

```
value = 0 = 0x0
->
```

2. To unmount an image file as a virtual block device, use the loopFsUnmount( ) function.

   For example:

```
-> loopFsUnmount 0x1dce0a0
value = 0 = 0x0
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 host:
  3 /bd0
value = 0 = 0x0
->
```

   For more information, see the **loopFsLib** library entry.

Parent topic: Loop File System and Tcplay on page 74

1.3. Working With an Encrypted Image File

Common operations used in working with an encrypted image file involve mounting the image, displaying information about mounted tcplay device, and un-mounting the tcplay volume.

About This Task

This example illustrates working with an encrypted image created by tcplay on another system.

Procedure

1. Display the contents of **/bd0**.

```
-> ls "/bd0"
/bd0/VXBOOT.IMG
/bd0/FOO.TC
value = 0 = 0x0
->
```

2. Mount the image with tcVolMount( ), and then display device information.

```
-> tcVolMount "/loop1", "/bd0/foo.tc", "miBadP@$$werd"
value = 0 = 0x0
->
-> devs
drv name
  0 /null
  1 /ttyS0
  1 /ttyS1
  8 host:
```


WNDRVR

```
  3 /bd0
  3 /loop1
value = 0 = 0x0
->
```

The third argument for tcVolMount( ) is the password.

3. Display the contents of **/loop1**, and then information about mounted tcplay device with tcVolShow( ).

   For example:

```
-> ls "/loop1"
/loop1/test
/loop1/2.txt
/loop1/1.txt
value = 0 = 0x0
->
-> tcVolShow "/loop1"
Device: /bd0/foo.tc
Cipher: AES-256-XTS
Key Length: 512 bits
Sector size: 512
Volume size: 1536 sectors
IV offset: 256
Block offset: 256
value = 0 = 0x0
->
```

   VxWorks must be configured with **INCLUDE_DOSFS_MAIN** to be able to use the ls command.

4. Un-mount the tcplay volume using the tcVolUnmount( ) function.

   For example.

```
-> tcVolUnmount "/loop1"
value = 0 = 0x0
-> devs
drv name
0 /null
1 /ttyS0
1 /ttyS1
8 host:
3 /bd0
value = 0 = 0x0
```

Parent topic: Loop File System and Tcplay on page 74

1.4. Adding Support for Auto-Mounting Encrypted Images

If you have configured VxWorks with the loopfs file system and tcplay, you can provide additional support for automatically mounting encrypted images.

To use the auto-mount facility, you must also choose one of the following for password support:

- Plain-text passwords.

WNDRVR

- A custom de-encryption function with encrypted passwords.
- The IPCOM key database.

See also

- Configuring VxWorks With loopFs and Encrypted Image Support on page 75
- Working With an Encrypted Image File on page 76

Adding Auto-Mount Support on page 78 To add auto-mount support for encrypted images, you must configure VxWorks with the required component, set a parameter to the locations of one or more encrypted images, and set another parameter to the mount points for each image.
Using Plain-Text Passwords on page 79 If you choose to use plain-text passwords with auto-mount support for encrypted images, you must set the passwords parameter to plain text passwords, and the password converter parameter to NULL.
Using a Custom Password De-Encryption Function With Encrypted Passwords on page 79 If you choose to use a custom password de-encryption function with auto-mount support for encrypted images, you must write your function, make it accessible to your VIP project, and set VIP project parameters to the name of the function and its parameters (if required), as well as to the encrypted passwords.
Using the IPCOM Key Database Support on page 80 If you choose to use the IPCOM key database with auto-mount support for encrypted images, you must configure VxWorks with the required VSB layer and VIP component, and set the configuration parameters for the database to the required values.

Parent topic: Loop File System and Tcplay on page 74

Adding Auto-Mount Support

To add auto-mount support for encrypted images, you must configure VxWorks with the required component, set a parameter to the locations of one or more encrypted images, and set another parameter to the mount points for each image.

About This Task

Procedure

1. Add **INCLUDE_TC_AUTOMOUNT** to your VIP project:.
2. Set the **TC_PATHS** parameter to one or more locations of the encrypted images, separating each path string with a comma. You must use NULL as the last entry.

   For example:

   ```
   "/romfs/test1.tc", "/romfs/test2.tc", NULL
   ```

3. Set the **TC_MOUNTPOINTS** to the mount points for the images.

   Each image identified with **TC_PATHS** must have a corresponding mount point.

   For example:

   ```
   "/loop0", "/loop1"
   ```

Parent topic: Adding Support for Auto-Mounting Encrypted Images on page 77

Using Plain-Text Passwords

If you choose to use plain-text passwords with auto-mount support for encrypted images, you must set the passwords parameter to plain text passwords, and the password converter parameter to NULL.

About This Task

Procedure

1. In your VIP project, set **TC_PASSPHRASES** to the plain-text passwords used to access each of the encrypted images.

   For example:

   ```
   "123", "Abc123"
   ```

2. Set the **TC_PASSPHRASES_CONVERTER** parameter to NULL.

   The passwords are not encrypted, so no de-encryption function is used.

3. Rebuild VxWorks.

Parent topic: Adding Support for Auto-Mounting Encrypted Images on page 77

Using a Custom Password De-Encryption Function With Encrypted Passwords

If you choose to use a custom password de-encryption function with auto-mount support for encrypted images, you must write your function, make it accessible to your VIP project, and set VIP project parameters to the name of the function and its parameters (if required), as well as to the encrypted passwords.

About This Task

Procedure

1. Write your own function to de-encrypt the passwords that you will assign to the **TC_PASSPHRASES** parameter in your VIP project.

   An example password converter function called usrTcPassphraseConvertDemo( ) is provided in *installDir* **/vxworks-7/pkgs_v2/storage/tcplay/cfg/usrTcplay.c**. The function uses Base64 encoding for the passwords. It is provided for illustrative purposes only; do not use it in deployed systems.

2. Add your decryption function to your VIP, layer code, or library file.

   Your VIP project must be able to find the symbol name of the converter function.

3. Set **TC_PASSPHRASES_CONVERTER** (the type is function pointer) to the name of the converter function.
4. If your converter function takes a parameter, set **TC_PASS_CONVERT_PARAM** (the type is void *) to the parameter name.
5. Encrypt all the passwords and then assign them to the **TC_PASSPHRASES** configuration parameter. See Configuring VxWorks With loopFs and Encrypted Image Support on page 75.

   If you wish to try the usrTcPassphraseConvertDemo( ) example, use Base64 encoding for the passwords.

6. Rebuild VxWorks.

WNDRVR

Results

At runtime, the auto-mount facility passes the encrypted passwords to the converter function, which returns the de-encrypted passwords, which are then used to mount the encrypted image.

Parent topic: Adding Support for Auto-Mounting Encrypted Images on page 77

Using the IPCOM Key Database Support

If you choose to use the IPCOM key database with auto-mount support for encrypted images, you must configure VxWorks with the required VSB layer and VIP component, and set the configuration parameters for the database to the required values.

Before You Begin

Before using the IPCOM key database with the auto-mount feature, you should be thoroughly familiar with how the database itself is used. See VxWorks 7 Cryptography Libraries Programmer's Guide.

About This Task

Procedure

1. Add the **OPENSSL** layer to your VSB project, and rebuild it.
2. Close your VIP project, and then reopen it (to pick up the VSB change).
3. Add **INCLUDE_IPCOM_USE_KEY_DB** to your VIP project.
4. Select and configure a key-encrypting password provider.

   For example, if you are using the password obfuscator KEP provider, select **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**, and set the **SEC_VAULT_FILE_ROOT** and **SEC_VAULT_KEY_ENCRYPTING_PW** parameters.

   For more information about the password obfuscator KEP provider, as well as information about other KEP providers, see VxWorks 7 Cryptography Libraries Programmer's Guide.

5. Set the **TC_KEYDB_PATHS** parameter to one or more locations of the encrypted images, separating each path string with a comma. You must use NULL as the last entry.

   For example:

   ```
   "/romfs/test1.tc", "/romfs/test2.tc", NULL
   ```

6. Set the **TC_KEYDB_MOUNTPOINTS** to the mount points. Each image identified with **TC_KEYBD_PATHS** must have a corresponding mount point.

   For example:

   ```
   "/loop0", "/loop1"
   ```

7. Set **TC_KEYDB_IDS** to the key IDs in the key database.

   For example:

WNDRVR

```
"KeyID1", "KeyID2"
```

8. Rebuild and boot VxWorks.
9. Import the encrypted image's password as a keydb buffer and store it in the key database.

   For information about this process, see VxWorks 7 Cryptography Libraries Programmer's Guide.

Parent topic: Adding Support for Auto-Mounting Encrypted Images on page 77

WNDRVR

12. FULL DISK ENCRYPTION

1. VxWorks Full Disk Encryption

VxWorks full disk encryption provides "data at rest" protection for sector-addressable devices (like hard disks), using the XEX-AES algorithm.

The feature encrypts the data on specified partitions of the storage devices. It uses the VxWorks secure vault facility to store the symmetric encryption keys. The dosFs and HRFS file systems are supported for full disk encryption.

For information about the secure vault and encryption keys, see the VxWorks 7 Cryptography Libraries Programmer's Guide.

For information about dosFs and HRFS, see the VxWorks 7 File Systems Programmer's Guide.

2. Configuring VxWorks for Full Disk Encryption

To configure VxWorks for full disk encryption, add the **DISK_ENCRYPTION** layer to the VSB project, and the **INCLUDE_DISK_ENCRYPTION** and **INCLUDE_DISK_ENCRYPTION_TOOLS** components to the VIP project. Optionally, add components for automatically mounting encrypted disk partitions.

Procedure

1. Create a VSB project.
2. Add the **DISK_ENCRYPTION** layer.
3. Build the VSB project.
4. Create a VIP project based on the VSB project.
5. Add the **INCLUDE_DISK_ENCRYPTION** component.

   An error message then indicates that there is a problem with your configuration. This is because the **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR** component has been added automatically, and its **SEC_VAULT_KEY_ENCRYPTING_PW** configuration parameter has not been set.

6. Set the **SEC_VAULT_KEY_ENCRYPTING_PW** parameter.

   This password is used to encrypt keys in the vault and key store. It should be set to a random string of at least 40 characters, using both lower and upper case, numbers and special characters. There is no default value. The project cannot be built unless it is set.

7. Add the **INCLUDE_DISK_ENCRYPTION_TOOLS** component.

   This component provides the tools for encrypting, mounting, and decrypting a disk partition.

8. Optionally, enable automatic mounting of encrypted partitions.

   (Partitions can also be mounted on-demand, with the diskMountCrypto( ) function.)

   1. Add one or more of the four auto-mount partition configurations.
      - **INCLUDE_DISK_ENCRYPTION_AUTO_MOUNT_1**
      - **INCLUDE_DISK_ENCRYPTION_AUTO_MOUNT_2**
      - **INCLUDE_DISK_ENCRYPTION_AUTO_MOUNT_3**
      - **INCLUDE_DISK_ENCRYPTION_AUTO_MOUNT_4**

WNDRVR

2. For each component that you add, set its configuration parameters to specify the partition name and the partition key ID.

   For example, set **PARTITION_NAME_1** to **"/ata0:1"**, and **PARTITION_KEY_ID_1** to **"0"**.

   The partition name and ID that you set here are used with the diskEncrypt( ) function to encrypt the specified partition at runtime.

9. Select the Advanced Encryption Standard (AES) algorithm to use.

   The disk encryption module supports the AES 256 ECB (the default) and the AES 128 ECB algorithms. The components are **INCLUDE_SEL_AES_256_ECB** and **INCLUDE_SEL_AES_128_ECB**.

10. Set the **INCLUDE_SEC_SECRET** parameter **SEC_VAULT_FILE_ROOT** to the directory under which the vault directory structure will be created.

    The **INCLUDE_SEC_SECRET** is included automatically. The disk encryption feature uses the VxWorks secure vault to store the symmetric encryption keys.

11. Add file system support.

    The disk encryption feature supports the dosFs and HRFS file systems. For information about them, see the *VxWorks 7 File Systems and I/O Programmer's Guide: Overview of VxWorks Local File Systems*.

12. Add any storage device driver components required for your target.
13. Build VxWorks.

3. Disk Encryption and Use

The full disk encryption facility provides APIs for disk encryption, decryption, and mounting partitions. Standard I/O functions are used with encrypted partitions.

CAUTION: **You must create a secure vault, and import a valid key into the vault, before the storage device is mounted; otherwise the mount operation will fail. For example, if you want to mount /ata0:1 and the secure vault device is /flash, you must make sure that /flash is accessible and that a valid key has been imported into it before /ata0:1 is mounted. For information about the secure vault and encryption keys, see** Key Store and Secrets Repository on page 48**.**

Partition Encryption

Call diskEncrypt( ) to encrypt a partition. Because encryption performance is lower on some platforms (for example, ARM), the tool may take some time to finish if the partition is very large.

CAUTION: **Before encrypting, be sure that a valid file system has been instantiated on the partition. (dosFs or HRFS).**

Do not attempt to halt encryption while it is in process. Doing so will corrupt the data in the partition.

```
diskEncrypt ("/ata0:1", "myId0");
```

The first argument identifies the partition, and the second is the key ID associated with the partition (which is used to fetch the master key from the vault).

WNDRVR

If you have configured VxWorks with the automount facility, the diskEncrypt( ) arguments should match the partition name and key ID that you specified with the configuration parameters for the **INCLUDE_DISK_ENCRYPTION_AUTO_MOUNT**_*N* component.

After the execution of diskEncrypt( ), the drive is mounted automatically. You do not need to call diskMountCrypto( ) at this point.

After encryption, VxWorks cannot access the partition unless you have imported a valid key to the secure vault. In addition, other operating systems (such as Windows) cannot recognize the partition, and would need to format it (destroying the existing data) before using it.

Partition Decryption

Call diskDecrypt( ) to decrypt an encrypted partition.

```
diskDecrypt ("/ata0:1", "myId0");
```

The first argument identifies the partition, and the second is the key ID associated with the partition (which is used to fetch the master key from the vault).

CAUTION: **Be sure that the key ID and associated key are correct before calling the routine. Otherwise, the data on the partition will be corrupted. Do not attempt to halt decryption while it is in process. Doing so will corrupt the data in the partition.**

Partition Mounting

If you have not enabled auto-mounting, or if a mount operation has failed, you can call diskMountCrypto( ) to mount an encrypted partition at runtime (programmatically or from the kernel shell).

```
diskMountCrypto("/ata0:1", "myId0");
```

The first argument identifies the partition, and the second is the key ID associated with the partition (which is used to fetch the master key from the vault).

Note that after the execution of diskEncrypt( ), the drive is mounted automatically. You do not need to call diskMountCrypto( ) at that point.

Encrypted Partition Use

Encryption operations are transparent to the file system and applications. If an encrypted disk partition has been mounted successfully, you can create, read, and write a file; create or delete a directory; and format the partition as a normal partition by using the standard APIs (for example, open( ), close( ), read( ), and write( )).

WNDRVR

13. TRUSTED COMPUTING

1. VxWorks Trusted Computing Support

VxWorks supports the Trusted Computing Group's (TCG) Trusted Software Stack for the TPM 2.0 specification. It provides Trusted Platform Module (TPM) device drivers, and VxWorks API extensions.

TPM 2.0 Software Stack

VxWorks supports the TPM 2.0 Trusted Software Stack (TPM2.0-TSS) open-source implementation.

The VxWorks VSB layer **TPM2_TSS** provides code that adapts the open source stack to VxWorks. It includes VSB layer configurations, makefiles, and so on. This layer contains the open source projects **tpm2_tss**, **tpm2-tss-engine** and **tpm2-tools**.

tpm2-tss provides hundreds of user accessible methods for developing applications using TPM 2.0 hardware. It is provided for VxWorks application developers as both static and shared libraries.

tpm2-tss-engine implements a cryptographic engine for OpenSSL for TPM 2.0 using the **tpm2-tss**. It supports RSA encryption and signatures as well as ECDSA signatures. It is provided for VxWorks application developers as both static and shared libraries.

tpm2-tools provides the TPM 2.0 tools based on **tpm2-tss**. It is provided for VxWorks users as RTP applications.

API references and example use of these open source projects are available from many Internet based sources.

TPM Device Drivers

VxWorks supports the Intel TPM driver and SPI TPM driver.

TPM devices conforming to the TCG 2.0 specification support over 110 different commands. For detailed information, see the TCG's *Trusted Platform Module Library, Part 3: Commands*.

TPM Device Driver APIs

End user access to TPM drivers is provided with the I/O sub-system interface. The API consists of the open( ), close( ), read( ), write( ), and ioctl( ) methods.

Third-Party Documentation

You may find the following documentation useful:

- For general information about trusted computing, see the Wikipedia entry for the Trusted Computing Group, at en.wikipedia.org and the Trusted Computing Group (TCG) itself, at http://www.trustedcomputinggroup.org/.

2. Configuring VxWorks with TSS and TPM 2.0 Driver

Configuring VxWorks with TSS and a TPM 2.0 driver involves simple VSB and VIP project configuration. Among the binaries produced by the VSB build are the libraries **libtpm20tss.a**, **libtpm20tss.so.1**, **libtpm2-tss-engine.a** and **libtpm2-tss-engine.so.1** and all are required for TSS application development.

WNDRVR

Procedure

1. Create a VSB project based on the itl_generic BSP.

   Note: Both Intel and arm BSPs are supported for use with TSS and TPM 2.0.

2. Add the **TPM2_TSS** layer.
3. Build the VSB project.

   When you start the VSB build, the **tpm2-t22**, **tpm2-tss-engine** and **tpm2-tools**, code is retrieved from https://github.com/tpm2-software and installed in the project directory.

   The user mode and kernel mode static libraries **libtpm20tss.a** and **libtpm2-tss-engine.a** are created in the VSB in *vsbProjName* **/usr/lib** and *vsbProjName* **/krnl/cpuName**. TSS applications can be linked to both libraries.

   The user mode shared libraries **libtpm20tss.so.1** and **libtpm2-tss-engine.so.1** are created in *vsbProjName* **/usr/lib**.

4. Create a VIP project based on your VSB project.
5. Add the **DRV_IA_TPM** or **DRV_SPI_TPM** TPM device driver component, and configure as required.

   See VIP Components for TPM Driver on page 86.

6. If the version of your itl_generic BSP is 1.0.2.0 or earlier, manually add the required TPM entries to the device tree source file (**.dts**) for your BSP.

   Edit the **itl_generic.dts** device tree source file that is in your VIP project directory.

   Add this content to the device node declaration section:

```
iaTpm: PNP0C31{
  reg = <DRV_IA_TPM_INSTANCE_0_BASE_ADDR
            DRV_IA_TPM_INSTANCE_0_REG_INTERVAL>;
  };
```

   Then add this content within the **chosen{ }** block:

```
tpmConfig{
     tpmConfig0{
     node = <0>;
            ios-dev-name = DRV_TPM_INSTANCE_DEV_NAME;
            };
           };
```

7. Build VxWorks.

3. VIP Components for TPM Driver

INCLUDE_TPM2_OPENSSL_ENGINE is the VIP component for the TPM2-TSS cryptographic engine for Open SSL. It implements a cryptographic engine for Open SSL on Trusted Platform Module (TPM 2.0) using the **tpm2-tss** software stack that follows the Trusted Computing Group (TCG) TPM Software Stack (TSS 2.0). It uses the Enhanced System API (ESAPI) interface of the TSS 2.0 for downwards communication. It supports RSA encryption and signatures, and ECDSA signatures.

WNDRVR

DRV_SPI_TPM

The main component for the SPI TPM device driver.

Configuration parameters:

- **DRV_TPM_INSTANCE_DEV_NAME** defines the I/O system name for the first TPM device.
- **DRV_TPM_INTERRUPT_ENABLE** enables interrupt mode. When it is set to FALSE, this disables the interrupt mode and the TPM driver continues to work in polling mode. When set to TRUE, the TPM driver will try to enable interrupt mode. If this attempt fails, then the driver will operate in polling mode instead.

DRV_IA_TPM

The main component for the IA TPM device driver. It automatically includes the **DRV_IA_TPM_INSTANCE_0** component.

DRV_IA_TPM_INSTANCE_0

Initializes the first instance (zero) of the IA TPM device driver.

Configuration parameters:

- **DRV_IA_TPM_INSTANCE_0_BASE_ADDR** defines the memory mapping register base address for first TPM device at locality 0.
- **DRV_TPM_INSTANCE_DEV_NAME** defines the I/O system name for first TPM device.
- **DRV_IA_TPM_INSTANCE_0_REG_INTERVAL** defines the memory mapping register address interval for first TPM device at locality 0.

Note: To define additional driver instances, edit the **itl_generic.dts** file.

4. How to Configure and Use the TPM2 OpenSSL Engine

Configure the TPM2 OpenSSL Engine

To use the TPM2 OpenSSL engine in the kernel, you must add the component **INCLUDE_TPM2_OPENSSL_ENGINE** to the VIP. The TPM2 OpenSSL engine will be registered to the OpenSSL library. To then use the engine functions, you must call OpenSSL API functions. For example:

```
tpmEngine = ENGINE_by_id("tpm2tss");
if (tpmEngine != NULL)
    {
    if (ENGINE_init(tpmEngine))
        {
        pkey = ENGINE_load_private_key(tpmEngine, filePath, NULL, NULL);
        RSA_private_decrypt (num1, ptext1, ptext2, pkey->pkey.rsa, RSA_PKCS1_PADDING);
        }
    }
```

Use TPM2 OpenSSL Engine to Create Key Pairs

To use the TPM2 OpenSSL engine to create key pairs, the following files in *vsbProjName* **/usr/root/llvm/bin** are required:

- **libc.so.1**
- **libcurl.so.1**

- **libHASH.so.1**
- **libllvm.so.1**
- **libnet.so.1**
- **libOPENSSL.so.1**
- **libtpm20tss.so.1**
- **libtpm2-tss-engine.so.1**
- **libunix.so.1**
- **libzlib.so.1**
- **tpm2tss-genkey.vxe**

Your VIP must include the components **INCLUDE_SHL** and **INCLUDE_RTP** to ensure that the **tpm2tss-genkey.vxe** RTP launches successfully.

The following command creates the key pair on the VxWorks target:

```
[vxWorks *]# /romfs/tpm2tss-genkey.vxe -a rsa -s 2048 /ram0/mykey0 --tcti=:/tpm
Launching process '/romfs/tpm2tss-genkey.vxe' ...
Process '/romfs/tpm2tss-genkey.vxe'
(process Id = 0xffff8000002db8d0) launched.
[vxWorks *]# ls /ram0
/ram0/mykey0
[vxWorks *]#
```

For more detail about the command, use the --help option.

Use the TPM2 Tools

To use the TPM2 tools, the directory *vsbProjName* **/usr/root/llvm/bin** must contain the following files:

- **libc.so.1**
- **libcurl.so.1**
- **libHASH.so.1**
- **libllvm.so.1**
- **libnet.so.1**
- **libOPENSSL.so.1**
- **libtpm20tss.so.1**
- **libtpm2-tools.so.1**
- **libunix.so.1**
- **libzlib.so.1**

The following RTP executable files must also be available:

- **tpm2_activatecredential.vxe**
- **tpm2_certify.vxe**
- **tpm2_createpolicy.vxe**
- **tpm2_createprimary.vxe**
- **tpm2_create.vxe**
- **tpm2_dictionarylockout.vxe**
- **tpm2_encryptdecrypt.vxe**
- **tpm2_evictcontrol.vxe**

- **tpm2_getcap.vxe**
- **tpm2_getmanufec.vxe**
- **tpm2_getpubak.vxe**
- **tpm2_getpubek.vxe**
- **tpm2_getrandom.vxe**
- **tpm2_hash.vxe**
- **tpm2_hmac.vxe**
- **tpm2_listpersistent.vxe**
- **tpm2_loadexternal.vxe**
- **tpm2_load.vxe**
- **tpm2_makecredential.vxe**
- **tpm2_nvdefine.vxe**
- **tpm2_nvlist.vxe**
- **tpm2_nvreadlock.vxe**
- **tpm2_nvread.vxe**
- **tpm2_nvrelease.vxe**
- **tpm2_nvwrite.vxe**
- **tpm2_pcrevent.vxe**
- **tpm2_pcrextend.vxe**
- **tpm2_pcrlist.vxe**
- **tpm2_quote.vxe**
- **tpm2_rc_decode.vxe**
- **tpm2_readpublic.vxe**
- **tpm2_rsadecrypt.vxe**
- **tpm2_rsaencrypt.vxe**
- **tpm2_send.vxe**
- **tpm2_sign.vxe**
- **tpm2_startup.vxe**
- **tpm2_takeownership.vxe**
- **tpm2_unseal.vxe**
- **tpm2_verifysignature.vxe**

Your VIP must include the components **INCLUDE_SHL** and **INCLUDE_RTP** to ensure that the TPM2 tool RTPs launch successfully.

Run the RTP application as follows:

```
[vxWorks *]# /romfs/tpm2_createprimary.vxe -H e -g sha256 -G 0x1 -C /ram0/context.p_B1 --tcti=/rom
fs/libtpm20tss.so.1:/tpm0
Launching process '/romfs/tpm2_createprimary.vxe' ...
Process '/romfs/tpm2_createprimary.vxe'
(process Id = 0xffff80000036b2f0) launched.
ObjectAttribute: 0x00030072

CreatePrimary Succeed!Handle: 0x80000000

[vxWorks *]#
```

For more detail about this command, use the --help option.

14. SECURITY EVENTS

1. VxWorks Security Events

The security events facility generates security event-based warning messages based on conditions that indicate a potential security risk.

A security event is a condition that might be security-related (for example, a failure to login). A security event is not necessarily an error condition or problem. The default security events handler displays a warning message when an event occurs. For example:

```
WARNING: Security event 0x12345678
```

Custom Security Events Hander

When the **SECURITY_EVENT** layer is added to the VSB, the user configlette **usrSecEventHandler.c** is added to the VIP. The file provides a default security event handler that prints a warning message. You can create your own security event handler to deal with security events as required by your system, and configure VxWorks to use it instead of the default.

The prototype for an events handler is:

```
int secEventHandler
    (
    unsigned int  eventId, /@ security violation event ID @/
    ...                  /@ optional arguments - see individual events for details @/
    );
```

The function returns 0 when no action from the caller is expected; and returns non-zero (user defined) if the event was handled and an action is expected from the caller.

You can check for security events using the macros associated with each event, and optionally make use of callback arguments that are passed to the event handler. For example:

```
   switch (eventId)
        {
        /*
         * Event:      Unsuccessful FTP login attempt
         * Parameters: const char*  username
         *             int          socketFd
         */
        case SEC_EVENT_FTP_LOGIN_FAILED:
            /* :TODO: */
            break;

        /*
         * Event:      Successful FTP login
         * Parameters: const char*  username
         *             int          socketFd
         */
        case SEC_EVENT_FTP_LOGIN_OK:
            /* :TODO: */
            break;
```

```
        /*
         * Event:      FTP logout
         * Parameters: const char*  username
         */
        case SEC_EVENT_FTP_LOGOUT:
            /* :TODO: */
            break;

        /*
         * Event:      An attempt was made to access an I/O object for which the RTP does not hav
e permission
         * Parameters: const char*  rtpName
         *             const char*  deviceName
         *             cosnt char*  filename
         */
        case SEC_EVENT_IO_PERM_ERROR:
            /* :TODO: */
            break;

        /* Any unhandled event ends up here */
        default:
            break;
        }
```

Note: Event identifiers are defined in the header file for the library that they apply to. For example, **SEC_EVENT_FTP_LOGIN_FAILED**, **SEC_EVENT_FTP_LOGIN_OK** and **SEC_EVENT_FTP_LOGOUT** are defined in **ipftps.h** and **SEC_EVENT_IO_PERM_ERROR** is defined in **rtpPartLib.h**.

For descriptions of events, including macros, IDs, and callback parameters, see Security Events Reference on page 92

2. Configuring VxWorks With Security Events

To configure VxWorks with security events, add the **SECURITY_EVENT** layer to the VSB project.

Procedure

1. Create a VSB project.
2. Add the **SECURITY_EVENT** layer.
3. Build the VSB project.
4. Create a VIP project based on the VSB project.
5. Build VxWorks.

3. Security Events Reference

VxWorks Security events can be used for developing applications that respond to various conditions that may affect system security. By default, the event IDs are displayed in a warning message by the default handler when an event occurs.

SEC_EVENT_FTP_LOGIN_FAILED

Description	Unsuccessful FTP login attempt.
ID	0x00880001
Callback arguments	Username (char *) Socket file descriptor (int)

SEC_EVENT_FTP_LOGIN_OK

Description	Successful FTP login.
ID	0x00880002
Callback arguments	Username (char *) Socket file descriptor (int)

SEC_EVENT_FTP_LOGOUT

Description	FTP logout.
ID	0x00880003
Callback arguments	Username (char *)

SEC_EVENT_IO_PERM_ERROR

Description	An attempt was made to access an I/O object for which the RTP does not have permissions.
ID	0x00d40002
Callback arguments	Name of RTP (char *) See **objAccessCtrlLib.c** for possible values Device name (const char *) Filename (char *)

SEC_EVENT_IOS_FDRESERVE_FAILED

Description	An attempt to open a file failed because of the limit set on the number of open files.
ID	0x000d0001

Callback arguments	Task Id (int)

SEC_EVENT_LOADER_MODULE_LOAD_FAILED

Description	A secure load operation failed.
ID	0x000e0001
Callback arguments	errno (int) ELF file name (char *)

SEC_EVENT_MEM_QUOTA_EXCEEDED

Description	An attempt to allocate memory from an RTP failed because of the limit set on memory use.
ID	0x00d40004
Callback arguments	Name of RTP (char *) Type of memory (int), 1 for normal, 2 for shared Amount of memory already used (size_t) Requested amount of memory (size_t) Maximum amount of memory that can be used (size_t)

SEC_EVENT_OBJ_ACCESS_FAILED

Description	An attempt was made to access a kernel object for which the RTP does not have permission.
ID	0x00d40001
Callback arguments	Name of RTP (char *) Object name (const char *) Object type (_Vx_obj_access_type), see **objAccessCtrlLib.h** for possible values

SEC_EVENT_RLOGIN_LOGIN_OK

Description	Successful remote login.
ID	0x002b0004
Callback arguments	Username (char *) Socket file descriptor (int)

SEC_EVENT_RLOGIN_LOGOUT

Description	Remote logout.
ID	0x002b0006
Callback arguments	Username (char *)

SEC_EVENT_SHELL_EXECUTE_FAILED

Description	Execution of a shell command failed because of permission denial.
ID	0x00ad0001
Callback arguments	User name (char *) Command string (char *)

SEC_EVENT_SHELL_LOGIN_OK

Description	Successful Shell login.
ID	0x00360002
Callback arguments	Username (char *)

SEC_EVENT_SHELL_LOGOUT

Description	Shell logout.
ID	0x00ad0003
Callback arguments	Username (char *)

SEC_EVENT_SHELL_SET_DISPLAY_FAILED

Description	A command to set or display configuration variables failed because of permission denial.
ID	0x00ad0002
Callback arguments	User name (char *) Variable name (char *) Variable value (char *)

SEC_EVENT_SSH_LOGIN_FAILED

Description	Unsuccessful SSH login attempt.
ID	0x00d60001
Callback arguments	Username (char *) Socket file descriptor (int) Authentication method (char *), which can be "password", "publickey" or "hostbased"

SEC_EVENT_SSH_LOGIN_OK

Description	Successful SSH login.
ID	0x00d60002
Callback arguments	Username (char *) Socket file descriptor (int) Authentication method (char *) - "password", "publickey" or "hostbased"

SEC_EVENT_SSH_LOGOUT

Description	SSH logout.
ID	0x00d60003
Callback arguments	Username (char *)

SEC_EVENT_SYSCALL_PERM_ERROR

Description	An attempt was made to call an unauthorized API from a RTP.
ID	0x00d40003
Callback arguments	Name of RTP (char *) System call number (long int)

SEC_EVENT_TELNET_LOGIN_FAILED

Description	Unsuccessful Telnet login attempt.
ID	0x002b0001

Callback arguments	User name (char *) Socket file descriptor (int)

SEC_EVENT_TELNET_LOGIN_OK

Description	Successful Telnet login.
ID	0x002b0003
Callback arguments	Username (char *) Socket file descriptor (int)

SEC_EVENT_TELNET_LOGOUT

Description	Telnet logout.
ID	0x002b0005
Callback arguments	Username (char *)

SEC_EVENT_UDB_HMAC_ERROR

Description	The user database has been tampered with.
ID	0x00d00001
Callback arguments	None.

15. TRUSTED EXECUTION ENVIRONMENT WITH OP-TEE

1. VxWorks and OP-TEE

VxWorks can be used with the Trusted Execution Environment (TEE) provided by the OP-TEE implementation. OP-TEE provides a secure parallel execution environment that isolated from VxWorks by the ARM TrustZone security hardware.

The i.MX6 Sabre Lite board is used in the implementation examples provided for a system with VxWorks and OP-TEE.

OP-TEE

OP-TEE is an open-source implementation. It is not included with the VxWorks distribution, and you must install it yourself. This guide provides an overview of the installation and configuration process, including VxWorks-specific instructions. For detailed information about OP-TEE, see https://github.com/OP-TEE and other resources on the Internet.

OP-TEE Client Demos

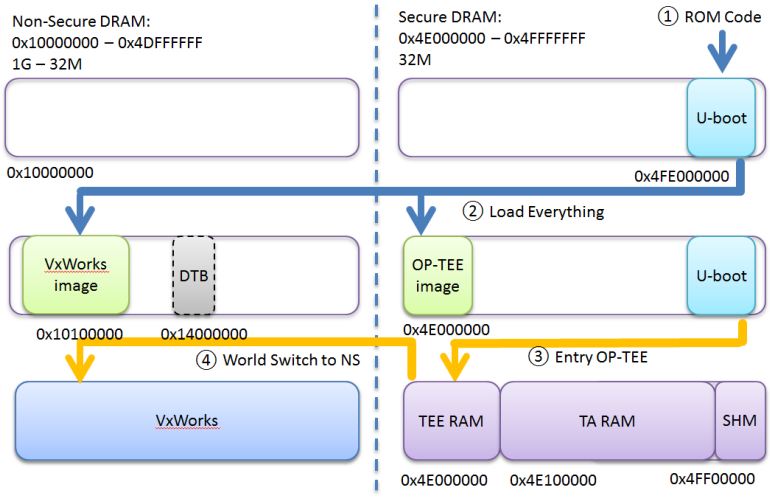
Two demos are provided. Each involves a VxWorks application that runs in the "normal world" and an OP-TEE application that runs in the "secure world," and that performs services in response to requests from the VxWorks application. One demo uses an OP-TEE trusted application to do encryption operations, and the other to provide secure storage.

System Memory

Because there are two different operating systems in different TEE "worlds" using the same CPU, the DDR SDRAM memory must be divided into "secure world" and "normal world" spaces, for OP-TEE and VxWorks, respectively.

Boot Process

1. After power on, the ROM code brings the board out of reset, and then loads and starts the boot loader (U-Boot).
2. The boot loader loads the VxWorks and OP-TEE binaries into memory.
3. The boot loader passes execution to OP-TEE.
4. OP-TEE carries out its own initialization in the "secure world," and then exits to the "normal world".
5. VxWorks boots in the "normal world."



2. Installing, Configuring, and Building OP-TEE

Install the required Linux packages, the OP-TEE source, and the build tools. Configure and build the OP-TEE OS. Optionally, build the trusted application demo code.

About This Task

OP-TEE must be built on a Linux host. These instructions are based on Ubuntu 14.04 use.

Procedure

1. Install the required Linux packages.

   For example:

```
$ sudo apt-get install android-tools-adb android-tools-fastboot autoconf bison \
cscope curl flex gdisk libc6:i386 libfdt-dev \
libglib2.0-dev libpixman-1-dev libstdc++6:i386 \
libz1:i386 netcat python-crypto python-serial \
python-wand uuid-dev xdg-utils xz-utils zlib1g-dev \
mtools libtool libftdi-dev libhidapi-dev \
u-boot-tools
```

2. Create the base OP-TEE directory and set an environment variable for it.

   For example:

```
$ bash
$ mkdir /home/xeno/optee
$ export OPTEE_HOME=/home/xeno/optee
```

3. Install the toolchains.

```
$ cd  $OPTEE_HOME
$ mkdir toolchains; cd toolchains
$ wget http://releases.linaro.org/archive/14.08/components/toolchain/binaries/gcc-linaro-arm-li
nux-gnueabihf-4.9-2014.08_linux.tar.xz
$ tar -Jxvf gcc-linaro-arm-linux-gnueabihf-4.9-2014.08_linux.tar.xz
```

4. Install the OP-TEE OS.

   Download the OS from https://github.com/OP-TEE/optee_os/releases/tag/2.3.0 to **$OPTEE_HOME** and extract it.

5. Optionally, install the OP-TEE test code, which provides the TAs used with the demo.

   Download the test code from https://github.com/OP-TEE/optee_test/releases/tag/2.3.0 to **$OPTEE_HOME** and extract it.

6. Change directory to **$OPTEE_HOME/optee_os-2.3.0**.
7. Build the OP-TEE OS.

   You must set build macros to enable OP-TEE to respond to request from a secondary core, to identify the entrypoint for VxWorks, to identify the address of the DTB (if it is separate from the VxWorks image), and to specify the platform.

   For example, with the i.MX6 Sabre Lite, and for a VxWorks image with a DTB that is built-in (rather than built and loaded separately), use the following command:

```
$ make CROSS_COMPILE=$OPTEE_HOME/toolchains/gcc-linaro-arm-linux-gnueabihf-4.9-2014.08_linux/bi
n/arm-linux-gnueabihf- \
CFG_BOOT_SECONDARY_REQUEST=y \
CFG_NS_ENTRY_ADDR=0x10100000 \
PLATFORM=imx-mx6qsabrelite
```

   Or, if the DTB is separate from VxWorks, use this command:

```
$ make CROSS_COMPILE=$OPTEE_HOME/toolchains/gcc-linaro-arm-linux-gnueabihf-4.9-2014.08_linux/bi
n/arm-linux-gnueabihf- \
CFG_BOOT_SECONDARY_REQUEST=y \
CFG_NS_ENTRY_ADDR=0x10100000 \
CFG_DT_ADDR=  0x14000000 \
CFG_DT=y
PLATFORM=imx-mx6qsabrelite
```

   For more information about the build macros and their board-specific values, see OP-TEE OS Build Macros and Values on page 105.

   CAUTION: **The addresses used in the OS build must match the addresses you use with the boot loader when you load the binaries. For example, when OP-TEE exits to normal world, it prepares the execution context by setting the program counter (PC) to CFG_NS_ENTRY_ADDR. This value must therefore match the location into which VxWorks has been loaded.**

   The output from the build is under **$OPTEE_HOME/optee_os-2.3.0/out/arm-plat-***PLATFORM***/**. The OP-TEE OS image is in the **core** sub-directory, which includes the binary, ELF, dump and MAP files (**tee.bin**, **tee.elf**, **tee.dmp**, **tee.map**). The **export-ta_arm32** sub-directory is used for the trusted application build.

8. Create a pure binary from the OS ELF file.

   The default **tee.bin** file includes an image header. Use objcopy to produce a pure binary. For example:

```
$ cd $OPTEE_HOME/optee_os-2.3.0/out/arm-plat-imx/core
$ $OPTEE_HOME/toolchains/gcc-linaro-arm-linux-gnueabihf-4.9-2014.08_linux/bin/arm-linux-gnueabi
hf-objcopy -O binary tee.elf optee.bin
```

9. Optionally, build the trusted OP-TEE test TAs.

   For example:

```
$ cd $OPTEE_HOME/optee_test-2.3.0
$ make CROSS_COMPILE_TA=$OPTEE_HOME/toolchains/gcc-linaro-arm-linux-gnueabihf-4.9-2014.08_linux
/bin/arm-linux-gnueabihf- \
TA_DEV_KIT_DIR=$OPTEE_HOME/optee_os-2.3.0/out/arm-plat-imx/export-ta_arm32 \
ta
```

   The **optee_test-2.3.0** directory includes both the client side test suite (for Linux OS) and the trusted applications. The build target "ta" is used so that only the TAs are built. The output of the build is in **$OPTEE_HOME/optee_test-2.3.0/out/ta**, with a sub-directory for each TA. They are named with a UUID and a **.ta** suffix. For example, **cb3e5ba0-adf1-11e0-998b0002a5d5c51b.ta**.

3. Configuring VxWorks for OP-TEE

To use VxWorks with OP-TEE, configure the VSB project with the TEE client API. Configure the VIP with the required hardware, storage, and tool components.

Procedure

1. Create a VSB project based on your BSP (fsl_imx6 is used here), and optionally using the SMP processor mode option.
2. Select the **OPTEE** layer.

   This layer provides the GlobalPlatform TEE Client API implementation for communication with OP-TEE.

3. Build the VSB project.
4. Create a VIP project based on the VSB project.
5. If you want to run the demo interactively and see the output, add support for the kernel shell.

   You can do so by selecting **PROFILE_DEVELOPMENT** when you create the project, or by adding **INCLUDE_SHELL** to the configuration.

6. Add the board support component for your target.

   For example, **INCLUDE_FSL_IMX6Q_SABRELITE**.

7. Add the required OP-TEE support components.

INCLUDE_OPTEE_CLIENT_API	TEE client client API for OP-TEE.
INCLUDE_OPTEE_RPC_FS	RPC Service: File System
INCLUDE_OPTEE_RPC_LOAD_TA	RPC Service: Trusted Application Storage

8. As required, re-set configuration parameters for the OP-TEE root file system and application storage path.

The default settings assume that you will configure the system with the ROMFS file system for the application, and a RAM drive file system for OP-TEE.

OPTEE_TA_LOAD_PATH	Path for trusted application storage. The “Loading TA” service uses this path to search for TAs to be loaded. The default is "**/romfs/ta**".
OPTEE_FS_ROOT_PATH	The root path for the OP-TEE Remote File System. The default for the root of the file system is "**/ram0**".

9. Add a native file system to host the OP-TEE remote file system.

   You can, for example use a dosFs file system with a RAM disk.

INCLUDE_RAM_DISK	RAM disk support. Be sure that the **RAM_DISK_DEV_NAME** configuration parameter setting matches the **OPTEE_FS_ROOT_PATH** setting. It is used to host OP-TEE's Remote File System. The default is "**/ram0**"
INCLUDE_RAM_DISK_FORMAT_DOSFS	Formats RAM disk as dosFs.

10. Add file system support for storing the OP-TEE trusted applications.

    For example, you can use the ROMFS file system, as the “loading TA” requests make read-only access to the file system.

    1. Add **INCLUDE_ROMFS** and **INCLUDE_ROMFS_DRV**.
    2. Create a ROMFS file system project, subordinate to your VIP project.
    3. In ROMFS, create a **ta** subdirectory (so you have **/romfs/ta**).

    The **/romfs/ta** directory name must be the same as the one defined with the **OPTEE_TA_LOAD_PATH** parameter.

11. Add the OP-TEE trusted applications to the file system directory that you have created for them.

    If you want to use the demos, copy the *UUID* **.ta** files (for example, **cb3e5ba0-adf1-11e0-998b0002a5d5c51b.ta**) from:

    - **$OPTEE_HOME/optee_test-2.3.0/out/ta/crypt/**
    - **$OPTEE_HOME/optee_test-2.3.0/out/ta/storage/**

    to the VxWorks ROMFS directory **/romfs/ta**.

    The RPC service will look for these files in this directory (as defined with the VxWorks configuration parameter **OPTEE_TA_LOAD_PATH**).

12. Optionally, add **INCLUDE_OPTEE_DEMO** for the VxWorks side of the OP-TEE demo.
13. Select DTB usage.

    You have the choice of using a separate DTB or a built-in DTB. If you want to use a DTB that is built into VxWorks image, add **INCLUDE_STANDALONE_DTB** ("standalone" refers to the VxWorks image, because it includes the DTB).

    The boot methods are difference for each configuration (see Booting the System and Running the Demos on page 103).

14. Build the VIP.

4. Preparing the System for Booting

Create a boot loader and copy it to an SD card. Set up the TFTP server on the host. Copy the OP-TEE and VxWorks image files to the TFTP root directory.

Procedure

1. Create and install a U-Boot boot loader.

   For information in this regard, select the appropriate BSP supplement guide from the list of BSP documentation on the Wind River Support Network site.

2. Start a TFTP server on the host.
3. Copy the OP-TEE and VxWorks image files (including the DTB file, if you are using a separate one) to the TFTP server root directory.

optee.bin	The OP-TEE binary, which is in **$OPTEE_HOME/optee_os-2.3.0/out/arm-plat-***PLATFORM***/core.**
vxWorks.bin	The VxWorks image, which is in *vipProjDir***/default**.
*bspName***.dtb**	If you are using a DTB that is separate from the VxWorks image. Use the file in *vipProjDir***/default**.

5. Booting the System and Running the Demos

Boot the target, load the OP-TEE and VxWorks images, and start the OP-TEE OS and VxWorks. Optionally, run the OP-TEE demos.

About This Task

The demos can be started from the VxWorks kernel shell. Both make use of operations that take place in the OP-TEE secure world. One involves encryption and decryption operations, and the other involves creation and use of a file. The OP-TEE code for the demos is in the **crypt** and **storage** directories under **optee_2.3.0/optee_test-2.3.0/ta**. The VxWorks code is in the **crypt** and **storage** directories under *installDir***/vxworks-7/pkgs_v2/security/optee_client_api/demo**.

Before You Begin

To run the demos, you must have configured OP-TEE and VxWorks with the appropriate components, and installed the trusted applications (TAs) in ROMFS.

Procedure

1. Power on the board.
2. At the U-Boot prompt, download the OP-TEE image to the secure TEE RAM address.

Board	TEE RAM Address
NXP i.MX6 Sabre Lite	0x4e000000
NXP TWR-LS1021A-PB	0xBC000000
NXP TWR-LS1021A	0xBC000000
Xilinx Zynq 7000 ZC702	0x3E000000
Xilinx Zynq 7000 ZC706	0x3E000000

For example, for the i.MX6 Sabre Lite board:

```
=> tftp 0x4e000000 optee.bin
```

3. Download the VxWorks image.

   The load address must be the same as the one set with the **CFG_NS_ENTRY_ADDR** macro for the OP-TEE OS build (see Installing, Configuring, and Building OP-TEE on page 99).

   For example, again for the i.MX6 Sabre Lite board:

```
=> tftp 0x10100000 vxWorks.bin
```

4. If you are using a DTB that is separate from VxWorks, download the DTB file as well.

   The load address must be the same as the one set with the **CFG_DT_ADDR** macro for the OP-TEE OS build (see Installing, Configuring, and Building OP-TEE on page 99).

   For example:

```
=> tftp 0x14000000 imx6q_sabrelite.dtb
```

5. Boot the OP-TEE OS.

   For example, for the i.MX6 Sabre Lite board:

```
=> go 0x4e000000
```

   OP-TEE boots and then VxWorks.

6. From the VxWorks kernel shell, run the devs command to verify that the **/tee** device is installed.
7. Run the OP-TEE demos.

   From the VxWorks kernel shell, start the encryption and decryption demo as follows:

```
-> sp opteeCryptDemo
```

   The output lists the input and ouput of the encryption and decription operations, which take place in the secure world.

   Start the storage demo as follows:

```
-> sp opteeStorageDemo
```

The output describes the various file operations that are performed.

6. OP-TEE OS Build Macros and Values

These OP-TEE OS build macros are used to support VxWorks. The values that must be assigned to them are board-specific.

Macro Descriptions

CFG_BOOT_SECONDARY_REQUEST	Set to 'y' to enable OP-TEE's responding to requests from secondary cores. This is required for an SMP configuration.
CFG_NS_ENTRY_ADDR	Identifies the entry address of the normal world OS (VxWorks). The OS must be loaded at this address by the boot loader. Execution begins at the address when OP-TEE exits to the non-secure world state.
CFG_DT_ADDR	The device tree data (DTB) memory address. Use with a separate DTB, which is not part of the VxWorks image. The DTB must be loaded at this address with the boot loader.
PLATFORM	Identifies the hardware platform.

Macro Values for Hardware Platforms

Board	CFG_NS_ENTRY_ADDR(Load Address for VxWorks)	CFG_DT_ADDR (Load Address for a Separate DTB)	PLATFORM
NXP i.MX6 Sabre Lite	0x10100000	0x14000000	imx-mx6qsabrelite
NXP TWR-LS1021A-PB	0x80100000	0x84000000	ls-ls1021atwr
NXP TWR-LS1021A	0x80100000	0x84000000	ls-ls1021atwr
Xilinx Zynq 7000 ZC702	0x200000	0x4000000	zynq7k-zc702
Xilinx Zynq 7000 ZC706	0x200000	0x4000000	zynq7k-zc702

16. ACHILLES LEVEL II CERTIFICATION

1. VxWorks Achilles Level II Certification

VxWorks can be easily configured to pass Achilles Level II Certification testing.

Achilles certification is designed to demonstrate the robustness of a system from a network point of view. It is intended to show that under stress, the device (running VxWorks in this case) is still able to perform its duties. See https://www.ge.com/digital/sites/default/files/download_assets/Achilles-System-Certification-FAQ.pdf for more information.

For a list of Achilles certified products, see https://www.ge.com/digital/applications/achilles-communications-certified-products.

Wind River has performed successful VxWorks Achilles Level II Certification testing with an NXP (Freescale) i.MX6 SABRE Lite reference board. The processor was an ARM Cortex-A9 with Quad-Core running at 1.0 GHz. A serial connection of 115200 baud was used for host-target communication. The board was connected to the network with a 1GB RGMII Ethernet port (or similar port). The board was configured to boot from a TFTP server on the network. U-Boot was used to load the VxWorks image. The test was run at a vendor defined rate (VDR) of 0.1%, and with the link speed of 1 Gbit/s.

The VxWorks image was based on the VxWorks Core Profile, and configured as described in VxWorks Configuration for Achilles II Certification on page 106. The Achilles test suite is more rigorous than many others (such as the Nessus vulnerability scanner), so minor network stack tuning is required to pass it.

2. VxWorks Configuration for Achilles II Certification

Wind River used this configuration of the VxWorks 7 Core Profile, with NXP (Freescale) i.MX6 SABRE Lite hardware, to pass Achilles II Certification testing.

Network Driver Configuration

The *installDir* **/vxworks-7/pkgs_v2/net/end-***version* **/drv/src/end/vxbFdtFslImxEnd.h** file must be edited to change the value of the **ENET_TUPLE_CNT** macro from 384 to (384*16).

VSB Configuration

The VSB project is based on the fsl_imx6 BSP, with the default configuration.

VIP Configuration

The VIP project must be based on the VSB project described above. The **PROFILE_DEVELOPMENT** must be selected during project creation, and then the components listed below must be added.

Use the default configuration parameters for all components.

Component	Description
INCLUDE_IPIFCONFIG_CMD	IPCOM ifconfig commands.
INCLUDE_IPECHO_SERVER_CMD	IPCOM echoserver commands.

Component	Description
INCLUDE_IPFTPS	FTP Server.
INCLUDE_IPSNTPS	SNTP Server daemon.
INCLUDE_SPY	Spy utility, which measures per-task CPU utilization.
DRV_FSL_I2C	Freescale I2C bus controller vxBus driver.
DRV_GPIO_FDT_FSL_IMX	GPIO driver for Freescale i.MX GPIO devices.
INCLUDE_GPIO_SYS	VxBus GPIO subsystem.

3. Set Up System and Run Test

Boot the system, set up the network interfaces, and run the Achilles test.

About This Task

Procedure

1. Boot the target.
2. Configure the network interfaces for testing.

   Use ifconfig to configure the interfaces for testing. Each interface must be tested independently. For example, using the kernel shell (with the command interpreter):

   ```
   [vxWorks *]# ifconfig enet0 down
   [vxWorks *]# ifconfig enet1 172.16.0.2 up
   [vxWorks *]# ifconfig enet2 down
   [vxWorks *]# ifconfig enet3 down
   ```

3. Start the Achilles test software, and run the scan.

   For information about running the Achilles software, see the Wurldtech documentation.

4. After the scan is finished, start the TCP echo server.

   For example:

   ```
   [vxWorks *]# echoserver -t tcp -p 7 172.16.0.2
   ```

5. Run the Achilles tests for the configured interface.

4. Wind River's Achilles II Testing for VxWorks 7

This setup and test process was used by Wind River to achieve Achilles II certification for VxWorks 7.

For Achilles Certification Level 2 - Embedded Device, the test results were determined by the following Achilles Satellite monitors:

- Discrete Monitor
- Analog Monitor
- Link State Monitor
- ICMP Monitor
- TCP Ports Monitor
- UDP Ports Monitor

Software

The software environment was VxWorks 7.

Hardware

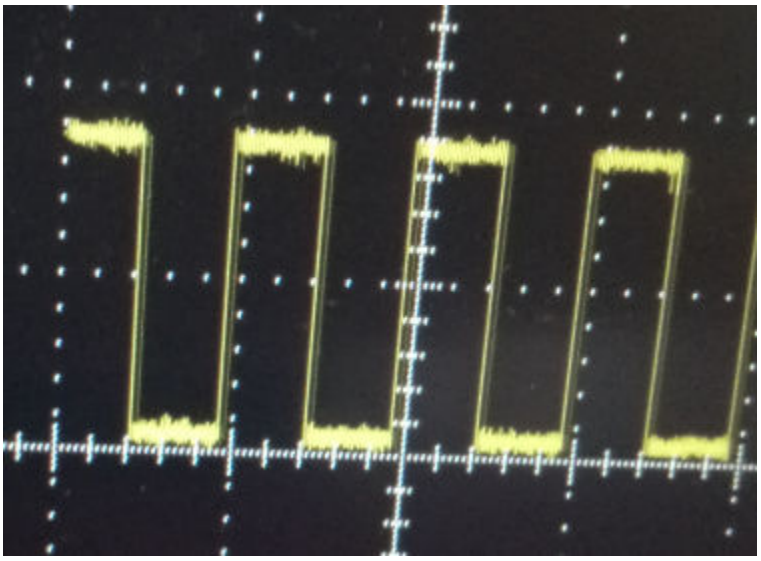
The hardware environment consisted of the following components:

- Achilles Test Platform
- Freescale i.MX6q SABRE Lite board
- MCP4725 i2c chip
- Voltage conversion I/O board
- Analog input module:
    - USB-AI12-16E
    - STB-68
    - C68PS18L

Digital Output Requirements on page 108
Analog Output Requirements on page 111
Test Procedure on page 112 Wind River used this procedure for certification of VxWorks 7.
Troubleshooting for the Achilles Certification Tests on page 113 These troubleshooting issues were encountered during testing.

4.1. Digital Output Requirements

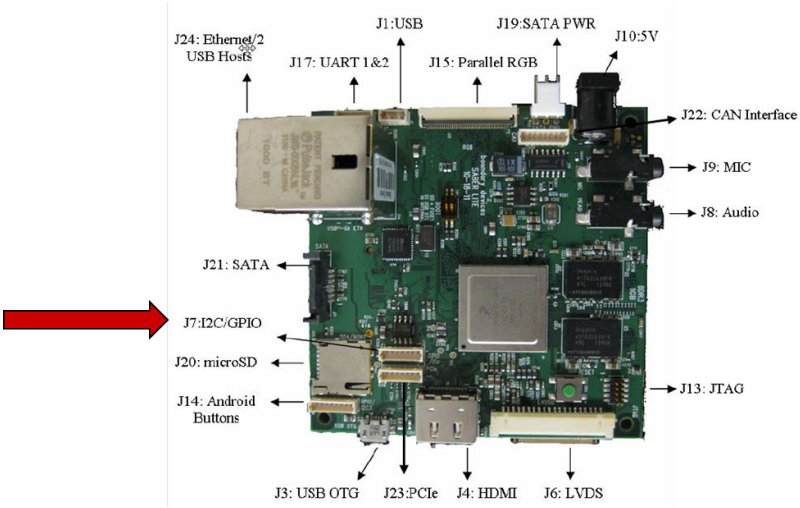
The device under test (DUT) was programmed to produce a square wave output from its discrete output module. The square wave consisted of an *on* period, during which the output was high, followed by an *off* period, during which the output was low. The cycle lasted one second (500 msec high and 500 msec low) and repeated indefinitely.



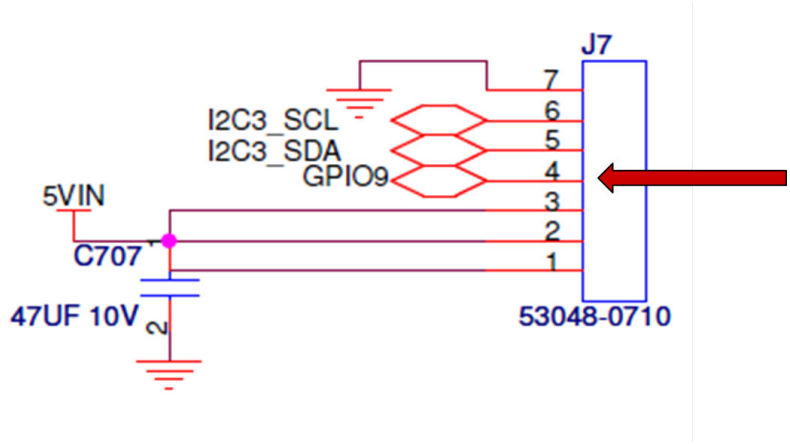
J7 and GPIO9 Usage

Discrete Monitor A was used to monitor the digital output. The GPIO9 on the Freescale i.MX6q SABRE Lite was used to output the digital signal. The GPIO9 (7 pins) is part of the J7 module.

The red arrow in the image below indicates the J7.

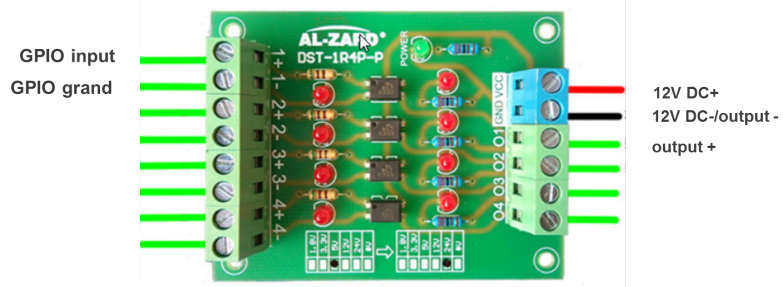


The GPIO9 communicates via the fourth of 7 pins on the J7 module.

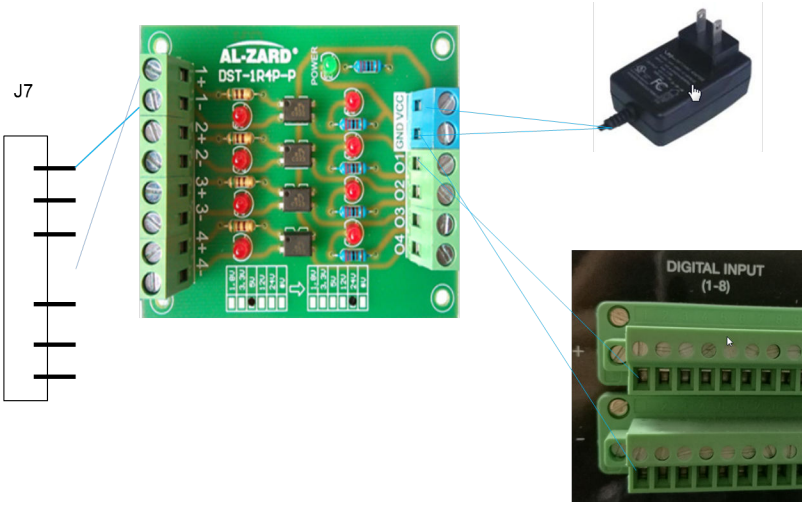


Voltage I/O Conversion

A conversion I/O board was added to raise the voltage from 3.3v to 12V (if the Achilles box is version r3, 3.3 volts is adequate and this conversion is not needed).



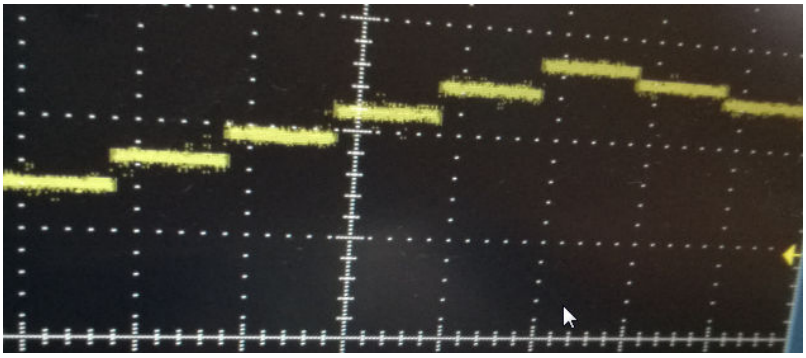
Digital Component Relationships



Parent topic: Wind River's Achilles II Testing for VxWorks 7 on page 108

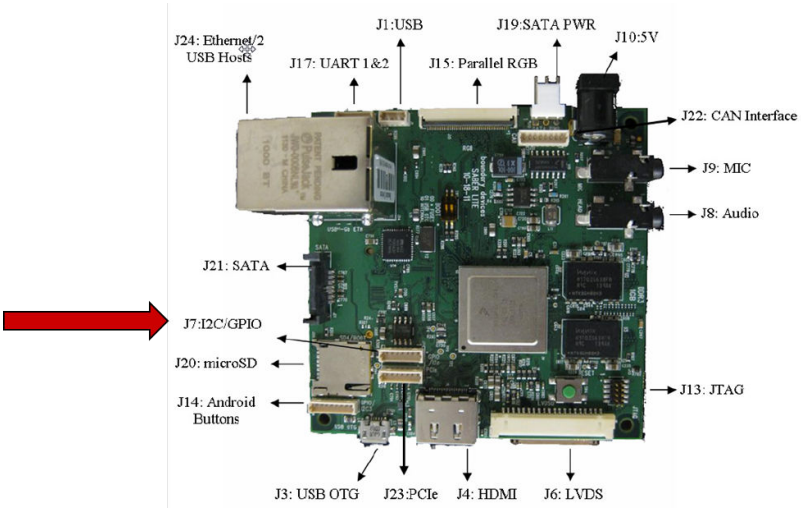
4.2. Analog Output Requirements

The device under test (DUT) was programmed to produce an analog waveform that consisted of 10 equal steps of increasing value and then 20 equal steps of decreasing value, both at one step per second, transitioning between the nominal minimum and maximum values of the output device.

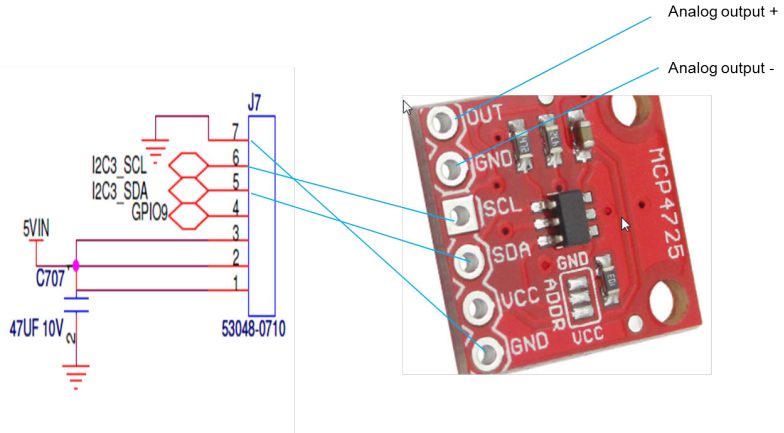


MCP4725 Usage

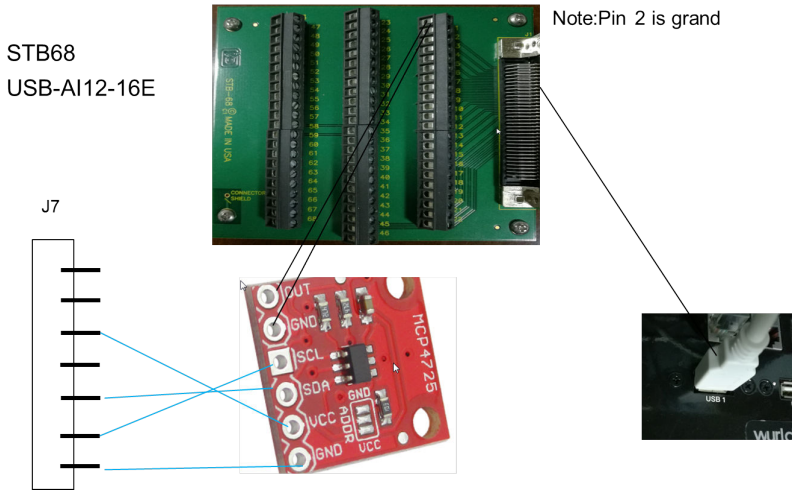
An analog monitor was used to monitor the analog output. Since the i.MX6q SABRE Lite does not have an analog chip, an MCP4725 was added to the board. The I2C is output by way of the J7 module.



Pins 3, 5, 6, and 7 of the J7 were used to connect the MCP 4725.



Analog Component Relationships



Parent topic: Wind River's Achilles II Testing for VxWorks 7 on page 108

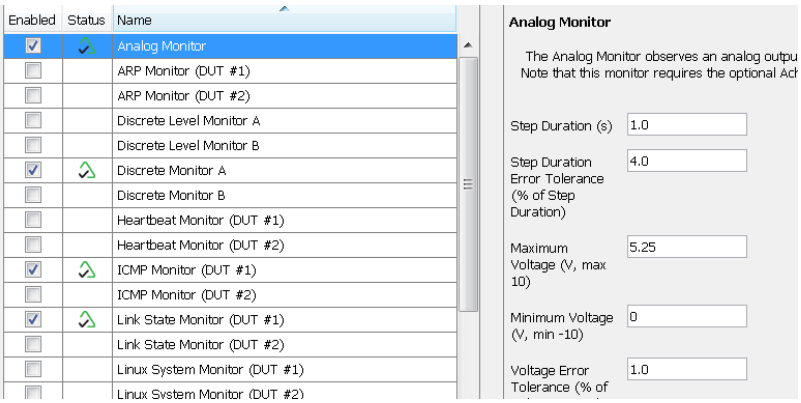
4.3. Test Procedure

Wind River used this procedure for certification of VxWorks 7.

1. The i.MX6q SABRE Lite was booted, and the IP address of **enet0** set to 172.16.0.2.
2. The echoserver was started as follows:

   ```
   [vxWorks] # echoserver -t tcp -p 7 172.16.0.2
   ```

3. The test case was loaded as follows:
   Achilles client > File > Restore test suite > Choose Level2 > Restore
4. As a prerequisite to testing, the voltage of the analog monitor was set to a minimum value of 0 and a maximum value of 5.25:
   Achilles client > environment > monitor > Analog Monitor



5. The monitors were enabled and their operation observed to ensure that they were operating normally.
6. It was determined that the server was reachable, by performing a TCP/UDP scan, as follows:
   Achilles client > Discovery > Start
   The echoserver was then cleared of the scan results.
7. The test case was run, as follows:
   Achille client > test execute > execute all.
8. The test report was produced, as follows:
   Achille client > History > choose item > save report .
9. The results were confirmed according to the following rules:
   - It was confirmed that the Discrete Monitor, Analog Monitor, and Link State Monitor were Normal for all tests below, at, and above the VDR.
   - It was confirmed that the ICMP Monitor, TCP Ports Monitor, and UDP Ports Monitor were Normal for tests run at or below the VDR.
     Warnings were permitted depending on the cause and severity of the behavior. Status could be Normal or Warning for storms run above the VDR.

Parent topic: Wind River's Achilles II Testing for VxWorks 7 on page 108

4.4. Troubleshooting for the Achilles Certification Tests

These troubleshooting issues were encountered during testing.

Issue	Potential Cause
Link Monitor Warning.	Check the cable connected to port1 and the imx6 interface.
TCP/UDP/ICMP warning before run.	• Check the IP address of the imx6 interface. • Check for information in: **Achilles Client > environment > peer**
UDP/TCP server scan failed.	Ensure that the FTP server /SNTP server component is included in the VIP project.

Issue	Potential Cause
Analog monitor failed before test run.	Measure the OUT voltage of the MCP 4725 and adjust the voltage setting of the analog monitor.
Analog monitor failed in some test cases	Unknown.

Parent topic: Wind River's Achilles II Testing for VxWorks 7 on page 108

5. Wind River's Achilles Level II Certificate of Compliance

Certificate Of Compliance

This is to certify that the:

VxWorks 7 RTOS

Software Version: 20161124091431

Platform: i.MX6 SabreLite

Category: Embedded Device

*This certificate applies to a particular configuration of VxWorks 7 RTOS with the specified hardware platform and custom applications.

Manufactured by:

Wind River

Is in compliance with the requirements set forth by:

Achilles Level II Certification

This certificate is restricted to the specified version of the referenced Device (including model number, hardware/firmware/software version, and control protocols) set forth in this Certificate. Any change to the Device might impair the Device's ability to pass the standards referenced in this Certificate. This Certificate is based upon limited testing, and is not a complete or comprehensive certification of the security of the Device. The Device may contain security vulnerabilities and deficiencies that are not covered by the standards referenced in this Certificate. This Certificate is subject to revocation by Wurldtech Security Technologies Inc. More information regarding this Certificate and the Achilles Communications Certification Standard is available at www.wurldtech.com and upon request from Wurldtech Security Technologies Inc.

CERTIFICATE 338-200117

Tom Mueller
Vice President
Product Management

wurldtech
A GE Company

17. GROUP DOMAIN OF INTERPRETATION—GDOI

1. VxWorks GDOI

The Group Domain of Interpretation (GDOI) is a cryptographic protocol for group key management.

The VxWorks implementation of GDOI is based on RFC 6407 and IEC62351. It supports both a client and a simple, statically-configured server. The GDOI protocol is implemented as a new DOI for IKE, which supports many clients with one server, instead of IKE's peer-peer model. GDOI specifies an IKEv1 main-mode exchange; it is not compatible with IKEv2.

For detailed information about the IKE, see the VxWorks 7 IKE Programmer's Guide.

2. Configuring VxWorks with GDOI

To configure VxWorks with GDOI, you must select the required VSB and VIP options, set VIP configuration parameters, and create a custom IKE configuration file.

Procedure

1. Create a VSB project.
2. Add the **IPNET_IPSECIKE** layer.
3. Build the VSB project.
4. Create a VIP project based on the VSB project.
5. Add the **INCLUDE_INTERNET_KEY_EXCHANGE** component.
6. Set the **SEC_VAULT_KEY_ENCRYPTING_PW** configuration parameter.

   This password is used to encrypt keys in the vault and key store. It should be set to a random string of at least 40 characters, using both lower and upper case, numbers and special characters. There is no default value. The project cannot be built unless it is set.

7. Build VxWorks.
8. Create a custom IKE configuration file **ike.cfg** (which you will download to the target at runtime).

   For general information about the IKE configuration file, see the *VxWorks 7 IKE Programmer's Guide: IKE Configuration File*.

3. IKE Configuration File Sections for GDOI

Implementation of GDOI requires a customized IKE configuration file that includes GDOI-specific sections.

IKE Configuration File Sections

An IKE configuration file for GDOI consists of three sections:

- **general**
- **ike_sa**
- **gdoi_group** or **gdoi_gcks**

WNDRVR

The **general** and **ike_sa** sections are similar to those used for a generic IKE implementation. The **gdoi_group** and **gdoi_gcks** sections are required for GDOI.

For detailed information about the IKE configuration file, see the VxWorks 7 IKE Programmer's Guide.

general Section

The **general** section included parameters that configure the IKE task, including the IP address of the interface at which the IKE task listens for incoming ISAKMP messages. The following simple example specifies one address, and implicitly uses the defaults for the other parameters:

```
general {
  listen 172.25.38.142;
}
```

ike_sa Section

The **ike_sa** section contains parameters for the negotiation of IKE security associations (SAs); that is, the entities with which the IKE daemon will talk.

A group member (GM) typically only needs an **ike_sa** entry for the Group Controller/Key Server (GCKS).

A GCKS needs an **ike_sa** entry for the GMs, each of which is referenced later in the group configuration.

For example:

```
ike_sa address 172.25.38.143 {
  ike_id default143;
  local_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vie
nto IKE responder;
  remote_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vi
ento IKE initiator;

  authentication {
    method rsa_signature;
    certificate /romfs/ike_rsa_resp.cer;
    private_key /romfs/ike_rsa_resp.key;
    private_key_password 1111;
    trusted_ca_store /romfs/trusted;
#    intermediate_ca_store /tgtsvr/intermediate;
  }

  proposal {
    dh_group 2,1;
    hash_algorithm sha1,md5;
    encryption_algorithm 3des,des;
    integrity_algorithm sha1,md5;
  }
}
```

Note: For GDOI, the **method** parameter must be assigned the **rsa_signature** option.

For information about the full set of **ike_sa** parameters, see the VxWorks 7 IKE Programmer's Guide.

WNDRVR

gdoi_group Section—for GM Only

This section is used for additional parameters for a GM. For example:

```
gdoi_group {
  ike_id default;
  kda    yes;
  push_mode yes;
}
```

The parameters are as follows:

ike_id

Links the **gdoi_group** section with its **ike_sa** section.

kda

If set to **yes**, SAs without 100% KDA will not be installed in the Security Associations Database (SADB).

push_mode

Set to **yes** to listen for PUSH updates. Omit or set to **no** not listen.

gdoi_gcks Section—for GCKS Only

This section defines the groups that the GCKS supports. It is composed of three subsections:

- **group_member**
- **data_sa**
- **key_sa**

For example:

```
group_member {
     ike_id default144;
     data_sa sa1,sa2;
     required yes;
     push_mode yes;
     key_sa sa3;
  }
```

The **group_member** subsection has the following parameters:

ike_id

Associates this **group_member** section with its **ike_sa** section.

data_sa

Specifies which traffic encryption key (TEK) SA parameters are allowed for this member.

required

Specifies whether or not this is a required member of the group, for KDA purposes. Allowed values are **yes** or **no**.

WNDRVR

push_mode

Enables PUSH messages to this GM when set to **yes**. Allowed values are **yes** or **no**.

key_sa

Specifies which KEK SA parameters are used with this GM.

Each **data_sa** subsection specifies the parameters for a specific TEK set. For example:

```
data_sa sa1 {
     id_type tunnel;
     address 233.12.1.1;
     encryption_algorithm aes128cbc;
     integrity_algorithm sha2hmac;
     lifetime 60;
     atd 40;
  }
```

The parameters are as follows:

id_type

OID type: **goose**, **tunnel** or **sv**.

address

Internal address.

encryption_algorithm

Either **aes128cbc** or **aes256cbc**.

integrity_algorithm

Either **sha2hmac** or **sha2hmac128**

lifetime

Key lifetime, in seconds. Must be at least 60 seconds.

atd

Activation time delay (ATD), in seconds. Must be between the values of **lifetime**/2 and **lifetime**.

ds_ref

Dataset reference, which is used to look up this SA.

The **key_sa** subsection specifies the attributes of the KEK. For example:

```
key_sa sa3 {
    encryption_algorithm aes128cbc;
    integrity_algorithm sha256;
    lifetime 65;
  }
```

The parameters are:

WNDRVR

encryption_algorithm

Either **aes128cbc** or **aes256cbc**.

integrity_algorithm

Either **sha2hmac** or **sha2hmac128**.

lifetime

Key lifetime, in seconds.

4. Example IKE Configuration File for GDOI Group Member

This file is an example of an IKE configuration file for a GM.

```
#####################################################
# gm_144.cfg - IKE Initiator configuration file example for GM
# 172.25.38.144 = GM IP address
# 172.25.38.142 = GCKS IP address
#####################################################

general {
  listen 172.25.38.144;
}

ike_sa address 172.25.38.142 {
  ike_id default;
  local_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vie
nto IKE initiator;
  remote_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vi
ento IKE responder;

  authentication {
    method rsa_signature;
    certificate /romfs/ike_rsa_init.cer;
    private_key /romfs/ike_rsa_init.key;
    private_key_password 1111;
    trusted_ca_store /romfs/trusted;
  # intermediate_ca_store /tgtsvr/intermediate;
  }

  proposal {
    dh_group 2,1;
    hash_algorithm sha1,md5;
    encryption_algorithm 3des,des;
    integrity_algorithm sha1,md5;
  }
}

gdoi_group {
  ike_id default;
    kda yes;
push_mode yes;
}
```

```
#######################################################
# END OF FILE
#######################################################
```

5. Example IKE Configuration File for GDOI Group Controller/Key Server

This file is an example of an IKE configuration file for a GCKS.

```
#######################################################
# gcks.cfg - IKE Initiator configuration file example for host 2 (responder)
# 172.25.38.142 = GCKS IP address
# 172.25.38.143, 144 = GM IP addresses
#######################################################

general {
listen 172.25.38.142;
}

ike_sa address 172.25.38.143 {
  ike_id default143;
  local_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vie
nto IKE responder;
  remote_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vi
ento IKE initiator;

  authentication {
    method rsa_signature;
    certificate /romfs/ike_rsa_resp.cer;
    private_key /romfs/ike_rsa_resp.key;
    private_key_password 1111;
    trusted_ca_store /romfs/trusted;
    # intermediate_ca_store /tgtsvr/intermediate;
  }

  proposal {
    dh_group 2,1;
    hash_algorithm sha1,md5;
    encryption_algorithm 3des,des;
    integrity_algorithm sha1,md5;
  }
}

ike_sa address 172.25.38.144 {
  ike_id default144;
  local_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vie
nto IKE responder;
  remote_identifier asn1_dn /C=SE/ST=Oaxaca/L=Oaxaca/O=Rio del Viento/OU=Development/CN=Rio del Vi
ento IKE initiator;

  authentication {
    method rsa_signature;
    certificate /romfs/ike_rsa_resp.cer;
    private_key /romfs/ike_rsa_resp.key;
    private_key_password 1111;
```

```
    trusted_ca_store /romfs/trusted;
    # intermediate_ca_store /tgtsvr/intermediate;
  }

  proposal {
    dh_group 2,1;
    hash_algorithm sha1,md5;
    encryption_algorithm 3des,des;
    integrity_algorithm sha1,md5;
  }
}


gdoi_gcks {
    group_member {
      ike_id default143;
      data_sa sa1,sa2;
      required yes;
      push_mode yes;
      # push_mcast_ip 233.1.2.3;
      key_sa sa3;
      }

    group_member {
      ike_id default144;
      data_sa sa1,sa2;
      required yes;
      push_mode yes;
      # push_mcast_ip 233.1.2.3;
      key_sa sa3;
      }

    group_member {
      ike_id default2;
      data_sa sa1, sa2;
      }

    data_sa sa1 {
      id_type tunnel;
      address 233.12.1.1;
      encryption_algorithm aes128cbc;
      integrity_algorithm sha2hmac;
      lifetime 60;
      atd 40;
      }

    data_sa sa2 {
      id_type goose;
      address 233.12.1.2;
      ds_ref gigi;
      encryption_algorithm aes128cbc;
      integrity_algorithm sha2hmac;
      lifetime 60;
      atd 40;
      }

    key_sa sa3 {
```

```
      encryption_algorithm aes128cbc;
      integrity_algorithm sha256;
      lifetime 300;
      }
}
#######################################################
# END OF FILE
#######################################################
```

6. VxWorks GDOI APIs

VxWorks provides a set of APIs for working with GDOI.

GDOI APIs

Function	Description
AddAtdCb_GDOI( )	Attach a function to the ATD callback.
AddPushCb_GDOI( )	Attach a function to the PUSH callback.
AddDeleteCb_GDOI( )	Attach a function to the DELETE callback.
AddExpireCb_GDOI( )	Attach a function to the EXPIRE callback.
AuthenticateReq_GDOI( )	Initiate IKE phase 1 authentication using configured certificates.
PullReq_GDOI( )	Initiate a GDOI PULL, retrieving the current set of SAs from the server.
PullReq_GDOI_2( )	Initiate a GDOI PULL, retrieving the current set of SAs from the server (using different parameters).

For more information, see the **ipike_gdio** API reference entry.

18. SIMPLE CERTIFICATE ENROLLMENT PROTOCOL—SCEP

1. VxWorks SCEP

The Simple Certificate Enrollment Protocol (SCEP) is designed for simple and scalable management of digital certificates.

SCEP is an Internet Draft in the Internet Engineering Task Force (IETF). It allows for the distribution of CA public keys and for Certificate Enrollment (obtaining a certificate).

For VxWorks, only the SCEP client is implemented. Clients send unsigned certificates to a SCEP server, which authenticates the request, signs them and then sends them back. They can also obtain a copy of the certification-authority (CA) certificate. To prevent this from being a security hole, a loose start to a chain of trust, the obtained CA certificates are verified against a pre-configured fingerprint

2. Configuring VxWorks with SCEP

To configure VxWorks with SCEP, add the **SECURITY_MISC** layer to the VSB project, and the **INCLUDE_SCEP** component to the VIP project.

Procedure

1. Create a VSB project.
2. Add the **SCEP** layer.
3. Build the VSB project.
4. Create a VIP project based on the VSB project.
5. Add the **INCLUDE_SCEP** component.
6. Build VxWorks.

3. VxWorks SCEP Server APIs

VxWorks provides several functions for working with a SCEP server.

SCEP Server APIs

Functions	Description
ipscep_get_cert( )	Get the client identification certificate from the SCEP server.
ipscep_get_cert_2( )	Get the client identification certificate from the SCEP server (using different parameters).
ipscep_get_ca_cert( )	Get the CA certificates from the SCEP server.
ipscep_rsa_generate_keys( )	Generate RSA keys.

WNDRVR

For more information, see the **ipscep** library reference entry.